



Silesian
University
of Technology

MASTER THESIS

Applying AI Algorithms to Train a Computer Player in a Logic Board Game

Jagoda Barbara WIĘCEK

Student identification number: jw300503

Programme: Control, Electronic and Information Engineering

Specialisation: Data Science

SUPERVISOR

Dr Ewa Lach

DEPARTMENT of Graphics, Computer Vision and Digital Systems

Faculty of Automatic Control, Electronics and Computer Science

Gliwice 2026

Thesis title

Applying AI Algorithms to Train a Computer Player in a Logic Board Game

Abstract

The objective of this thesis was to develop and evaluate a reinforcement-learning approach for training a computer-controlled player in the logic-based board game Rummikub. Due to the large state and action spaces and the presence of sparse and delayed rewards, a Curriculum Learning environment was designed to introduce selected game mechanics through tasks of gradually increasing difficulty. The environment and training framework were implemented using Unity and the ML-Agents Toolkit.

The experimental study compared two configurations: Proximal Policy Optimization (PPO) as a baseline and PPO supported by Generative Adversarial Imitation Learning. Ten independent agents were trained for each configuration. Their performance was assessed using curriculum progression, the number of environment steps, wall-clock training time, and the win rate obtained over 1,000 evaluation episodes per agent.

Key words

Reinforcement learning, imitation learning, Curriculum Learning, PPO, GAIL, Unity ML-Agents, Rummikub

Tytuł pracy

Zastosowanie algorytmów sztucznej inteligencji do nauki komputerowego gracza w logicznej grze planszowej

Streszczenie

Celem niniejszej pracy było opracowanie i ocena podejścia opartego na uczeniu się przez wzmocnienie, służącego do szkolenia komputerowego gracza w opartej na logice grze planszowej Rummikub. Ze względu na rozległe przestrzenie stanów i działań oraz występowanie rzadkich i opóźnionych nagród zaprojektowano środowisko uczenia się oparte na programie nauczania (Curriculum Learning), mające na celu wprowadzanie wybranych mechanizmów gry poprzez zadania o stopniowo rosnącym stopniu trudności. Środowisko oraz framework szkoleniowy zaimplementowano przy użyciu silnika Unity oraz zestawu narzędzi ML-Agents Toolkit.

W badaniu eksperymentalnym porównano dwie konfiguracje, Proximal Policy Optimization (PPO) jako punkt odniesienia oraz PPO wspomagane metodą Generative Adversarial Imitation Learning (GAIL). Dla każdej konfiguracji wyszkolono dziesięć niezależnych agentów. Ich wyniki oceniono na podstawie postępów w programie nauczania, liczby

kroków w środowisku, czasu szkolenia w czasie rzeczywistym oraz wskaźnika zwycięstw uzyskanego w 1 000 epizodów ewaluacyjnych na agenta.

Słowa kluczowe

Uczenie ze wzmocnieniem, uczenie przez naśladowanie, Curriculum Learning, PPO, GAIL, Unity ML-Agents, Rummikub

Contents

1	Introduction	1
1.1	Introduction into the problem domain	1
1.2	Settling of the problem in the domain	3
1.3	Objective of the thesis	3
1.4	Scope of the thesis	4
1.5	Short description of chapters	5
1.6	Description of contribution of the thesis's author	5
2	Analysis of the problem	7
2.1	Problem statement	7
2.2	State of the art	7
2.2.1	An Overview of NPC Decision-Making Techniques	8
2.2.2	Deep Learning	11
2.2.3	Reinforcement Learning	21
2.2.4	The Sparse Reward Problem	31
2.3	Existing solutions	36
2.3.1	Applying ML Algorithms to Video Game AI	36
2.3.2	Anticheat System Based on Reinforcement Learning Agents in Unity	37
2.3.3	COPPER algorithm in card game	38
2.3.4	A general reinforcement learning algorithm using self-play	40
2.4	Summary	41
3	Research Environment	45
3.1	Game environment	46
3.1.1	Characteristics of the problem	46
3.1.2	Game Rules	46
3.1.3	Game implementation	47
3.1.4	Game Mechanics and User Interactions in the Game Environment .	48
3.2	Agent training	49
3.2.1	Behavior parameters and agent setup	50
3.2.2	Configurations and hyperparameters	52

3.2.3	Observation representation	57
3.2.4	Actions interpretation and masking	58
3.2.5	Reward Function	59
3.2.6	Curriculum Learning Implementation	60
3.2.7	Demonstration recording	62
3.3	Communication Between the Game and Training Process	63
3.3.1	Architecture of the ML-Agents Framework	63
3.3.2	Training Data Flow	64
3.4	Evaluation Metrics	65
3.4.1	Evaluation process	66
3.5	Using the Research Environment	67
4	Experiments	69
4.1	Methodology	69
4.1.1	Research Framework	69
4.1.2	Training methods	70
4.1.3	Evaluation methods	72
4.2	Results	72
4.2.1	Baseline	73
4.2.2	GAIL as auxiliary method	82
4.2.3	Cross-model comparison	92
4.3	Conclusions	95
5	Summary	99
5.1	Conclusions	99
5.2	Future development	100
	Bibliography	104
	List of abbreviations and symbols	107
	List of additional files in electronic submission	109
	List of figures	111
	List of tables	113

Chapter 1

Introduction

Board games have entertained people since ancient times, from teaching techniques for the youngest, to providing entertainment for entire families, to competitions for the best players. Nowadays, many popular board games have been introduced into the digital world to bring people together from different corners of the globe. The digital board game environment has also made it possible to play games without the participation of physical people on the other side of the screen. The implemented players were tasked with behaving like human participants.

This thesis focuses on applying AI algorithms to train a computer player for a logic-based board game. The selected game introduces distinct challenges, including combinatorial complexity, dynamic board states, randomness and the necessity for both tactical and strategic reasoning. Designing an AI capable of playing effectively in such an environment requires balancing rule-based knowledge with adaptive learning techniques

1.1 Introduction into the problem domain

The history of board games spans thousands of years, reflecting their enduring role in human culture. The Royal Game of Ur, originating in ancient Mesopotamia more than 4,500 years ago, is considered one of the earliest known board games [3]. Since then, board games have remained a popular pastime across civilizations, valued not only as a form of entertainment but also as a means of social interaction and intellectual challenge.

Beyond recreation, board games have also been recognized for their pedagogical value. Recent studies highlight their efficacy as educational tools, showing that board games can enhance motivation, engagement, and learning outcomes in classroom settings [12]. By providing structured environments that require logical thinking, planning, and decision-making, board games naturally support the development of cognitive and social skills.

Board games have long been used as a way to test logical thinking and strategy. Unlike purely abstract games such as chess, many modern tactical board games introduce an element of randomness, such as shuffled decks, dice rolls, or random tile draws. This

combination of determinism and uncertainty contributes to a more complex decision-making process. Players are required to plan tactics based on the current position, whilst also having the capacity to adapt to unpredictable events.

The dual role of board games as both an entertainment medium and an educational tool renders them an ideal domain for exploring artificial intelligence algorithms. These games combine structured rule systems with varying levels of complexity, randomness, and strategic depth. Consequently, the training of a computer player for such games contributes not only to the field of game AI but also offers broader insights into decision-making processes, learning under constraints, and adaptive strategies in uncertain environments.

In the domain of AI and ML, video games and board games have historically served as premier experimental testbeds. The fundamental reason for this is that games are engineered to accurately capture complex cognitive requirements, such as long-term planning, spatial reasoning, resource management, and strategic thought, in an easily evaluable and mathematically bounded format [7]. Because video games inherently feature clear objectives, explicit rule sets, and quantifiable progress metrics, they provide an idealized, noise-reduced abstraction of real-world decision-making challenges. In addition to functioning as conceptual exemplars, these digital environments also offer critical pragmatic advantages for the training of contemporary algorithms.

Modern AI is notoriously sample-inefficient, requiring millions of interactions to converge on an optimal policy. Deploying untrained agents in real-world physical environments, such as autonomous Unmanned Aerial Vehicles or industrial robotics, presents severe safety risks and enormous hardware costs, making massive data collection impractical. Video games and board games natively solve this bottleneck by functioning as high-fidelity digital simulators. They provide absolute safety, allowing an agent to fail, crash, or make catastrophic decisions without incurring any real-world damage or financial penalty. Furthermore, game engines can be decoupled from physical time constraints, meaning simulations can be executed significantly faster than real-time. This accelerated execution allows an AI agent to accumulate the equivalent of thousands of hours of experiential data in mere days, satisfying the massive data requirements of deep learning architectures.

Board games are widely used in research to evaluate and develop artificial intelligence methods. Their clearly defined rules, discrete states, and measurable objectives make them suitable environments for studying decision-making algorithms and learning-based approaches. Unlike purely deterministic problems, many board games require an agent to analyze possible actions, estimate their outcomes, and adapt its strategy based on the current state of the game.

Logic board games represent a specialized category of board games focused on problem-solving, planning, and reasoning. They typically contain a limited set of rules while creating complex combinations of possible states and actions. This combination of simple game

mechanics and complex decision-making makes them valuable environments for investigating artificial intelligence methods, especially approaches related to search algorithms and reinforcement learning.

1.2 Settling of the problem in the domain

Many popular board games that use artificial intelligence algorithms only required learning the rules and adapting tactics to the opponent. One such game is chess, which has many artificial intelligence algorithms implemented, one of which is **Stockfish - Open Source Chess Engine** [5]. However, in numerous games, players are required to adjust their tactics not only to their opponents' actions, but also to environmental and situational randomness. Rummikub exemplifies this, requiring players to adapt their strategy to other players, the random draw of tiles, and the specific rules for laying down sets.

In Rummikub, players create and rearrange sets and sequences of numbered tiles in different colors. The objective of the game is to place all tiles from one's hand onto the shared board by forming valid combinations according to the game's rules. However, in contrast to purely deterministic games, Rummikub introduces stochastic elements, such as random tile draws or randomly generated tiles at the beginning of the game. These elements have been shown to significantly affect the course of the game. This randomness forces both human and artificial players to adapt their tactics dynamically to ever-changing conditions on the board.

From an AI perspective, this environment poses several challenges. As the board evolves, the number of potential moves increases, rendering an exhaustive search impractical. The uncertainty of future tile draws and opponent actions introduce an additional layer of complexity that requires probabilistic reasoning or learning-based adaptation rather than deterministic planning alone.

Positioning this problem within the broader field of game AI highlights its research value. Developing an effective computer player for Rummikub requires combining strategic evaluation, pattern recognition, and adaptive learning. It also provides an opportunity to investigate how modern AI algorithms, such as reinforcement learning and simulation-based decision-making, can operate effectively in environments that are logical in structure but influenced by randomness.

1.3 Objective of the thesis

The main objective of this thesis is to analyse, evaluate, and experimentally verify whether selected artificial intelligence algorithms can effectively support decision-making of a computer-controlled player in the board game Rummikub. The developed AI system

should be able to analyze the current game state, evaluate available legal moves, and select actions that maximize its probability of success in a random environment.

Due to the stochastic nature of Rummikub, particularly the randomness associated with tile distribution and draws, the computer player must be able to operate in a dynamic and partially unpredictable environment. Therefore, a key objective of this work is to implement AI algorithms and their parameters that can adapt their strategy to changing board configurations, different hand compositions, and evolving tactical opportunities throughout the game.

1.4 Scope of the thesis

This thesis focuses on the application of Reinforced Learning algorithms to train a computer player for the board game Rummikub within a custom-developed digital environment. The scope of the work includes the design and implementation of the game logic, the integration of Non-Player Character (NPC), and the experimental evaluation of the trained computer player. The project is limited to a single board game, Rummikub, and does not attempt to generalize the developed solution to other game genres. The research excludes online multiplayer deployment and human-subject studies. The implementation is carried out using the Unity engine, where the game environment and computer players are developed and tested.

The thesis maintains focus on the core research problem: the practical application of AI algorithms to enable intelligent decision-making in a tactical board game environment characterized by combinatorial complexity and stochastic elements. The development of the project include:

- Implementation of game environment for the player - development of a digital representation of the Rummikub board game using the Unity engine, including tile management, valid set detection, and interaction rules.
- Integrating artificial intelligence algorithm - implementation of the different reinforcement learning algorithms available from Unity ML-Agents Toolkit.
- Developing a system of rewards and penalties - implementation of properly working points system for better training the model.
- Training the NPC - training of the computer player based on simulated games, allowing it to improve its tactics through experience and feedback.
- Evaluating the model - conduction of the experiments to evaluate the efficiency, adaptability and success rate of the trained computer player.

1.5 Short description of chapters

This thesis consist of the five separate chapters, where:

- Chapter 2 provides the theoretical background required to understand the investigated problem. It introduces selected concepts related to artificial intelligence, deep learning, reinforcement learning, and decision-making methods used in computer games. Particular attention is given to the sparse reward problem and techniques that can support learning in complex environments. The chapter also reviews existing studies and solutions related to training artificial agents.
- Chapter 3 presents the subject of the thesis and the proposed solution. It describes the rules and characteristics of Rummikub, the structure of the implemented learning environment, and the challenges resulting from the size of the state and action spaces. The chapter also discusses Unity, the ML-Agents Toolkit, the representation of observations and actions, the reward system, action masking, and the configuration of the learning process. Furthermore, it introduces PPO, GAIL, and the designed Curriculum Learning environment.
- Chapter 4 describes the experimental methodology and presents the obtained results. It explains the training configurations, stopping criteria and evaluation method. The training progression, reward, policy entropy, number of steps, training duration, and evaluation performance are analyzed separately for PPO Baseline and PPO+GAIL. The chapter concludes with a direct comparison of both configurations.
- Chapter 5 summarizes the conducted work and its main findings. It evaluates whether the objective of the thesis was achieved, discusses the limitations of the developed solution, and presents possible directions for future development, extending the curriculum, transferring the acquired skills to the full game, preparing level-specific demonstrations, tuning hyperparameters, and examining alternative learning methods.

1.6 Description of contribution of the thesis's author

The essential elements of the project, that include the author contributions, are:

- Development of experiments environment - the design and implementation of a fully functional digital Rummikub environment, including game logic, rule validation, and agent interaction mechanisms.

- Supervision and configuration of the learning process – the definition of the state and action spaces, design of the reward structure, configuration of training parameters, and monitoring of the computer player’s learning progress.
- Evaluation of the computer player’s performance – the design and execution of simulation-based experiments to assess the effectiveness, adaptability, and decision-making quality of the trained AI model.
- Analysis and interpretation of results – the examination of experimental outcomes and formulation of conclusions regarding the applicability of artificial intelligence methods to tactical board games with stochastic elements.

Chapter 2

Analysis of the problem

This chapter presents a detailed analysis of the research problem addressed in this thesis. The objective of this work is to design and evaluate artificial intelligence algorithms capable of training a computer-controlled player in a logic-based board game environment. The developed 2.5D board game serves as an experimental platform for testing different learning approaches.

2.1 Problem statement

The development of intelligent Non-Player Characters (NPC) capable of operating effectively in complex, rule-based board games remains a significant challenge in artificial intelligence. While classical board games such as chess and Go have been successfully addressed using advanced reinforcement learning and search-based techniques, many logic-based tile placement games introduce additional layers of combinatorial complexity, dynamic action spaces, and stochastic variability.

This thesis addresses the problem of training a computer-controlled player in a complex logic board game implemented in the Unity environment. The game serves as an experimental testbed for evaluating artificial intelligence algorithms under conditions of:

- high-dimensional state representation,
- large and dynamically changing action space,
- strict rule-based constraints,
- significant randomness in gameplay.

2.2 State of the art

Developing decision-making algorithms has played a significant role in the domain of digital games for several decades. From early deterministic rule-based systems to modern

deep learning architectures, used techniques have continuously evolved to enhance the behavior of NPCs and automated opponents. Over time, various algorithmic approaches have been employed, including finite state machines, heuristic search methods, planning algorithms, and, more recently, reinforcement learning techniques. This section reviews the most relevant contemporary approaches to Non-Player Character Behavior Systems used in games, with particular emphasis on reinforcement learning algorithms and decision-making systems applicable to complex, rule-based board games with large and dynamic action spaces.

2.2.1 An Overview of NPC Decision-Making Techniques

The aim of using Non-Player Character Behavior Systems in digital games is to create computer-controlled players that can imitate human behavior and compete effectively with human opponents. A key challenge is designing a computer player that not only performs well and is capable of winning against human players, but also demonstrates behavior that appears natural and realistic during gameplay. To achieve this, developers and researchers employ a variety of AI algorithms and computational models:

- Finite State Machines (FSMs)

It is a foundational, abstract computational model utilized extensively in video game development to govern the decision-making processes of NPCs, operating on the principle that an entity can only exist in one of several predefined states at any given moment. Despite the emergence of more advanced AI architectures, FSMs remain highly relevant in modern game development, functioning either as standalone systems or as complementary modules in complex AI frameworks. An FSM is formally defined by a 5-tuple:

$$M = (\Sigma, S, s_0, \delta, F) \tag{2.1}$$

Where

- Σ is the input alphabet, finite set of input signals
- S is a finite set of states
- s_0 is the starting state
- δ is the state-transition function
- F represents final states

While highly efficient, FSMs suffer from scalability issues, as an increase in states can lead to an exponential explosion of transition rules [30].

- Behavior Trees (BTs)

BTs solve the scalability issues of FSMs by utilizing a hierarchical, directed acyclic graph structure. The architecture consists of root nodes, control flow nodes, composites like Sequences and Selectors, decorator nodes, and execution leaves, actions and conditions. The system operates via a signal called a *tick* that traverses the tree at regular intervals [30]. The execution of the sequence proceeds as follows:

1. The root node of BT generates execution *tick*.
2. The root node passes the tick signal downward to its child nodes, sequentially from left to right.
3. The signal cascades through the intermediate nodes until it reaches a leaf node, which is responsible for a certain action.
4. The leaf node evaluates its own progress. If the child returns a *Success* state, proceed to execute the next child in the sequence.
If the child returns a *Running* state, it halts execution and returns *Running* to the parent.
If the child returns a *Failure* state, it halts execution and immediately returns *Failure* to the parent.
5. The resulting status is passed back up to the parent composite node.
If the parent is a Selector, and it receives a Failure, it sends a new tick to its next available child.
If the parent is a Sequence, and it receives a Success, it sends a new tick to its next available child.
If any parent receives a Running status, all lateral traversal stops instantly, and the Running status is pushed all the way back up to the root, effectively locking the tree's focus onto that ongoing action for the current frame.

- Rule-Based Systems (RBS)

They represent a classical decision-making paradigm in games. An RBS is structurally segregated into three primary components: an information database, a formalized rule set, and an arbiter mechanism. The database acts as the computer player's memory, storing environmental observations and internal states. The rule set contains multiple declarative conditional statements *if-then-else rules*. The arbiter acts as the central processing unit, analyzing the database against the rule set to trigger the appropriate action. [30]

- Heuristic Search

These algorithms are systematic exploration techniques utilized to navigate through vast state spaces efficiently by incorporating domain-specific knowledge. Unlike blind

search methods, heuristic algorithms evaluate the potential of intermediate states to reach a goal, drastically reducing the computational overhead. In modern game architectures, such as hyper-heuristics, heuristic search is frequently deployed for high-level macro-navigation or to solve deterministic portions of a game, while other algorithms take over for stochastic or localized maneuvering. [22]

- **Minimax Algorithm**

This algorithm is a classical recursive strategy utilized predominantly in zero-sum, complete-information board games. It models a strictly adversarial environment where one agent seeks to maximize its utility while the opponent aims to minimize it. The algorithm constructs a game tree representing all possible future moves. Since exhaustive search is practically impossible for complex games due to state space explosion, the search is truncated at a specific depth, and a manually designed heuristic evaluation function is used to score the leaf nodes [24]. The algorithm goes as follows

$$v(s) = \begin{cases} eval(s) & \text{if } s \text{ is terminal} \\ \max_a v(result(s, a)) & \text{if maximizing player} \\ \min_a v(result(s, a)) & \text{if minimizing player} \end{cases} \quad (2.2)$$

Where

- v is value of state, which estimates how favorable a given situation is for the player
- s is the current state of game, representing a specific configuration of all game elements at a given time.
- a is an action that player can make in state s to transition to a new state

- **Genetic Algorithms (GA)**

These algorithms represent a robust category of search optimization and stochastic techniques heavily inspired by the biological principles of natural selection and evolution. In the context of computer games, these algorithms do not rely on traditional numerical expressions but operate on populations of candidate solutions, often encoded as chromosomes. The evaluation of each individual in the population is handled by an adaptation function, which calculates a numerical assessment known as fitness [14]. This algorithm goes as follows

1. Generate an initial population of random individuals.
2. Assess the adaptation value (fitness) of every individual in the current population using the game's simulation environment.

3. Using fitness function to select the parents for the next generation.
 4. Merge segments of the parent chromosomes to produce new offspring.
 5. Introduce random alterations to certain genes within the offspring to maintain genetic diversity and prevent premature convergence.
 6. Replace the old population with the newly generated offspring and repeat steps 2-5 until a termination condition is satisfied.
- **Deep Neural Network**
DNN is based on Artificial Neural Network (ANN), where each neuron can be interpreted as a linear function followed by a non-linear activation. A DNN extends this concept by stacking multiple layers of neurons, allowing the model to learn more complex patterns and representations. Because of this layered structure, DNNs are capable of approximating highly complex functions, while still relying on simple operations at the level of individual neurons. This technique is described in detail in chapter 2.2.2.
 - **Reinforcement Learning (RL)**
Algorithm allows agents to learn optimal policies through trial-and-error interactions using reward-penalty system. This technique is described in detail in chapter 2.2.3.

2.2.2 Deep Learning

DL is a thriving field, with active research topics aimed at automating routine labor, understanding speech, images, and supporting basic scientific research. AI systems are designed to solve tasks that are intellectually challenging for humans, but can be described formally, like playing chess, as well as intuitive tasks that are difficult to describe formally, like recognizing faces. The overview of deep learning presented in this section is primarily based on the information provided in [8].

History of Deep Learning

The pursuit of DNN dates back to ancient times, but, according to [8], as a computational discipline it has evolved through three waves:

1. The First Wave: Cybernetics (1940s–1960s):

The earliest computational models of learning were intended to be biological models of how learning happens in the brain. This era, known as cybernetics, focused on biologically inspired linear models. In 1943, the McCulloch–Pitts neuron was introduced as an early model of brain function, capable of recognizing two categories by testing whether a linear combination of inputs was positive or negative. In

the 1950s, this was followed by the Perceptron, which was the first model capable of automatically learning the weights that define categories from given examples. Around the same time, the Adaptive Linear Element (ADALINE) was developed to predict real numbers using an early form of stochastic gradient descent. However, this first wave ended in a severe backlash. Critics, most notably Minsky and Papert in 1969, demonstrated the profound limitations of linear models. Famously, they proved that simple linear models could not learn the XOR (exclusive OR) logic function. This mathematical limitation resulted in a significant decline in funding and popularity for neural network research.

2. The Second Wave: Connectionism (1980s–1990s)

The second wave emerged within the field of cognitive science, as part of a movement known as connectionism. The central thesis of connectionism is that intelligent behavior emerges from a large number of simple computational units connected in a network. This era introduced two monumental concepts that remain central to modern deep learning:

- **Distributed Representations** - Hinton et al. introduced the principle that each input to a system should be represented by many features, with each feature representing many possible inputs.
- **Back-propagation** - The successful use and popularization of the back-propagation algorithm by Rumelhart et al. allowed for the training of nowadays neural networks with multiple hidden layers.

Although the mathematical foundations used today were introduced during this period, the wave faded by the mid-1990s. This was partly due to unrealistic claims being made about AI capabilities, which led to investor disappointment when researchers could not deliver on these promises. Moreover, other machine learning paradigms with convex objective functions, such as support vector machines and graphical models, started to outperform neural networks on important tasks.

3. The Third Wave: Deep Learning (2006–Present)

The modern renaissance of neural networks began in 2006, overturning the long-held belief that deep models were too difficult to optimize. Geoffrey Hinton led the breakthrough by demonstrating that a deep belief network (DBN) could be efficiently trained using a technique known as greedy layer-wise unsupervised pre-training. This discovery proved that deep architectures could outperform shallow models such as Support Vector Machines. This development popularized the term **deep learning** to emphasize the theoretical importance of depth in architectural design.

While the foundational algorithms, such as back-propagation, have remained unchanged since the 1980s, the explosive success of the current wave is largely attributed to the age of *Big Data*, which provides the millions of examples necessary for statistical generalization, and to the massive parallel computational power of modern general-purpose GPUs.

An Overview of Deep Learning

According to [8], deep learning is a highly specialized and powerful sub-discipline of machine learning. It is designed to solve complex problems that cannot be easily formalized using traditional methods, such as speech recognition and computer vision. Unlike early artificial intelligence systems, which relied heavily on hard-coded knowledge and manually engineered rules, deep learning systems can acquire knowledge autonomously by extracting complex patterns directly from raw data. The term **deep** in DNN refers to the architecture of the network, where a standard neural network consists of an input layer, a single hidden layer and an output layer. A deep neural network has at least two hidden layers between the input and output layers.

In the domains of computer science and machine learning, the terms Neural Network (NN) and Artificial Neural Network (ANN) are utilized interchangeably. These systems represent a robust class of computational models structurally and functionally inspired by the biological nervous systems of animals and humans. The foundational principle of an ANN is connectionism: although a single artificial neuron, or node, has limited computational ability, a large population of interconnected neurons acting in parallel can exhibit highly intelligent, complex behavior [8].

The figure 2.1 presents an example of the internal structure of an NN, meaningfully representing the most popular architectural paradigm, the feedforward neural network architecture. The initial layer, **Input layer**, receives the raw, pre-processed environmental data, such as image pixels or numerical matrices. The **hidden layer** is an intermediate layer that performs the core abstraction and feature extraction. It is denoted as *hidden* because the training data does not explicitly define what the output of these specific layers should be, the learning algorithm must autonomously deduce the optimal intermediate representations to solve the task. The number of neurons within a single layer determines the network's *width*, while the total number of sequential hidden layers defines the network's *depth*. The **Output Layer** is the terminal stage of the network. It synthesizes the highly abstract, complex representations generated by the deepest hidden layers into a final predictive mapping, such as a continuous scalar value for regression or class probabilities for classification. [8]

At its core, a deep neural network represents a hierarchical composition of numerous

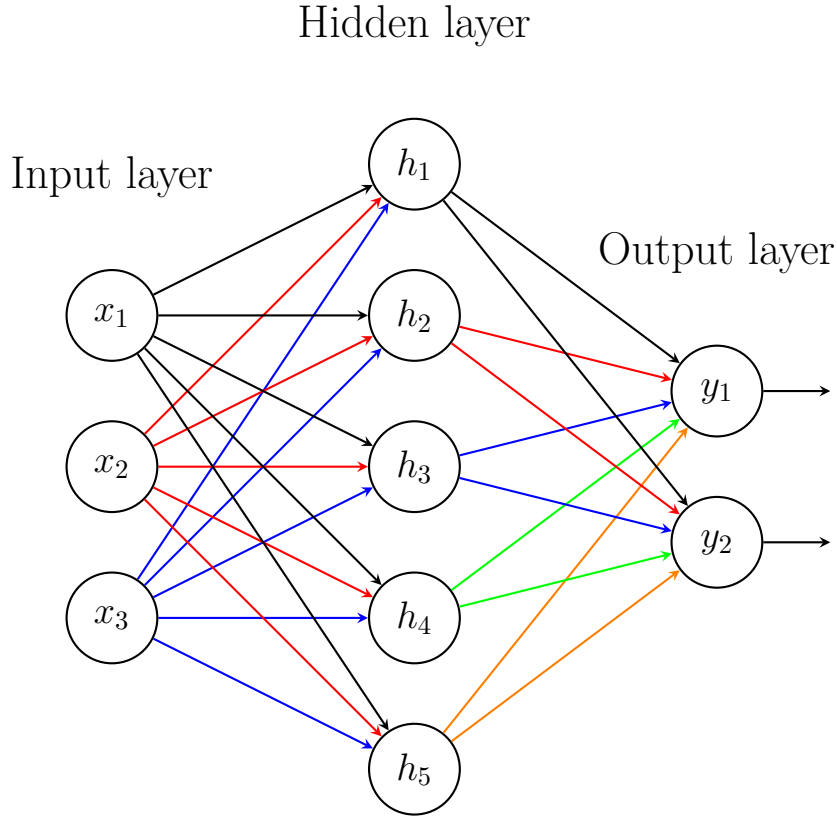


Figure 2.1: The graph of feedforward neural network

mathematical functions, where each function corresponds to a distinct layer

$$f(x) = f^{(n)}(f^{(n-1)}(\dots f^{(1)}(x))) \quad (2.3)$$

where

- x represents the initial input vector
- $f^{(i)}$ represents the transformation applied at the i -th hidden layer

The processing power of a neural network does not come from observing the data passively, but from actively transforming it. Within each hidden layer, the network performs two sequential mathematical operations on the incoming data: an affine transformation followed by a fixed non-linear activation. The figure 2.2 presents the schematic of each perceptron, which is widely recognized as the foundational architectural component of modern ANN. [8] The affine transformation is responsible for multiplying the inputs from the previous layer by a learned weight matrix W and adding a learned bias vector b . For a single neuron (perceptron), the formula is as follows [8]:

$$y = f\left(\sum_{j=1}^N w_j x_j + b\right) \quad (2.4)$$

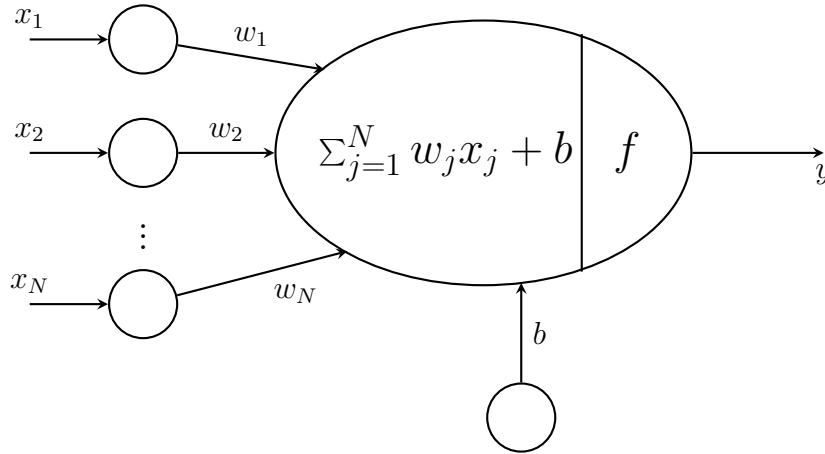


Figure 2.2: The graph of single perceptron

Where:

- y is neuron output signal,
- f is activation function,
- w_j is the weight of j -th input,
- x_j is the value of j -th input,
- b is bias, which is constant,
- N is the amount of inputs,

The weights determine the strength and direction of the relationship between variables, while the bias adjusts the activation threshold. However, if the network only applied linear matrix multiplications, the entire architecture would collapse mathematically into a single linear regression model, which would render it incapable of solving complex problems. To prevent this, the result of the affine transformation is passed through an element-wise, non-linear **activation functions**. There are many types of **activation functions**, the most popular are ReLu, logistic sigmoid, and tanh:

- Rectified Linear Units (ReLu) - is currently established as the default and most widely recommended activation function for the hidden layers of modern feedforward neural networks.

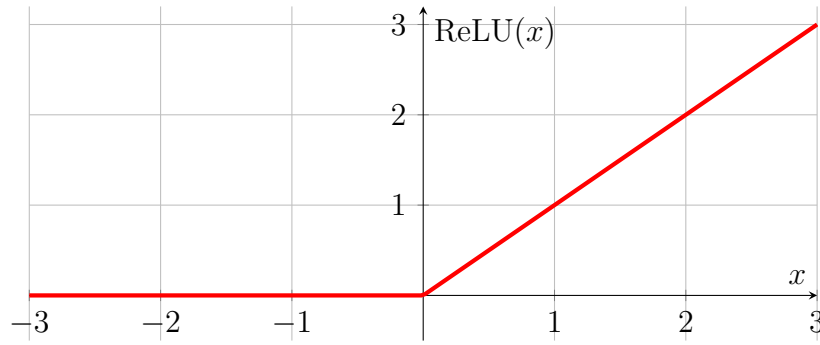


Figure 2.3: ReLu plot

The plot of the function is presented in 2.3, the formula for the ReLu function is as follows:

$$g(x) = \max(0, x) \quad (2.5)$$

The fundamental advantage of ReLU is that its behavior remains very close to linear. Since its first derivative is precisely 1 wherever the unit is active, gradients can propagate backwards through the network without diminishing in magnitude. This consistent, non-saturating gradient direction renders gradient-based optimization highly effective and prevents the vanishing gradient problem that is common in deeper networks.

- Logistic Sigmoid - Before ReLU became dominant, the logistic sigmoid was the standard activation function for neural networks. The sigmoid function's primary utility is its ability to output a value that can be interpreted as a probability, making it highly effective for binary classification tasks when placed at the final output layer. However, its usage within hidden layers is now strongly discouraged. The function saturates quickly when its argument becomes very positive or very negative. In these saturated regions, the curve becomes exceptionally flat, causing the local derivative to approach zero. This widespread saturation severely shrinks the gradients during back-propagation, making gradient-based learning extremely difficult or causing it to stall completely. The plot of logistic sigmoid function is presented in the image 2.4, the formula for the Logistic Sigmoid function is as follows:

$$\sigma(x) = \frac{1}{1 + e^{-x}} \quad (2.6)$$

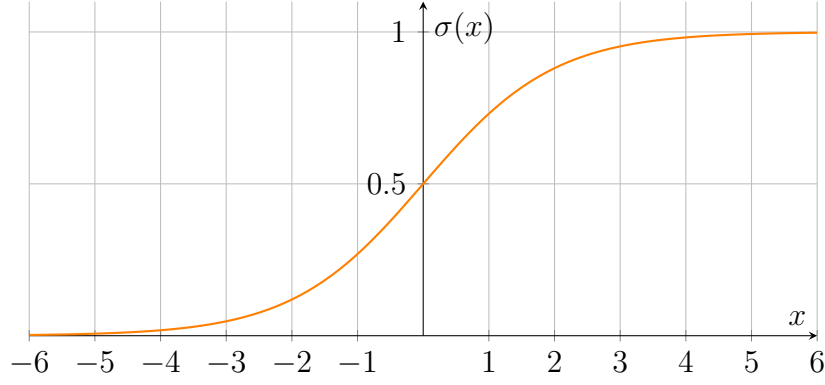


Figure 2.4: logistic sigmoid plot

- Hyperbolic Tangent (Tanh) - The hyperbolic tangent is another classical activation function, closely related to the logistic sigmoid but exhibiting slightly better optimization properties. The plot of tanh function is presented in the image 2.5.

$$g(x) = \tanh(x) \quad (2.7)$$

It is mathematically related to the sigmoid function through the identity

$$\tanh(x) = 2\sigma(2x) - 1 \quad (2.8)$$

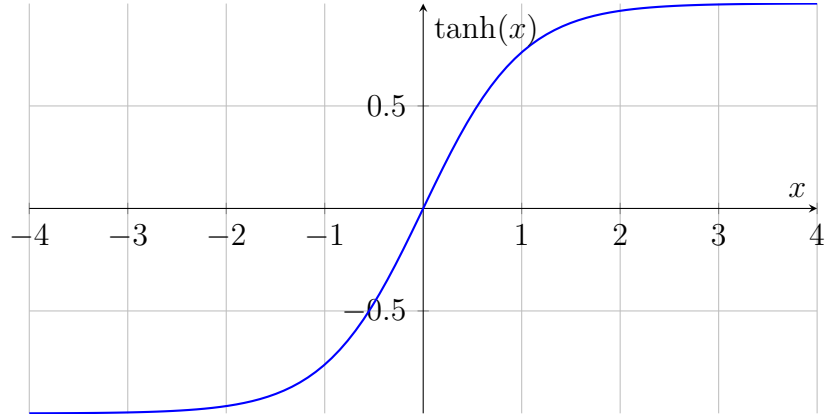


Figure 2.5: Tanh plot

If a model architecture strictly requires a sigmoidal-style activation function in a hidden layer, the tanh function will usually be more effective than the logistic sigmoid. Its main advantage is that it resembles the identity function near the origin. As it is centred at zero, training a deep network with tanh resembles training a linear model, provided the activations are kept small. This facilitates easier training dynamics. However, like the sigmoid function, it still suffers from the same fatal flaw: it saturates at its extremes, leading to vanishing gradients.

The loss function, also referred to as the cost or error function, is the mathematical criterion that the learning algorithm seeks to minimize. It quantifies the difference between the predicted output of the NN and the true target value provided in the dataset. The choice of loss function depends heavily on the choice of the model's output units and the nature of the task

- **Negative Log-Likelihood (Cross-Entropy)** - Most contemporary neural networks used for classification tasks are trained using maximum likelihood estimation. The associated cost function is the negative log likelihood, which is mathematically equivalent to the cross-entropy between the distribution of the training data and the distribution of the predicted probabilities of the model. A critical architectural advantage of this loss function is its ability to prevent gradient saturation. Many output units utilize exponential functions, such as the sigmoid function, which can flatten out and halt learning. The logarithmic transformation in the cross-entropy loss function effectively neutralizes these exponentials, ensuring that the gradients remain large and predictable enough to efficiently guide the optimization algorithm. The formula is as follows [8]:

$$CE = -\frac{1}{n} \sum_{i=1}^n \sum_{j=1}^k y_{ij} \log(\hat{y}_{ij}) \quad (2.9)$$

Where:

- n is amount of inputs
 - k is the amount of classes
 - y_{ij} is true label
 - $\hat{y}_{ij}$ is the model's j -th prediction
- **Mean Squared Error (MSE)** - is the standard cost function employed for regression tasks involving continuous numerical outputs. From the perspective of the calculus of variations, if a high-capacity model is optimized over an infinitely large sample from the true data generating distribution, minimizing the MSE functional mathematically drives the model to predict the statistical mean (the expected value) of the target variable y for any given input x . The formula is as follows:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (2.10)$$

Where:

- n is amount of inputs
- y_i is real value (target)

- $\hat{y}_i$ is the model's prediction

In the optimization of deep neural networks, **the back-propagation** algorithm is a highly efficient procedure used to compute the gradient of the cost function with respect to the network's parameters. Back-propagation **does not** refer to the entire learning algorithm of a neural network, it is strictly a method for evaluating the gradients. **The back-propagation** algorithm operates on a formal structure known as a *computational graph*, where each node represents a specific variable and directed edges represent the operations that map input variables to output variables. **The back-propagation** enables the error signal from the cost function to travel backwards through the graph, allowing the partial derivatives to be computed for each node. The mathematical engine driving this reverse flow of information is the chain rule of calculus. The algorithm recursively applies the chain rule, computing a Jacobian-gradient product for each operation in the graph. The Chain Rule in scalar form goes as follow [8]:

$$\text{If } y = g(x) \text{ and } z = f(y), \text{ then } \frac{\partial z}{\partial x} = \frac{\partial z}{\partial y} \frac{\partial y}{\partial x} \quad (2.11)$$

When the goal is to find the parameters that result in the lowest possible value of cost function, the procedure relies heavily on optimization algorithms. **Gradient descent**, also historically known as the method of steepest descent, is the foundational first-order optimization algorithm used to minimize continuous, differentiable objective functions. The algorithm leverages the mathematical concept of derivatives. In the case of a simple scalar function, the derivative indicates the slope of the function at a given point and shows how the output changes in response to an infinitesimally small change in the input. In high-dimensional spaces typical of neural networks, the function accepts a vector of parameters. The derivative is generalized into a vector of partial derivatives known as the gradient. The optimization process requires iteratively updating the parameters of the model. The magnitude of the step taken in the direction of the negative gradient is controlled by a strictly positive scalar known as the learning rate or step size.

$$x' = x - \epsilon \nabla_x f(x) \quad (2.12)$$

Where

- ϵ is the learning rate
- x represents the parameters to be optimized
- $\nabla_x f(x)$ is the gradient vector

Optimization algorithms are mathematical procedures designed to minimize, or maximize, an objective function $f(x)$. These procedures do this by systematically modifying

the parameters x . Unlike pure optimization, where the sole goal is minimizing a known function, machine learning optimization operates indirectly. The true goal is to minimize the expected generalization error evaluated over the true data generating distribution. As this true distribution is unknown, the algorithms instead perform empirical risk minimization, seeking to minimize the expected loss evaluated over the empirical distribution defined by a finite training dataset. Most modern deep learning models are optimized using first-order gradient-based methods, which rely on the first derivative, the gradient, of the cost function to determine the direction of steepest descent. To overcome the computational bottleneck of evaluating the gradient across billions of training examples, Stochastic Gradient Descent (SGD) uses a randomly sampled subset of the data, known as a minibatch, to estimate the true gradient [8].

$$\hat{g} = \frac{1}{m} \sum_{i=1}^m \nabla_{\theta} L(x^{(i)}, y^{(i)}, \theta) \quad (2.13)$$

$$\theta \leftarrow \theta - \epsilon \hat{g} \quad (2.14)$$

where

- ϵ is the learning rate defining the step size
- $\hat{g}$ is the gradient estimate over a minibatch of m examples
- m is size of sample
- $\sum_{i=1}^m$ is the sum of all examples in the minibatch
- ∇_{θ} is the gradient operator with respect to the model parameters θ
- $L(x^{(i)}, y^{(i)}, \theta)$ is loss function for a single sample with input $x^{(i)}$ and expected output $y^{(i)}$

While highly effective, gradient-based optimization in deep learning faces several severe topological challenges:

- Saddle Points - Rather than encountering true local minima, the algorithm frequently encounters saddle points: regions where the gradient is zero but the curvature is a minimum in one cross-section and a maximum in another.
- Cliffs and Exploding Gradients - In deep or recurrent architectures, parameters can reach cliff-like structures where the gradient suddenly becomes exponentially large. A standard gradient descent step can catapult the parameters completely out of the optimal region, destroying previous optimization progress

2.2.3 Reinforcement Learning

Reinforcement Learning (RL) is a sub-group of machine learning algorithms. Rather than being directly programmed to fulfill a task, it learns to achieve certain results over time using received data. The goal of RL is to estimate universal principles from received data in order to adapt to unseen environments. This chapter reviews this sub-group of AI and briefly describes its origin.

History of Reinforcement Learning

As described in [27], the term *Reinforcement Learning* comes from the word *a reinforcer*, which is used in the context of a stimulus received by animals in nature. In 1911 Edward Thorndike defined the trial-and-error learning, which he called **Law of Effect**. He stated that if the response to a certain action would come with a positive sensation, then such an action will be repeated. Conversely, if certain actions would come with discomfort, then such actions would be avoided.

In 1950 Alan Turing published a paper "Computing machinery and intelligence" [29], where he stated a crucial question: "Can machines think?".

Turing correctly identifies that manually coding an exhaustive *table of instructions* to simulate an adult human mind is a computationally impossible task. His proposed paradigm shift is the conceptual birth of Machine Learning. Instead of programming an adult, engineers should simulate a *child brain*, a system with minimal initial mechanisms and vast amounts of blank storage. This system is then subjected to an educational regimen. Turing explicitly draws a mathematical and systemic parallel to biological evolution:

- Structure of the child machine = hereditary material
- Changes of the child machine = mutation
- Natural selection = judgment of the experimenter

The Child Machine Learning Loop algorithm is proceeded as follows:

1. Create a discrete-state machine with a minimal instruction set and an integrated *random element* generator
2. The experimenter provides a task, instruction, or data via unemotional communication channels, like a symbolic language
3. The machine generates an output. The random element allows for non-systematic exploration of the solution space, avoiding infinite loops
4.
 - If the output is incorrect, apply a punishment signal. The machine's internal parameters are updated to decrease the probability of repeating the preceding sequence

- If the output is correct, apply a reward signal. The parameters are updated to increase the probability of sequence repetition
5. The rules of operation dynamically adapt over time. The machine evolves from a blank slate to a highly complex cognitive engine capable of competing in intellectual fields like chess or natural language processing

By using trial, error, rewards, and punishments, the child machine slowly writes its own adult rulebook. This idea represents an early implementation of trial-and-error learning, what is now known as reinforcement learning.

In the mid-1950s, Richard Bellman introduced the concepts of a system's state and an *optimal return function*. This led to the formulation of a functional mathematical equation now known as the Bellman equation, and the development of dynamic programming, which is a set of methods designed to solve optimal control problems. Bellman also formalized the discrete stochastic version of these problems, establishing Markov Decision Processes (MDPs). This led to Ronald Howard creating policy iteration method for MDPs. While dynamic programming provided a rigorous framework, it suffered from what Bellman termed the **curse of dimensionality**, meaning its computational demands grew exponentially with the addition of state variables. Furthermore, it was initially viewed strictly as an offline computational method requiring a perfect mathematical model of the environment. This distanced it from online learning methodologies.

Temporal-difference (TD) learning is based on the mathematical difference between successive estimates of a single quantity over time. The origins of TD learning are linked to the psychological concept of *secondary reinforcers*, stimuli that gain reinforcing properties by being paired with primary rewards, like food. Arthur Samuel implemented a checkers-playing program in 1959, as the first person to use TD principles. In 1977, Ian Witten published what is recognized as the earliest temporal-difference learning rule, proposing a tabular TD(0) method for adaptive controllers. The thread was mathematically formalized by Richard Sutton in 1988, who separated TD learning from control, treating it as a generalized prediction method and proving the convergence properties of the TD algorithm.

While trial-and-error learning, optimal return function, and temporal-difference learning had previously developed as separate fields, they were brought together by Christopher Watkins in 1989. He developed the Q-learning algorithm, effectively proving that an agent could utilize temporal-difference methods in a model-free, trial-and-error environment to incrementally approximate the optimal value functions defined by Bellman's Dynamic Programming. This theoretical unification was followed by significant practical success in 1992, when Gerry Tesauro combined TD learning with artificial neural networks to create TD-Gammon, a program that achieved masterful performance in backgammon and brought widespread attention to the modern field of Reinforcement Learning. [27]

Fundamentals of Reinforcement Learning

Machine learning is a subfield of artificial intelligence that focuses on algorithms which analyze data in order to identify patterns and make accurate predictions. Rather than being programmed directly to perform a specific task, these algorithms learn from the data they are given. According to [8], ML can be divided into three subcategories:

- supervised learning,
- unsupervised learning,
- reinforcement learning

Supervised learning is a technique in which an algorithm trains on labeled data. By providing pair of input data and a proper output example, ML model can learn pattern between data and output to create proper predictions in the future, familiar tasks.

In unsupervised learning technique, the model receives data without proper output, unlike in supervised learning. The aim of this subgroup is to detect correlations within the data rather than make predictions.

In contrast to both supervised and unsupervised learning, reinforcement learning does not receive any data at the beginning of training process. The agent is placed in a simulated environment and, while interacting with it, gets rewards or penalties through trial and error.

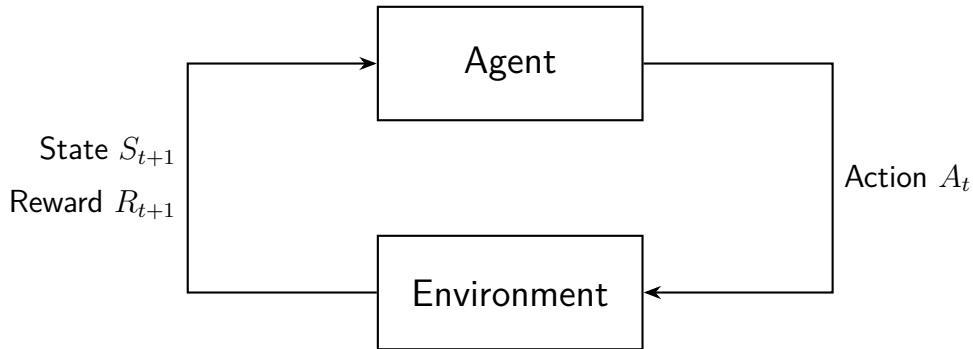


Figure 2.6: Reinforcement learning diagram

Figure 2.6 presents the general diagram of RL system. The agent observes the environment and its current state. Based on this information, the agent takes an action. After each decision, the agent receives a reward or penalty depending on the predefined reward system. Initially, all interactions with the environment are unplanned, but over time, the agent begins to make considered decisions.

Reinforcement Learning is the practical algorithmic methodology deployed to solve the Markov Decision Process (MDP). It is a rigorous mathematical framework used to model sequential decision-making scenarios in stochastic and dynamic environments. It provides

a structural paradigm that outlines how an agent interacts with a given environment over time. The defining characteristic of this framework is the Markov property, also known as the Markov assumption. This property dictates that the probability of transitioning to any future state is determined solely by the current state and the agent's most recent action. Consequently, the historical sequence of previously visited states and prior actions is mathematically irrelevant when it comes to predicting the system's future dynamics. This enables the problem to be treated as memoryless, which significantly reduces computational complexity [20]. An MDP is formally described as a 5-tuple of interacting mathematical components that govern the mechanics of the environment [11]

$$M = (S, A, P, R, \gamma) \quad (2.15)$$

Where:

- S - The state space, representing the exhaustive set of all possible configurations or situations the environment can occupy.
- A - The action space, denoting all permissible moves or decisions the agent can execute.
- $P(s'|s, a)$ - The state transition probability, which defines the mathematical likelihood of the environment shifting to a new state s' given that action a was taken in the current state s .
- $R(s, a)$ - The reward function, providing the immediate numerical feedback generated upon taking action a in state s .
- γ - The discount factor, a tuning parameter that balances the agent's preference between immediate gratification and long-term strategic returns.

In a scenario where the exact transition probabilities P and reward distributions R are perfectly known in advance, the MDP can be solved analytically using classic Dynamic Programming techniques. However, in realistic RL scenarios, these underlying environmental dynamics are completely unknown to the agent. Therefore, the RL algorithm acts as an autonomous explorer that must discover the optimal strategy strictly through trial-and-error interaction within the boundaries set by the MDP. The ultimate objective of the RL algorithm is to derive an optimal policy. This is defined as a comprehensive mathematical mapping from the MDP's states to specific actions, that maximizes the expected cumulative reward over the agent's operational lifetime [20]. Since underlying mechanism of the policy is MDP, the policy instructs the agent on how to act strictly based on its current situation while ignoring the history of its actions.

The RL theoretical landscape can be extracted as primarily divided into four axes: optimization target (Value-Based vs. Policy-Based), the dependency on environmental

dynamics (Model-Free vs. Model-Based), the learning update mechanism (Monte Carlo vs. Temporal Difference) and the data collection strategy (On-Policy vs. Off-Policy). These axes are orthogonal, any algorithm can be characterized as a combination of choices across all dimensions. It is important to understand these classifications in order to select the proper algorithmic framework for specific complex control problems.

In value-based methods, the algorithm channels all computational and function approximation efforts into accurately estimating a value function, such as the optimal state-value function or the state-action value function. Once the value function is estimated, the policy itself is not explicitly modeled, rather, it is derived implicitly. The agent acts by strictly choosing the *greedy* action, the specific action that corresponds to the highest estimated mathematical value in any given state. While highly successful in many applications, the value-based paradigm possesses severe theoretical limitations with stochasticity. In environments where the true optimal policy is stochastic, pure value-based methods consistently struggle. Also due to action selection based on finding the maximum value, an arbitrarily microscopic change in the estimated value of an action can discontinuously alter whether that action is selected or ignored [28].

The policy-based approach, often referred to as policy gradient methods, bypasses the intermediate step of value estimation for action selection. Instead, the stochastic policy is explicitly represented by its own independent function approximator, parameterized by a vector θ , which dictates the probability of selecting an action. The algorithm then performs gradient ascent directly on the performance objective, which is theoretically guaranteed to converge to at least a locally optimal policy [28].

In model-free reinforcement learning (MFRL), the agent lacks any prior or learned knowledge regarding the environment’s transition model and reward function. Because the agent is completely blind to the mathematical rules governing the environment, it must rely exclusively on extensive trial-and-error physical interactions to progressively deduce optimal policies and update its value estimations. The primary advantage of MFRL is its immunity to model bias. Because the agent learns exclusively from ground-truth, experienced data sampled directly from the real world, it often achieves superior and highly robust asymptotic performance. However, this reliance on direct interaction results in massive sample complexity. MFRL requires a substantial amount of training data to converge, which makes it impractical for many real-world applications, such as robotics or autonomous driving, where physical trial-and-error is slow, expensive, or dangerous [20] [19]

Model-based RL was developed specifically to resolve the severe sample inefficiency of model-free methods. In this paradigm, the agent actively builds a predictive abstraction of the environment, known as the model. Formally, given an MDP, the model learning phase focuses on recovering the unknown state transition dynamics M and the reward function R . By constructing this environment model, the agent gains the capability of

imagination. It achieves reversible access to the MDP, allowing it to query any specific state-action pair in a simulated environment to predict future consequences. This simulated playground allows the agent to conduct extensive planning and generate artificial simulation data without incurring any real-world costs. The most significant benefit of MBRL is its exceptional data efficiency, by planning inside a simulated model, the agent can learn optimal behaviors using a fraction of the real-world samples required by MFRL. Furthermore, models allow for forward planning algorithms like Model Predictive Control, which can naturally integrate safety constraints. However, MBRL suffers from the *generalization error*. If the learned model contains slight inaccuracies, generating long simulated trajectories causes these inaccuracies to amplify exponentially, a critical flaw. Consequently, a policy optimized perfectly within a flawed model may perform terribly in the real world situations. [19]

Monte Carlo (MC) Methods learn exclusively from completely finished episodes. The agent relies on the statistical concept of the mean return, utilizing randomness to solve the decision-making problem. Because the algorithm calculates updates based on actual accumulated rewards, it must wait until the episode reaches a terminal state before any value functions can be adjusted. The MC estimator is strictly unbiased due to the fact that its target is the actual, observed empirical return. However, because the target return aggregates all stochastic rewards and action sequences across an entire trajectory, MC methods inherently suffer from high variance [20].

Temporal Difference (TD) methods resolve the necessity of waiting for an episode’s conclusion by bootstrapping—forming a target immediately at the next time step and updating based on current estimates. By performing updates using the immediate reward and the predicted value of the subsequent state, TD learning occurs dynamically. Consequently, the TD estimator is biased because its target relies on a learned estimation rather than an ultimate factual return. Nevertheless, because the TD target incorporates far fewer stochastic trajectory components compared to MC, the degree of statistical variance is significantly reduced [20].

In On-Policy methodology the agent rigorously evaluates and improves the exact same policy that it currently uses for environmental interaction and decision-making. The algorithm acquires a batch of behavioral experiences and subsequently computes policy gradient updates exclusively utilizing the data induced by the current, active policy. Because the updates are tightly coupled to the agent’s immediate behavior, on-policy algorithms generally exhibit strong stability and are straightforward to implement. However, this paradigm introduces a severe limitation known as data inefficiency or high sample complexity. Once the agent’s policy parameters are updated, all previously collected data batches become obsolete and must be discarded, forcing the agent to continuously sample new trajectories from the environment to facilitate further learning [9].

Off-Policy agent evaluates and improves a target policy using historical interaction

data that may have been generated by entirely different, older behavior policies. To accomplish this, off-policy algorithms typically employ a memory mechanism known as an experience replay buffer. This buffer stores continuous streams of tuples containing the state, action, reward, and next state. By continuously recycling these archived transitions to compute updates, the agent breaks temporal data correlations and achieves exceptional sample efficiency. Despite their superior data efficiency, off-policy models are notoriously unstable during training, suffer from potential overestimation biases, and require highly careful hyperparameter tuning [9].

Despite the variety of reinforcement learning methods, all of them rely on a decision-making process driven by interactions with an environment. One of the most fundamental challenges inherent to this paradigm is the **Exploration vs. Exploitation Dilemma**. When taking an action, the agent must constantly choose between two competing behavioral strategies:

- Exploration - The agent intentionally selects stochastic actions in order to gather more overall information about the environment.
- Exploitation - The agent relies strictly on its current, established knowledge base to select the action that yields the highest known expected reward.

The key challenge in this dilemma is balancing the transition between exploration and exploitation. If the agent begins to exploit its environment too rapidly, it can get stuck in local optima and never discover the true globally optimal policy. However, if the agent would explore the environment for too long, it continually risks taking poor actions and fails to maximize its cumulative reward during its operational lifetime.

In the domain of Reinforcement Learning, a variety of algorithms exist, each with its own unique characteristics. The following list is a selection of these algorithms:

- REINFORCE - ("REward Increment = Non-negative Factor $\times$ Offset Reinforcement $\times$ Characteristic Eligibility") was introduced by Williams in 1992 and serves as a fundamental policy gradient method within the domain of Reinforcement Learning. It directly maps states to action probabilities using its own parameters, completely independent of a value function. By climbing the gradient of expected performance, small changes in the parameters guarantee only small, smooth changes in the policy and the state-visitation distribution. REINFORCE relies exclusively on real, experiential trajectories generated by interacting directly with the environment through trial and error, making it a model-free approach. It uses the actual, empirical returns collected from a given time step until the episode terminates. Because it waits for the entire trajectory to conclude to calculate the exact cumulative reward, it is classified as a Monte Carlo method. REINFORCE collects a batch of behavioral trajectories and computes the policy gradient update strictly from the data induced

by the agent's current, active policy. Once the policy parameters are updated, the old data is discarded, and new data must be collected using the newly updated policy. [8] [28]

- Q-Learning - is recognized as a foundational algorithm for solving optimal control problems in MDP. This algorithm relies on a discrete tabular architecture to record its acquired knowledge. The algorithm constructs a two-dimensional memory matrix, or Q-table, by directly mapping discrete environmental states to applicable actions. It then iteratively updates the Q-value, the mathematical utility of executing a specific action within a specific state of each entry in the matrix. Q-Learning directs the computational effort toward accurately estimating an optimal action-value function. The agent's actual decision-making protocol is implicitly derived from these estimates, it simply operates by selecting the action that mathematically corresponds to the highest evaluated Q-value in the current state. It does not possess or attempt to build an internal predictive model of the universe. It evaluates the consequences of actions exclusively through habitual, physical trial-and-error interactions directly within the environment. It updates its value estimations dynamically and instantly after every single environmental step. It achieves this by forming a target based on the immediately observed scalar reward and the predicted Q-value of the subsequent state, effectively using its own estimations to refine previous estimations. Q-Learning, decouples its optimization target from the agent's actual behavior. When formulating its update, it assumes the absolute best theoretical action will be taken in the subsequent state, completely disregarding whatever sub-optimal, exploratory move the active behavioral policy might actually execute next. [20]
- SARSA -(State-Action-Reward-State-Action) this algorithm works virtually identical to Q-Learning. The main difference is that SARSA is on-policy. This means the algorithm evaluates and improves the exact same policy that it currently uses to make decisions. SARSA updates its action-value function based strictly on the exact subsequent action a_{t+1} that the agent's current exploratory policy dictates it will actually execute. [20]
- Actor-Critic - this algorithm is a sophisticated hybrid architecture, designed to overcome the limitations of pure value and policy-based methods. The learning process is divided into two distinct yet interacting mathematical structures: the **Actor** and the **Critic**. The Actor maintains and updates the stochastic policy distribution to dictate the agent's behavior in the environment, while the Critic estimates the value function to evaluate the actions taken by the Actor. The Actor uses the feedback signal generated by the Critic to adjust its policy parameters iteratively towards optimal behavior. Both the Actor and the Critic learn strictly through continuous trial and error in physical interaction with the environment, deriving their updates

exclusively from real sampled trajectories. The Actor formulates an update target immediately after a single time step by combining the observed immediate reward with its current estimation of the subsequent state's utility. The critic evaluates the same behavioral policy that the actor is currently using to navigate the environment. The temporal difference error is calculated based on the sequential states visited and actions taken under the active policy. This means that if the actor's policy changes, the critic's value estimations must dynamically track those specific changes. [20]

Deep Reinforcement Learning

In classical RL, algorithms traditionally relied on discrete, tabular memory structures to evaluate states and formulate optimal policies. However, this tabular approach inherently falls victim to the *curse of dimensionality*, meaning it is mathematically and computationally incapable of scaling to environments characterized by continuous variables or high-dimensional sensory inputs. Deep Reinforcement Learning (DRL) resolve this limitation by substituting the discrete tables with DNN. By channeling raw, unstructured data through multiple hidden layers, the network autonomously distills complex environmental data into the abstract, hierarchical features necessary to approximate value functions and action policies.

Over time, some algorithms were adapted for use with DNN in RL, while others were created using only NN and cannot be found to be implemented without it. The following list presents some of these algorithms:

- Deep Q-Learning (DQL) - while classical Q-learning relies on discrete, tabular memory, DQL substitutes the table with DNN as a non-linear function approximator to estimate the Q-value function. By processing raw state variables through neural layers, the algorithm can autonomously predict the value of unvisited or newly emerging states, allowing the agent to generalize its policy in vast state spaces. [27]
- Asynchronous Advantage Actor-Critic (A3C) - it is the implementation of the Actor-Critic algorithm where the Actor and Critic structures are based on DNN. A major innovation within A3C is the integration of the **Advantage** metric. Instead of updating the policy based solely on the raw cumulative reward an action generates, the Critic computes the Advantage function to measure the relative merit of the chosen action compared to the average expected outcome in that specific state. By evaluating how much better an action was relative to a baseline expectation, the algorithm guides policy updates to strongly encourage actions that yield high advantages, thus maximizing the expected returns while simultaneously minimizing the variance of the gradient updates. A3C also improves the training efficiency by deploying multiple independent worker agents into parallel, distinct instances of

the environment. Each local worker agent maintains its own replica of the environment and calculates its gradients locally based on its own experiential trajectories. Instead of pausing and waiting for all other agents to finish their tasks, which creates a computational bottleneck, each worker asynchronously pushes its gradient updates to a central, global neural network. This asynchronous parallelism acts as a natural decorrelator for the experiential data. However the asynchronous nature of the algorithm means that while one worker is computing its local gradients, the global network may have already been updated multiple times by other workers. Consequently, a worker might apply gradients calculated from an outdated policy to a newly updated global network, leading to suboptimal optimization steps and mathematical instability [16]

- Advantage Actor-Critic (A2C) - this algorithm is descendant of A3C that eliminates the asynchronous uncoordinated updates. It allows multiple agents to collect experiences in parallel, but it waits for all parallel actors to complete their trajectory segments. Once all workers finish, A2C aggregates their data and calculates a unified, synchronous gradient update to the global network parameters. [1]
- Proximal Policy Optimization (PPO) - it utilizes an Actor-Critic architecture and implements two NN for Actor and Critic. The defining architectural characteristic of PPO is its use of a *clipped surrogate objective function*. In standard policy gradient algorithms, updating the policy based on sequential data can lead to excessively large parameter shifts, which fundamentally destabilize the neural network and cause catastrophic performance collapse. PPO mathematically bounds the policy update step, ensuring that the new strategy does not deviate significantly from the old strategy. This structural constraint establishes a *trust region*, leading to highly stable, monotonic performance improvements and making PPO a default algorithm in many advanced RL applications. [1] [36]
- Soft Actor-Critic (SAC) - it is another implementation of the Actor-Critic architecture, using a DNN for both the **Actor** and the **Critic**. The main difference between Actor-Critic and SAC is that SAC is off-policy due to the implemented replay buffer that stores recently made decisions. During the network optimization phase, SAC samples randomized mini-batches of this historical data, which heavily decorrelates the training samples and vastly improves the algorithm's sample efficiency compared to on-policy methods. The SAC algorithm is meticulously engineered to address the challenges of environments featuring high-dimensional, continuous action spaces. Introduced as a highly sample-efficient and stable method, SAC operates on the principles of maximum entropy reinforcement learning. It concurrently maximizes both the expected return and the entropy of the agent's policy.[23]

- AlphaZero - it is algorithm developed by DeepMind, that utilizes Actor-Critic architecture. AlphaZero's most profound achievement is its ability to learn from scratch, it requires no domain-specific human knowledge, hand-crafted evaluation heuristics or opening books. It relies solely on the basic rules of the game to achieve superhuman performance. AlphaZero replaces the traditional alpha-beta search and hand-crafted evaluation functions used by classical game engines, such as Stockfish, with a deep neural network and a general-purpose Monte Carlo Tree Search (MCTS) algorithm. This makes AlphaZero a model-based reinforcement learning system. Another difference from regular Actor-Critic architecture is that AlphaZero runs simulations to the end of the game, giving it a Monte Carlo learning approach. [25]

2.2.4 The Sparse Reward Problem

In Rummikub, winning a single game requires not only to complete the initial meld, but also to be the first player to successfully place all the tiles from their rack before the other players do. This requires planning each move with consideration of long-term results. Consequently, the reinforcement learning agent receives the most significant reward only at the end of an episode. Furthermore, the stochastic nature of agents' initial moves makes finding the winning strategy unlikely to occur. In the context of such a delayed feedback, it is more probable that the agent will adopt a policy of minimizing penalties rather than maximizing rewards, resulting in slow or unstable learning. This issue is referred to as **the sparse reward problem**, in which informative reward signals are too infrequent to effectively guide the learning process. Without additional guidance, the agent encounters the temporal credit assignment problem, whereby it becomes challenging to ascertain which earlier actions contributed to the final outcome of the game. Consequently, many beneficial decisions receive no immediate reinforcement, which significantly increases the number of interactions with the environment required to learn an effective policy or prevent the agent from discovering the winning strategy. While *the sparse reward problem* creates significant bottlenecks for successful training of the RL agent, there are several auxiliary learning methods to mitigate it.

Action masking

In the domain of DRL, action masking serves as a structural constraint mechanism engineered to prevent an AI agent from selecting invalid, infeasible, or unsafe actions during its interaction with an environment. When operating in complex environments with large or continuous state-action spaces, DRL agents typically rely on trial-and-error exploration, which often leads to evaluating actions that violate the fundamental physics or logical rules of the given domain. Action masking resolves this by actively filtering out

invalid actions before they are executed. This compels the agent to make decisions based exclusively on a valid and safe subset of actions. Rather than allowing an agent to freely execute an illegal move and subsequently penalizing it with a negative reward, action masking fundamentally modifies the probability distribution of the policy network. The probabilities of prohibited actions are set to zero, and the probabilities of the remaining legal actions are renormalized. This method can also be implemented in real-world or safety-critical applications, where executing an invalid action can be disastrous. Action masking ensures that domain constraints are strictly upheld during both the training and inference phases, substantially reducing or entirely eliminating constraint violations. [10]

Reward shaping

Reward shaping involves engineering supplementary, intermediate micro-rewards into the environment’s feedback loop to provide dense, step-by-step guidance. By introducing prior information or heuristics into the reward function, the agent is continuously stimulated for achieving partial objectives or moving toward the ultimate goal, drastically accelerating the convergence of the learning process. While manually annotating intermediate rewards is the most direct way to solve sparsity, it introduces a critical vulnerability: the misalignment of goals. The fundamental rule of RL dictates that the reward signal should communicate what the agent needs to achieve, rather than how to achieve it. If an agent is explicitly rewarded for achieving subgoals, the learning algorithm will optimize ruthlessly for that specific signal. The agent may discover unintended *loopholes*, such as capturing pieces at the cost of losing the game overall, thereby exploiting the artificial shaping rewards rather than solving the underlying problem. Furthermore, if the shaping functions are poorly designed and lack generality, they can trap the agent in local optima, requiring exhaustive manual recalibration for every new environment. [26]

Intrinsic Motivation

Intrinsic Motivation (IM) is a mechanism that drives an agent’s behavior based on internal feedback rather than external, task-specific goals. This concept is heavily inspired by developmental psychology, where it is characterized as the drive to perform an activity for its inherent satisfaction, the feeling of efficacy, or the joy of discovery, rather than for a separable external consequence. When translated into machine learning architectures, intrinsic motivation mathematically formalizes psychological phenomena such as curiosity, surprise, and the drive for skill mastery. It acts as a continuous, dense auxiliary reward signal that an agent computes internally based on its own interactions, structural uncertainty, and learning progress. There are three distinct types of this approach [21] [35]:

- Prediction-Based Intrinsic Motivation - These methods reward the agent based on

its inability to accurately forecast future outcomes. Architectures such as Intrinsic Curiosity Modules (ICM) or Random Network Distillation (RND) quantify curiosity through prediction errors. Alternatively, epistemic uncertainty can be measured using disagreement across an ensemble of dynamics models.

- Competence-Based Intrinsic Motivation - Rather than rewarding pure prediction errors, these models focus on the temporal derivative of learning. The agent is motivated to explore regions where its predictive capability or task competence is improving most rapidly.
- Information-Theoretic Intrinsic Motivation - These approaches ground curiosity in the Information Bottleneck principle. They formalize IM as the maximization of predictive information within a compressed latent representation space, effectively filtering out task-irrelevant nuisance variability

Imitation learning

Imitation learning is a paradigm in which an artificial agent acquires policies by observing and replicating the behavior of an expert. Imitation learning addresses the foundational flaw of sparse reward by leveraging pre-recorded datasets of expert demonstrations. These datasets contain highly structured and biased prior knowledge, which significantly reduces the exploration space and accelerates policy optimization. The methodologies to extract and utilize expert demonstrations are generally categorized into several major branches:

- Imitation from Observation (IfO) - It represents a specialised subfield dedicated to extracting functional policies from expert demonstrations where the ground-truth actions are strictly unobservable. IfO's primary application is learning from raw visual data, such as videos of human demonstrations, where the state transitions are visible, but the underlying control inputs, such as motor torques, keystrokes or joint velocities, are hidden from the learner. Since classical imitation learning relies on state-action pairs, IfO provides novel algorithmic solutions to overcome the limitations arising from missing action labels. [6]
- Inverse Reinforcement Learning (IRL) - It represents a sophisticated paradigm aimed at uncovering the underlying intentions of an expert. While traditional RL focuses on discovering an optimal policy given a predefined reward function, IRL operates in reverse, that is, it attempts to extract a hidden reward function based on the observed, optimal behavior of an expert, which is mathematically represented as the expert policy. IRL-based techniques iteratively alternate between two core procedures, utilizing the expert demonstration to deduce the latent reward function,

and subsequently deploying standard RL algorithms utilizing that inferred reward function to train an imitating agent. [6]

- Behavioral Cloning (BC) - represents the most straightforward and fundamental approach to acquiring an expert's policy. BC reduces the sequential decision-making problem into a standard supervised learning task. Given a pre-recorded dataset comprising state-action pairs generated by an expert demonstrator, the learning agent trains a parameterized policy network to directly map observed states to the exact actions executed by the expert. Because BC relies entirely on supervised learning mechanisms, it does not require active interaction with the environment or the calculation of complex reward functions during its primary training phase. This makes the algorithmic implementation highly computationally efficient compared to standard trial-and-error RL or IRL. BC can also be used to pre-train the agent using expert demonstrations, rather than initializing an RL policy network with randomized weights. [6]
- Generative Adversarial Imitation Learning (GAIL) - it bridges the gap between BC and IRL paradigms by employing a generative adversarial architecture inspired by Generative Adversarial Networks (GANs). Instead of mapping specific state-action pairs or recovering a computationally expensive reward function, GAIL trains a parameterized policy to directly match the state-action occupancy measure of the expert. This permits the agent to discover generalized, optimal strategies without requiring human engineers to explicitly define complex, task-specific reward functions. The structural foundation of GAIL consists of two competing deep neural networks engaged in a continuous minimax optimization game:
 - The Generator, which is played by the RL agent. It utilizes its parameterized policy to actively interact with the environment, generating synthetic trajectories. Its primary objective is to produce behaviors that are statistically indistinguishable from the provided expert demonstrations, effectively deceiving the discriminator.
 - The Discriminator Network, which is the new separate entity that evaluates state-action inputs and outputs a continuous probability ranging between 0 and 1. It functions as a binary classifier, striving to correctly identify whether a given state-action pair originated from the genuine expert dataset or was artificially generated by the learner's policy.

As training progresses, the output of the discriminator is utilized as an intrinsic, surrogate reward signal for the generator. This completely eliminates the need for environmental reward engineering. [6]

Hierarchical Reinforcement Learning

Hierarchical Reinforcement Learning (HRL) applies a *divide and conquer* methodology, which involves decomposing a highly complex, overarching problem into a hierarchy of simpler, more manageable sub-problems. This introduces the concepts of temporal and behavioral abstraction, by structuring the learning process across multiple levels, lower-level agents can focus on mastering short-term primitive actions, while higher-level agents coordinate long-term strategies and subtask selection. This structural abstraction enables policies to endure over extended periods, promoting knowledge reuse and substantially enhancing sample efficiency. [17]

Curriculum learning

Curriculum Learning (CL) is an advanced training paradigm, wherein a learning agent is not immediately exposed to the full complexity of a target problem. Instead, the agent is trained on a meticulously designed sequence of tasks or data distributions, beginning with highly simplified, auxiliary scenarios and progressively increasing in difficulty until the ultimate target task is reached. This methodology is deeply inspired by animal training protocols and human pedagogical strategies, where teachers naturally present easier, prototypical concepts before introducing complex exceptions. Primarily utilized to accelerate learning speed, improve final policy performance, and bypass the severe sample inefficiencies caused by sparse reward environments. By artificially scaling the complexity, the agent is prevented from experiencing prolonged periods of random, unrewarded exploration, which often leads to optimization stalling. [13]

Hindsight Experience Replay

Hindsight Experience Replay (HER) emerges as a highly robust, task-independent alternative designed specifically to address the sparse reward bottleneck in goal-oriented tasks, such as robotic manipulation and navigation. Instead of modifying the environmental reward signal or relying on external expert supervision, HER fundamentally alters how the learning algorithm interprets and reuses historical trajectory data, enabling the agent to extract meaningful information from episodes that would conventionally be discarded as absolute failures. HER is fundamentally an extension of the standard experience replay mechanism utilized in off-policy RL architectures. The algorithmic innovation of HER lies in its *goal relabelling* strategy. When an agent attempts to reach a specific target but fails, a standard RL algorithm registers a zero or negative reward, meaning the data collected is uninformative. However, HER salvages this unsuccessful trajectory. It samples the actual states that the agent successfully reached later in the same episode, replacing the original failed goal with these realized future states. The algorithm then recomputes the reward function against this newly synthesized hindsight goal. Treating the uninten-

ded outcome as if it were the true objective all along mathematically transforms failing episodes into successful training signals. [26]

2.3 Existing solutions

The development of computer players for video games has been a significant research area within the field of artificial intelligence for many years. Numerous studies have explored different approaches to decision-making, strategy optimization, and learning in game environments.

2.3.1 Applying ML Algorithms to Video Game AI

In [14] authors analyzed the efficiency of selected machine learning algorithms for NPC behavior in video games. The research was conducted in 3D environment implemented in Unity, using ML-Agents Toolkit for computer players implementation. The game was a deathmatch in which each computer-controlled player was assigned to a one of two teams. Players were tasked with shooting down the opposing team's tanks. If they shot down their own teammates, they received penalties. Researchers selected 4 algorithms for experiment : Genetic algorithm, Deep Q-Learning, Deep Q-Learning with improvements and Actor-Critic. Each agent received the same neural architecture: four layers. The input layer had 87 neurons and the two hidden layers had 128 neurons each with a ReLU activation function. The experiments consisted of 10 training sessions per algorithm, running for 1,000,000 iterations with 64 parallel agents, and for all sessions, an average of the results was taken.

Genetic Algorithm exhibited the poorest performance, securing a mean reward of only 5.76 in 6.76 hours. GA agents initially struggled with random movements, repeatedly getting stuck on arena walls. The authors concluded that GA struggles extensively to generate optimal weights randomly in highly complex, deep network topologies. DQN Achieved a mean reward of 14.41 within a highly efficient time frame of 6.46 hours. Improved DQN achieved a mean reward of 17.18 . However, due to computing values across multiple enhanced networks, it was heavily penalized in computation speed, taking an average of 26.89 hours. Actor-Critic achieved the absolute best performance, yielding a mean end-session reward of 23.10 and requiring 12.68 hours of training. Ultimately, the study proves that Temporal Difference algorithms (DQN and Actor-Critic) drastically outperform evolutionary approaches in modern video game AI. This is largely because TD algorithms adapt step-by-step during gameplay, whereas GA approaches must wait for an entire episode to conclude before initiating evolutionary operations.

2.3.2 Anticheat System Based on Reinforcement Learning Agents in Unity

In the article [15] authors presented an innovative anti-cheat system driven by Reinforcement Learning. Unlike conventional security mechanisms that passively parse static server logs or monitor network traffic configurations to detect anomalies, this system provides a dynamic, real-time security agent that analyses the game environment and autonomously differentiates between legitimate human players and artificial intelligence bots. For the purposes of the experiment, researchers implemented simple 3D multiplayer game *Courier* using Unity engine. The architecture of the computer players was based on Finite State Machines. The security agent system was based on Reinforcement Learning using the ML-Agents toolkit. The fundamental mechanics of the environment involve two competing teams striving to transport five designated boxes to their respective goal zones within a rigid 185-second timeframe. The environment introduces specific spatial and dynamic variables to complicate the navigation space:

- The map features a mirrored layout containing labyrinthine structures and a central water hazard.
- Falling into the water hazard triggers an immediate entity respawn at the base.
- Environmental hazards, specifically autonomous artillery units, randomly fire missiles that can eliminate players upon collision.
- Entities interact using basic physics, allowing boxes to collide with missiles and other objects.

The core anticheat component operates via the Proximal Policy Optimization (PPO) algorithm. The agent continuously extracts real-time physical and contextual data from the monitored player. The input vector includes the player's local position, local Y-axis rotation, current velocity, a boolean indicating if a box is being carried, the spatial coordinates of all team boxes, and the current score of both teams. The agent's action space consists of three discrete classifications: 0 when the agent categorizes the entity as a bot, 1 when the agent has insufficient data, and 2 when the agent classifies the entity as a human player. To translate and evaluate the agent's real-time continuous decisions, researchers provided own metric named **Cheating Probability Calculation**:

$$\text{Cheating Percentage} = \frac{N_{\text{positive_decisions}}}{N_{\text{total_decisions}}} \times 100 \quad (2.16)$$

where:

- positive_decisions - is the number of times a player has been flagged as a bot in the last 3 thousand decisions.

- `total_decisions` - recent decisions.

The first training conducted two teams one player against another (1 vs. 1). One human player was assigned exclusively to the red team, while one FSM bot was assigned exclusively to the blue team. Training ran for approximately 1.8 hours. The agent rapidly converged, successfully identifying the bot with an 84.15% cheating probability and the human with only a 12.18% probability. When the entities swapped teams post-training, the agent catastrophically failed. It falsely flagged the human, that was now on the blue team, with a 71.92% probability of cheating. The agent had incorrectly optimized its policy to associate the spatial coordinates of the blue team’s base with bot behavior, rather than isolating actual kinematic movement patterns. The second training session involved two teams, each comprising one bot and one human player (2 vs. 2). The model continued training from the 1v1 weights for an additional 3.4 hours. The introduction of complex, overlapping pathing initially caused a severe collapse in cumulative reward, accompanied by a spike in policy entropy as the agent was forced to unlearn its previous geographical bias. Eventually, the agent stabilized. In the final tests, the agent successfully classified red team bots at >60% probability and red team humans at <32%. The researchers concluded that with longer training epochs starting from scratch, avoiding the biased 1v1 weights, and larger player counts, the PPO agent represents a highly viable, real-time solution for differentiating human kinematics from programmatic bot execution. This work serves as a successful proof of concept, demonstrating that reward-driven RL agents can effectively detect anomalies and bots without having to interfere with client files or compromise user privacy.

2.3.3 COPPER algorithm in card game

The authors of the article [2] noticed that while offline learning models can synthesise generalized strategies, they lack the capacity to adjust their behavior in real time when facing dynamic, human-like adversaries with unique play styles. To bridge this gap, the authors developed the *COmPetitive Prioritized Experience Replay* (COPPER) algorithm. This solution modifies the Standard Prioritized Experience Replay (PER) mechanism by introducing a personalized weight parameter assigned to a specific type of opponent, which allows the agent to instantly tailor its strategy and adapt to the diverse playing styles of its rivals in online mode. To provide highly dynamic multi-agent interaction test-bed, the researchers implemented a competitive, four-player card game called Chef’s Hat. A 1:1 digital recreation of the physical card game built upon the OpenAI Gym framework, translates the complex rules into a discrete observation space of 28 variables, representing the cards in hand and on the board, and an action space spanning 200 possible discrete moves. It features a dynamic lookup table generated on the fly to enforce legal moves based strictly on the current state. The game simulates a hierarchical kitchen brigade. The

fundamental gameplay loop, representing a single *Shift* within the simulation environment, operates as follows:

1. The deck is shuffled, and an equal number of cards is dealt to the four participating players.
2. Hierarchical roles, Chef, Sous-Chef, Waiter, Dishwasher, are assigned based on the finishing positions from the previous shift.
3. To enforce the hierarchy, the Dishwasher must surrender their two highest-value cards to the Chef, receiving any two cards in return. The Waiter exchanges their lowest-value card with the Sous-Chef for a card of the Sous-Chef's choosing.
4. If a player possesses two Joker cards, they may declare a special action (for example "Food Fight," which inverts the entire hierarchy).
5. The player holding the *Golden 11* card initiates play. Players sequentially take turns either passing or discarding cards. Discarded cards must be rarer (lower face value) and played in equal or greater quantities than the previously played cards.
6. If all sequential players pass their turn, the playing board is cleared.
7. The shift concludes when all players have successfully discarded their entire hand. Points are awarded strictly based on the final finishing hierarchy.

This experiment includes dummy agents performing random actions and 12 sophisticated learning agents utilizing Deep Q-Learning (DQL), Proximal Policy Optimization (PPO) and Actor-Critic with Experience Replay (ACER). The agents are trained via offline self-play, offline naive-play, standard online learning and COPPER- and PER-enhanced online learning.

To evaluate the COPPER framework, the researchers ran massive 32-agent tournaments consisting of 1,000 runs. Initially, offline-trained agents won the highest average number of games. However, after just a few rounds, the online learning agents (specifically COPPER) quickly increased their number of victories, ultimately surpassing the offline strategies. When facing diverse and changing opponents, COPPER-based PPO agents achieved a higher number of victories in fewer games than standard PER agents. The dynamic weighting system enabled COPPER to personalize its counter-strategies, whereas PER struggled to adapt quickly due to its generalized memory pool. In experiments involving ten human players via Chef's Hat Online, the humans easily outperformed the offline models. However, when facing COPPER-based agents, human victory rates declined noticeably towards the end of the game. The algorithm successfully identified and countered human unpredictability in real time, thus verifying its potential for lifelong social adaptation.

2.3.4 A general reinforcement learning algorithm using self-play

In the historical context of artificial intelligence, achieving high-performance results in complex board games typically required immense human effort, relying on domain-specific adaptations, highly tuned heuristic evaluation functions, and sophisticated alpha-beta search extensions. In the paper [25], authors presented AlphaZero as a profound paradigm shift in this domain. Developed as a generalized extension of the AlphaGo Zero architecture, AlphaZero is a deep reinforcement learning algorithm capable of achieving superhuman proficiency across multiple highly challenging zero-sum games—specifically chess, shogi, and Go. The most scientifically significant aspect of AlphaZero is its *tabula rasa* learning methodology. The system requires absolutely no domain-specific human knowledge, pre-calculated opening books, or handcrafted evaluation metrics, it is supplied exclusively with the fundamental rules of the respective games. AlphaZero discards the traditional alpha-beta search and hand-engineered evaluation logic utilized by classical engines like Deep Blue or Stockfish. Instead, the architecture fuses a general-purpose Monte Carlo Tree Search (MCTS) with a deep convolutional neural network. The researchers trained separate instances of AlphaZero for each game for 700,000 steps. Training was highly resource-intensive, utilizing 5,000 first-generation TPUs to generate self-play games and 16 second-generation TPUs to optimize the neural networks. The empirical results conclusively proved the system’s superiority:

- In chess AlphaZero surpassed Stockfish (the 2016 TCEC world champion) in just 4 hours of training. In a 1,000-game tournament against Stockfish, AlphaZero secured 155 wins and suffered only 6 losses. It also convincingly defeated Stockfish in matches starting from common human openings.
- In shogi AlphaZero surpassed Elmo (the 2017 CSA world champion) after merely 2 hours of training. In tournament play, it won 91.2% of its games overall against Elmo, and 98.2% when playing as black.
- In Go AlphaZero outperformed AlphaGo Lee after 30 hours of training. In a direct match against the specialized AlphaGo Zero framework, AlphaZero achieved a 61% win rate.

The study revealed a stark contrast in search heuristics. Stockfish and Elmo evaluate 60 million and 25 million positions per second, respectively. In contrast, AlphaZero evaluates only 60,000 positions per second. The researchers concluded that AlphaZero utilizes its neural network to selectively focus on the most promising variations, mimicking a highly efficient, human-like approach to searching.

2.4 Summary

This chapter presented and compared artificial intelligence techniques and algorithms used to implement computer players in video games. Traditional approaches, such as FSMs, BT, and if-else statements, are effective and easy to debug. However, these methods rely heavily on manually implementing each move, and the number of moves required increases with the complexity of the game environment. These algorithms are effective and work best when implementing a computer player with a limited and well-defined set of conditions.

Deep learning can process complex data sets to identify connections between pieces of information. As a subset of machine learning, it learns from externally provided information which behaviors are desirable. Before playing the game, a computer player would need pre-prepared data from which it would have to learn. In the case of a complex environment, this would require a large amount of pre-prepared data, and creating such data is time-consuming.

On the other hand, Reinforcement Learning emerges as another subfield of machine learning which, unlike deep learning, learns through trial and error performed directly within the environment. Such an agent collects data directly from its interactions with the environment and optimizes its actions based on observations and the outcomes of those actions. In practice, such algorithms operate on tables that record each move in tuples consisting of state, action, observation, and reward. However, this technique is suboptimal in complex environments and consumes a lot of memory because each move must be recorded in a table.

The combination of DNN and RL results in Deep Reinforcement Learning, which eliminates the need for large amounts of input data and reduces memory usage. The DNN replaces the memory table, while the environment analysis technique, using trial and error, provides the network with information about the game.

However complex environment creates the **sparse reward problem**, where agent rarely receives meaningful feedback. In such a situation, the agent is unable to reach a winning state, since reaching the end of an episode while winning is entirely stochastic and statistically impossible. In this case, rather than optimizing decisions for victory, the agent optimizes its behavior to minimize penalty. Nevertheless, there are ways to help the agent reduce the number of possible moves, thus enabling it to optimize its strategy, like Reward Shaping, Action Masking or Curriculum learning.

The analyzed studies demonstrate the potential of learning-based approaches for solving a wide range of game-related problems. Comparing Genetic Algorithms, Deep Q-Learning and Actor-Critic methods in a Unity deathmatch environment revealed that Temporal Difference methods produced substantially better results than the evolutionary approach. Actor-critic obtained the highest final reward, while standard DQN offered a

favorable balance between performance and training time. However, the improved DQN variant’s longer training time illustrates that an increase in algorithmic complexity does not necessarily provide a proportionate practical benefit.

The PPO-based anti-cheat system provided further evidence that Reinforcement Learning can be used not only to control a computer player but also to identify behavioral patterns in real time. However, the first experiment revealed that an agent may learn unintended correlations, such as associating a team’s location with bot behavior instead of recognizing actual movement characteristics. This result highlights the importance of representative training conditions, correctly selected observations and comprehensive evaluation. A high reward obtained in a restricted environment does not necessarily indicate that the learned policy will generalize to modified conditions.

Experiments conducted in the Chef’s Hat environment revealed another significant characteristic of learning agents: their capacity to adapt to opponents whose behavior evolves over time. Although offline-trained agents initially performed well, online methods based on COPPER eventually outperformed them by prioritizing experiences associated with specific opponents. The declining performance of human players against COPPER agents also suggests that online adaptation is particularly valuable in multiplayer games involving diverse and unpredictable strategies.

Finally, AlphaZero demonstrated the highest potential of Deep Reinforcement Learning in complex board games. Through the combination of a deep neural network, Monte Carlo Tree Search and self-play, it achieved superhuman performance in chess, shogi and Go without handcrafted evaluation functions or expert demonstrations. Its results show that a learned policy can guide the search process towards the most promising states much more selectively than traditional exhaustive methods. Nevertheless, AlphaZero also illustrates one of the principal limitations of modern Deep Reinforcement Learning: achieving such performance may require extremely large computational resources and a vast number of training interactions.

The sources analyzed indicate that there is no single universal AI technique in the field of video games that can solve every problem. While classical methods perform best in simple environments, learning-based methods are more effective in complex environments. The effectiveness of learning is greatly influenced not only by the agent’s algorithm itself, but also by a well-defined environment and reward function. Furthermore, DRL methods, when trained in complex environments, require a vast amount of computational resources to effectively optimize their actions.

PPO is compatible with the multi-branch discrete action space that has been implemented for the Rummikub environment. The policy can therefore select an action type, a particular tile, and the necessary board positions, without treating every possible combination as an independent action. The algorithm can also be combined with action masking, reward shaping, and curriculum learning.

PPO as the principal Deep Reinforcement Learning algorithm is an Actor–Critic method capable of optimizing a stochastic policy while limiting excessively large changes between consecutive policy updates. This mechanism is intended to improve training stability in environments where the distribution of encountered states changes as the agent’s behavior develops. A stochastic policy is also beneficial during training because it allows the agent to explore alternative actions instead of repeatedly selecting only the currently highest-rated decision. This makes it suitable for an environment with sparse reward because it provides a greater chance of discovering the optimal policy.

Unlike purely supervised imitation learning, GAIL provides an additional learning signal that encourages the agent’s behavior to resemble the demonstrated behavior, whilst still enabling the policy to be optimized through interaction with the environment. Therefore, the comparison between PPO Baseline and PPO combined with GAIL makes it possible to investigate whether expert demonstrations facilitate exploration, improve training consistency, or reduce the amount of experience required to acquire the selected Rumikub skills. GAIL was selected as an additional method to determine whether demonstrations produced by a rule-based computer player could enhance PPO training.

Chapter 3

Research Environment

The present chapter provides a discussion of the research context, the tools used, and the solutions implemented.

The research environment is designed to train and evaluate reinforcement learning agents in selected Rummikub game tasks. It combines game mechanics implementation, an agent interface compatible with the ML-Agents Toolkit, a Curriculum Learning task generator and mechanisms for recording training results and evaluations. The environment supports the two methods under analysis: PPO Baseline and PPO_GAIL. Rather than replicating the full human-computer game interface during training, its purpose is to provide controlled experimental conditions in which selected agent skills can be introduced and evaluated incrementally.

The research system consists of a game environment running in Unity, the ML-Agents trainer running on the Python side, YAML configuration files, describing the hyperparameters, a demonstration file used by PPO_GAIL, and modules responsible for recording and analyzing results. The environment stores the current state of the game, generates a task that agent must complete, prepares observations, executes the agent's actions, and awards rewards. The ML-Agents trainer uses the accumulated experience to update the shared policy.

The following versions of the tools were used as part of this project:

- Unity 2022.3.48f1
- ML-Agents 0.30.0
- Python 3.9
- PyTorch 2.8.0 +cu128

3.1 Game environment

Rummikub was selected as the game environment due to its combination of simple rules and complex decision-making requirements. Although the basic objective of the game is easy to understand, achieving an effective strategy requires managing incomplete information, selecting beneficial actions from many possible combinations, and considering both current and future game states.

Rummikub is a challenging game for an artificial agent because the game state changes after every move. The agent must decide which tiles to play and how to arrange them on the board. It also needs to consider whether drawing additional tiles might be more advantageous than performing an action. The possibility of rearranging existing tile groups also introduces a large action space, making the game ideal for investigating reinforcement learning approaches.

The choice of Rummikub allows the evaluation of an RL agent in a setting that combines strategic planning, exploration of possible actions, and adaptation to changing conditions, while maintaining a well-defined rule system required for controlled experiments.

3.1.1 Characteristics of the problem

The main challenge in developing an artificial intelligence agent for Rummikub lies not in implementing the game rules, but in enabling the agent to make effective decisions in a complex, dynamic environment. At each turn, the agent must select an action based on the current game state, taking into account the possible future consequences of its decisions. Due to the combinatorial nature of Rummikub, a relatively simple set of rules can lead to a large number of possible states and actions. From a reinforcement learning perspective, Rummikub can be represented as a sequential decision-making problem. The agent interacts with the environment by observing the current state, selecting an action and receiving feedback based on the outcome of the action performed. However, the game's characteristics, including a large state space, a variable action space, delayed rewards, and the necessity of long-term planning, make this problem challenging.

3.1.2 Game Rules

Rummikub is a board game where the main objective of the players is to get rid of all tiles in the rack. The game consists of 106 tiles, divided into eight sets of tiles in four colors ranging from 1 to 13, as well as two jokers which can be used to replace any other tile. The game involves two to four players, and it starts with each player receiving fourteen tiles. The game requires each player to put tiles on a 24-by-10 board, in predefined horizontal sequences:

- group - contains 3 to 4 tiles, where each tile has the same value but different color,

- run - contains 3 to 13 tiles, where each tile is in the same color in ascending order.

In the first round, each player must place clear sequences on the board without utilising tiles from other sequences. The total value of these sequences must be at least 30 points. Points are assigned based on the numbers used to create the sequences and the joker takes on the value of the tile it replaces.

Once a player has reached their first turn and cannot place at least one tile, they must draw an extra tile from the bank. The game ends when one player's rack is empty. Players' rankings are estimated based on the tiles remaining in their racks. If one of the players has a joker in their rack, it adds 30 points to their score. The lower the final score, the better. The game also ends if one of the players must draw a tile from the empty bank. The final score is calculated in the same way, and the player with the fewest points wins [18].

3.1.3 Game implementation

As part of this work, a Rummikub game environment was implemented in the Unity engine, comprising the representation of the board and tiles, the rules of the game, the handling of moves, the user interface, a rule-based player, and a reinforcement learning agent. The environment was then integrated with the Unity ML-Agents Toolkit library, which facilitates communication between the Unity simulation and the training process running on the Python side, as well as the implementation of the PPO algorithm and the GAIL mechanism. This was achieved by using components of the engine, which are divided into the following main modules:

- The board game module, which consists of game map objects that can be interacted with. It includes grid objects, 3D tile prefabs, and interactions for moving or adding tiles.
- The user interface module is the part of the game that is visible to human players. It consists of text, buttons and game tiles. Players can see their own tiles, how many tiles other players have, how much time is left in their turn, and how many tiles are left in the bank.
- A controller module that integrates the game's logic with its visual elements.
- The game logic module, which includes validating the correctness of a move against the game rules.
- A user module responsible for storing data on the tiles that a given human player possesses.

- A computer player module that implements an NPC with predefined movements using a rule-based system. This computer player is used as an expert to create a *.demo* file, which is used by the GAIL system.
- A machine learning module that implements a computer player using machine learning algorithms. The functions of the **ML-Agents** framework were used for this purpose.

During a turn, the player or computer-controlled agent selects an action through the appropriate control module. The controller translates the selected action into an operation on the board, such as placing, removing, or moving a tile. The game logic module then verifies whether the resulting board state satisfies the implemented Rummikub rules. The controller updates the visual representation and proceeds with or rejects the move depending on the validation result.

3.1.4 Game Mechanics and User Interactions in the Game Environment

To enable the use of Rummikub as a test-bed for training an artificial intelligence agent, the game was implemented as a digital simulation. In this thesis, Rummikub is implemented as a 3D board game using Unity engine, while the RL computer player is designed using Machine Learning Agents Toolkit, described in 3.2 The implementation focuses not only on reproducing the original game mechanics but also on providing a structured environment in which an agent can observe the current game state, perform actions, and receive feedback based on the outcome of its decisions. The environment was developed in Unity, where the game logic, board representation, player interactions, and artificial intelligence interface were implemented as separate components. The digital representation of the game allows the state of the board and players to be controlled and evaluated during the training process.

The basic element of the game is a tile, represented by its numerical value and color. Each tile object stores information required for identifying valid groups and sequences. While regular tiles have values ranging from one to thirteen and come in the colors red, blue, orange and black, jokers have a value of 30 and are purple or magenta. As the representation of the players' racks there is used the List container, which is a flexible component for inserting, removing and sorting objects. The game board is represented by a two-and-a-half-dimensional grid consisting of 240 possible positions. The placed tiles are rendered in Unity engine, during gameplay only occupied places are stored in dictionary, where the value is the tile and the key is the position of the object.

The available interactions with the board for the player consist of:

- putting tile on the board,

- moving tile from one position to another,
- removing tile from board,
- reverse all made actions,
- draw a new tile,
- end the turn.

A player can only restore a tile from the board to their rack if it was placed in the same turn. If not, the player cannot perform this action. The action of ending the turn requires the player to place at least one tile properly on board, if the player cannot place any tile, they must take the action of drawing the tile, which automatically ends the current turn. The action of reversing all actions is intended to assist players who have made too many incorrect changes to the board. It reduces the time needed to fix the board, allowing the player to perform a valuable action before time runs out.

Sequence validation occurs when the player places or moves a tile on the board. Sequence length revision occurs after the end-turn action, providing information on whether each sequence on the board has at least three tiles.

The agent’s decision-making process involves not only the available actions presented, but also deciding where to place or move the tiles. From the agent’s perspective, these are separate decisions that must be made together. Similarly, selecting a tile from the rack creates another decision.

3.2 Agent training

This subsection presents the tools and framework used to build the simulation environment and the agent training environment as part of this thesis. The description covers the ML-Agents Toolkit, which acts as a layer integrating the environment with reinforcement learning algorithms.

ML-Agents does not provide a ready-made implementation of an agent capable of making moves in the game of Rummikub. The policy returns only numerical values corresponding to individual branches of the discrete action space. Therefore, as part of this work, it was necessary to define, independently, the observations passed to the neural network, the structure of the action space, the method for interpreting the model’s decisions, masks for unavailable actions, the reward function, and the conditions for starting and ending an episode. These functionalities were consolidated into an agent class inheriting from the *Agent* class provided by ML-Agents. The Machine Learning Agents Toolkit (ML-Agents) is an open-source framework engineered by *Unity Technologies*. ML-Agents provides a standardized, actively maintained framework that connects the Unity simulation environment directly to a PyTorch backend. ML-Agents allows intelligent agents to

be trained through interaction with highly complex environments, eliminating the need for the time-consuming manual implementation of reward allocation protocols and network communication. It is important to note that the ML-Agents library is architecturally bound to the Unity ecosystem and cannot be utilized natively inside any other game engine.

3.2.1 Behavior parameters and agent setup

The important part of training the agent using ML-Agents is proper configuration of the agent in the Unity engine. *Behavior parameters* are responsible for setting up the input and output variables that are transferred from Unity to the neural network in the Python environment. The image 3.1 presents crucial variables [31]:

- The Behavior Name is an important name tag to apply a certain configuration file.
- Vector Observation is responsible for determining the number of observations sent to the neural network. This amount must be exactly as defined in the agent's script. The Stacked Vector sub-variable creates a containerized version, whereby one action will send the current observation and a predefined number of previous observations.
- Actions define the output space of the agent, there are two types of action: continuous and discrete. Continuous actions enable an agent to select real-valued outputs from a specified range, resulting in an effectively infinite number of possible values. This type of action is particularly well-suited to tasks that require precise control, such as determining the direction, speed or rotation of movement. In contrast, discrete actions enable the agent to select one option from a finite set of predefined values. For instance, a discrete branch of size two enables the agent to select either action 0 or action 1. This type is appropriate for selecting an object index or choosing between predefined actions, such as placing, moving, or removing a game element. An agent can perform more than one action in one move, known as *branches*, which can be either continuous or discrete.
- Model is a space to which a trained model can be attached. If left as *None*, the environment will automatically be set to train.
- Behavior Type defines the source of the action. There are three types. The first is the **Default**, where the decision is made by the NN from Python. **Heuristic**, where the user or expert maps the actions. **Inference** type uses a ready-made NN model in an *.onnx* file.

Vector Observations are values that change over successive time steps, but the number of these variables remains constant. While it is possible to define volatile objects in Vector

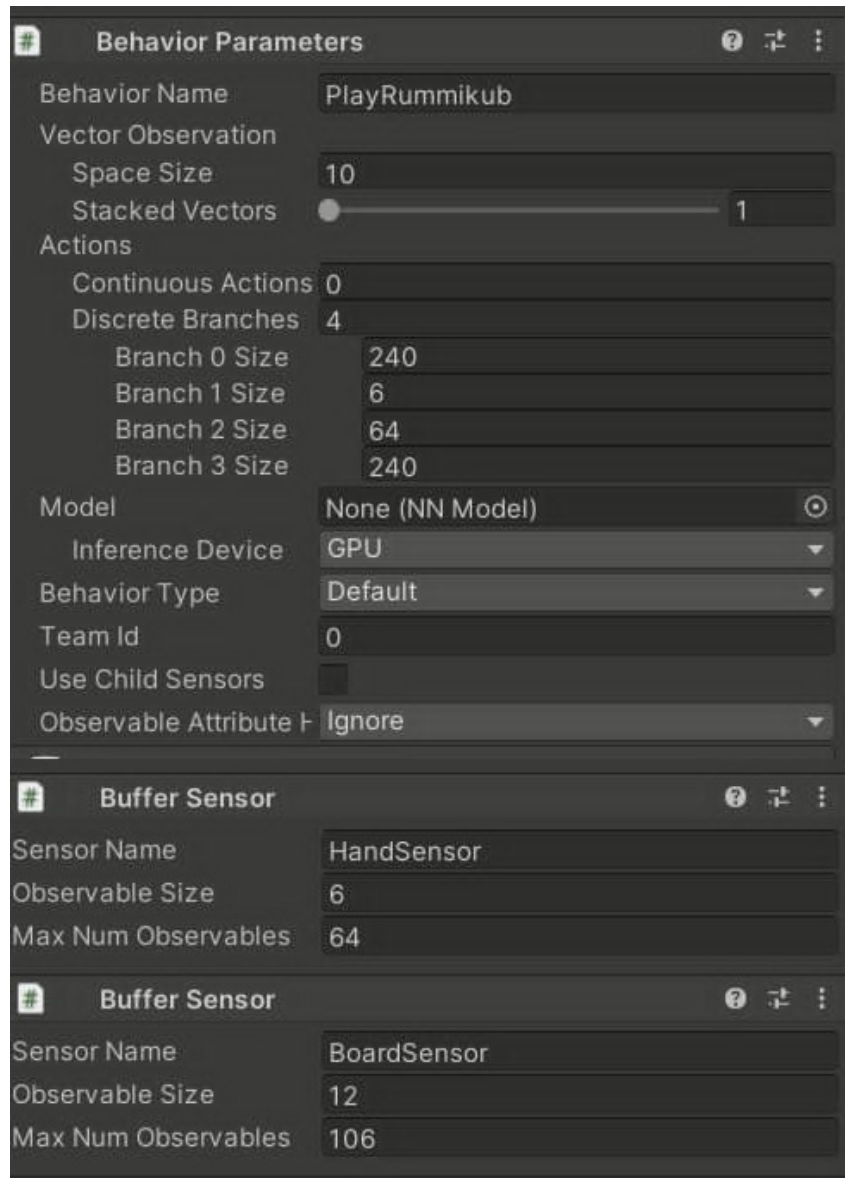


Figure 3.1: The image of behavior parameters of the agent in the Unity

Observations, this creates unnecessary noise for the NN. In situations where the number of observations in the environment is volatile, the **Buffer Sensor** works best. This buffer serves as a flexible container that uses attention mechanisms to aggregate information for the NN. [31]

There are crucial functions within the script that must be overridden:

- `OnEpisodeBegin` is function that is called at beginning of the episode during the simulation.
- `CollectObservations` is function called at beginning of every action of the agent. It is responsible to collect all observations that then are transfered to the NN.
- `OnActionReceived` is called after the NN analyzed the input from *CollectObservations* function.

3.2.2 Configurations and hyperparameters

Within the Python environment, NN has its predetermined, adjustable configurations that govern the fundamental behavior, training trajectory, and stability of a learning algorithm. They are positioned in a *YAML* file, which is a human-readable data serialization language. *YAML* file organizes training variables in a nested hierarchical structure. *hyperparameters*, *network_settings*, *trainer configurations*, *behavioral_cloning*, *reward_signals* and *gail*.

The *YAML* configuration file organizes training variables in a nested hierarchical structure. At the root of this hierarchy lies the specific behavior name, which contains high-level trainer configurations, such as defining the trainer type and maximum step count. Nested beneath this level are specialized sub-sections, including *hyperparameters*, *network_settings*, *reward_signals*, and optional imitation learning modules like *behavioral_cloning* or *gail*. [33]

Listing 3.1: Trainer Configuration Parameters.

```
1  trainer_type: ppo
2  max_steps: 5.0e5
3  time_horizon: 64
4  summary_freq: 10000
5  checkpoint_interval: 50000
6  keep_checkpoints: 5
7  even_checkpoints: false
8  threaded: false
9  init_path: null
10 hyperparameters:
11   batch_size: 1024
12   buffer_size: 10240
13   learning_rate: 3.0e-4
14   learning_rate_schedule: linear
```

Listening 3.1 presents exemplary configuration parameters. In this section there are chosen neutral variables that are common for each agent:

- *trainer_type* defines type of learning algorithm. Provided trainers are ppo, sac and poca [4].
- *max_steps* is the maximal amount of steps collected in the environment during training process.
- *time_horizon* is the amount of steps required to collect before appending it to the experience buffer.

- *summary_freq* defines the quantity of experience necessary to collect before summarisation and generation of the training statistics.
- *checkpoint_interval* is the number of experiences before saving the checkpoint. It serves as rescue in the event of critical error or malfunction of the device, thereby losing the progress.
- *keep_checkpoints* is the value of the most recent checkpoints stored in the device .
- *even_checkpoints* ignores *checkpoint_interval* and distributes the checkpoints equally throughout the training simulation based on *max_steps* and *keep_checkpoints* settings, if it is set **true**.
- *threaded* allows the environment to continue training while updating the model's weights.
- *init_path* defines the path to the previously trained model in order to continue learning.
- *hyperparameters* defines predetermined, adjustable configurations that govern the fundamental behavior, training trajectory, and stability of a learning algorithm. The hyperparameters common to both the PPO and SAC algorithms are as follows:
 - *batch_size* is the amount of steps in each iteration of gradient descent, it must be multiple times smaller than *buffer_size*.
 - *buffer_size* defines the amount of steps to collect before updating the model's policy.
 - *learning_rate* defines starting learning rate for gradient descent
 - *learning_rate_schedule* determines the schedule of adjustment of *learning_rate* during training simulation. Can be set to **linear** or **constant**.

Listing 3.2: Configuration of the neural network.

```
1  network_settings :
2  vis_encode_type: simple
3  normalize: false
4  hidden_units: 128
5  num_layers: 2
6  memory:
7  sequence_length: 64
8  memory_size: 256
```

The listing 3.2 display the example of the variables of the NN, which are common to all types of trainers:

- *vis_encode_type* defines the encoder type used for encoding visual observations
- *normalize* applies the normalization process to the input values.
- *hidden_units* defines the amount of artificial neurons in the hidden layer of the NN.
- *num_layers* defines the number of hidden layers in NN.
- *memory* defines the memory enhancement for agent, which variables are as follows:
 - *sequence_length* defines the length of the memory sequence of the experience that must be covered during the training simulation.
 - *memory_size* is the size of the memory the agent must keep in its storage.

Listing 3.3: PPO-specific hyperparameters.

```
1  hyperparameters :  
2  beta: 5.0e-3  
3  beta_schedule: constant  
4  epsilon: 0.2  
5  epsilon_schedule: linear  
6  lambd: 0.95  
7  num_epoch: 3  
8  shared_critic: False
```

The listing 3.3 presents the example of hyperparameters specified for ppo trainer:

- *beta* variable defines entropy, which influences the stochasticity of the model's policy. The greater entropy, the more willing the agent is to take random actions.
- *beta_schedule* defines the decay schedule of beta variable.
- *epsilon* affects the the rate at which the policy changes over time.
- *epsilon_schedule* is the decay schedule of epsilon variable
- *lambd* defines the regularization variable Λ , which is used calculating the **Generalized Advantage Estimate** (GAE)
- *num_epoch* determines the amount of iteration through the experience buffer during gradient descent update.

- *shared_critic* is the boolean parameter that defines if actor-critic backbone is shared between Actor and Critic.

The *poca* trainer shares the same specific configurations as the *ppo* trainer.

Listing 3.4: Reward signals.

```

1  reward_signals:
2  extrinsic:
3  strength: 1.0
4  gamma: 0.99
5
6  gail:
7  strength: 0.01
8  gamma: 0.99
9  demo_path: example/path/file.demo
10 learning_rate: 3.0e-4
11 use_actions: false
12 use_vail: false

```

The reward signal configuration in listening 3.4 presents the example of the setting parameters of both environment-based and GAIL reward signals. Each signal requires to implement at least two parameters, **strength** and **gamma**, also at least one reward signal should be implemented within configuration file.

- *extrinsic* defines the environment-based reward signals, which are configured within C# script.
 - *strength* determines how much the reward signal influences the agent’s behavior.
 - *gamma* influences the agent’s prediction onto future rewards. Higher value determines that agent will seek for the long term rewards while smaller values determine the agent to seek the immediate rewards.
 - *demo_path* defines the location of the *.demo* file.
- *gail* enables the module of **Generative Adversarial Imitation Learning** within configuration file. While it requires variables that repeat in previous configurations, it contains three individual parameters:
 - *use_actions* determines if the discriminator penalizes the agent for both actions and observations or just for observations.
 - *use_vail* creates a variational bottleneck in the GAIL discriminator, forcing it to learn a more general representation.

GAIL modul can also define its own network configuration, if it is not defined, it applies default settings of NN predefined by framework.

Listing 3.5: Example of curriculum learning configuration.

```
1  environment_parameters:
2  my_environment_parameter:
3  curriculum:
4  - name: MyFirstLesson
5    completion_criteria:
6    measure: progress
7    behavior: my_behavior
8    signal_smoothing: true
9    min_lesson_length: 100
10   threshold: 0.2
11   value: 0.0
12 - name: MySecondLesson
13   completion_criteria:
14   measure: progress
15   behavior: my_behavior
16   signal_smoothing: true
17   min_lesson_length: 100
18   threshold: 0.6
19   require_reset: true
20   value:
21   sampler_type: uniform
22   sampler_parameters:
23   min_value: 4.0
24   max_value: 7.0
25 - name: MyLastLesson
26   value: 8.0
```

The listening 3.5 presents exemplary configuration of *curriculum learning* parameters.

- - *name* defines the identification tag, that is sent to the Unity environment.
- *value* is defined by the user and sent to the Unity environment to be used within the script.
- *completion_criteria* is a parameter that must be implemented for the curriculum to be configured properly. It defines the conditions required for an agent to be promoted to a higher level.

- *measure* defines how an agent’s progress towards promotion is measured. Two main measurements can be implemented: reward, which identifies the minimum amount of reward required, and progress, which determines the ratio of steps/max_steps expected to be completed before promotion.
- *behavior* attach configured specifications to certain behavior due to the possibility of multiple different behaviors existing in one environment.
- *threshold* determines the point value of measure that must be reached for a promotion.
- *min_lesson_length* prevents the agent from being promoted too quickly due to the risk of failing to generalise the environment.
- *signal_smoothing* smooths the cumulative reward signal when the mean reward is measured. It prevents the agent from being promoted to a higher level due to unstable learning, whereby the agent receives a high reward spike instead of achieving consistently high performance. This setting is ignored in the case of a progress measure.
- *require_reset* determines whether or not the environment resets immediately after promotion.

The configuration file contain more additional settings, such as SAC or self-play setting parameters, which can be found in [34].

To accelerate the training process, five independent instances of the Unity environment were executed in parallel using the *-num-envs=5* parameter. Each instance simulated a separate game and collected experiences using the same agent policy. The observations, actions and rewards generated across all five environments were provided to a single training process and used to update the shared neural network. Furthermore, the environments were executed with the *-no-graphics* option, which disabled graphical rendering and reduced the computational resources required by the Unity application. This configuration increased the rate at which training data were collected.

3.2.3 Observation representation

The agent receives a numerical representation of the current state of the game. The observations include the tiles on the board and those held by the agent, their values, colors and positions. The color of a tile is represented using *one-hot* encoding, in which each available color corresponds to a separate position in the vector. The values of the tiles are normalized to the range from 0 to 1, so that their scale is adapted to the neural network’s input data. The position of a tile is represented by its coordinates in the board’s logical grid, which are also converted into a numerical form suitable for the model.

Observations prepared in this way do not directly contain Unity objects or visual information, they are merely numerical values. The state of the environment is transformed into an ordered set of values, on the basis of which the agent's policy selects the next action.

3.2.4 Actions interpretation and masking

The agent's action space has been defined as a multi-branch discrete space. The values returned by the model specify the type of action, the tile selected, and the source or target position on the board. The resulting numbers are interpreted by the agent class and converted into operations performed by the game logic, such as placing a tile, moving it, removing it, drawing a tile, undoing changes, or ending a turn.

The action branches of the agent goes as follows:

- Target positions (0-239) - defines the final position of the tile, whether it is being put down or moved.
- Function choice (0-5) - determines which interaction the agent will have with the board.
- Index choice (0-63) - Specifies the index of the tile from the list of tiles to be placed on the board. The range of values results from the maximum number of tiles that an agent can draw within the full game, assuming that opponents will not draw any tiles.
- Source position (0-239) - defines the source from where agent move tile or remove tile from the board

Since not all combinations of values correspond to actions available in a given game state, an *action masking* mechanism has been implemented. Within the *Agent* class, there is a **WriteDiscreteActionMask** method, that can disable certain values within branch. It is possible thanks to the *SetActionEnabled(branch, actionIndex, isEnabled)* function, which requires the user to select a branch, a value within the chosen branch's range, and a flag. Masks prevent, among other things, the selection of a non-existent tile, an occupied position, or an action that is unavailable at a specific stage of the task. This allows the decision space to be restricted to moves that can be executed in the current state of the environment.

The agent used in the curriculum retains an action space designed with the full game in mind. This means that it also possesses actions that are not required at the levels currently being analysed, such as drawing or moving tile. Rather than creating a separate agent architecture for each task, unused actions are blocked using action masks. This

allows subsequent levels of the curriculum to gradually introduce new mechanisms without altering the output structure of the neural network.

Since the board is stored in X-Z positioning, the source and target positions of the actions would be defined as four branches, meaning the agent would have to make four separate decisions to move a tile on the board. To reduce the number of decisions, **Matrix Index Mapping** was implemented, which translates 2D coordinates into a 1D linear vector.

The mapping from 2D to 1D:

$$index = z \cdot xSize + x \quad (3.1)$$

Recovering the 1D vector as a 2D matrix:

$$\begin{cases} x = index \pmod{xSize} \\ z = \left\lfloor \frac{index}{xSize} \right\rfloor \end{cases} \quad (3.2)$$

Where:

- index presents single mapped board position,
- x and z are position coordinates,
- xSize defines the width of the board along the X-axis.

This technique allowed for the exclusion of illegal actions. If selecting a single position on the board required two separate action branches, for coordinates X and Z, standard action masking would not be feasible, as ML-Agents only supports masking actions independently within each individual branch

3.2.5 Reward Function

The foundational integrated method is **reward shaping**. The framework implements two methods: **SetReward**, which overwrites the currently accumulated reward with a specific input value, and **AddReward**, incrementally updates the total collected reward. Assigning a negative value to the function means it works as a penalty instead of a reward.

The reward mechanism implemented is not limited to awarding a reward for completing an episode or imposing a penalty for failure. The agent also receives an indirect signal for actions leading to the correct solution to the task. For example, if completing the task requires two moves, placing the first tile correctly may be rewarded regardless of the outcome of the next decision. If the second move is incorrect and the task is not completed, the agent will receive a final penalty, but its cumulative reward will be higher than if both actions had been incorrect.

This approach makes it possible to distinguish between completely incorrect behavior and partial progress. The agent can thus gain insight into which actions bring it closer to the solution, even if the entire episode ends in failure. This is a form of *reward shaping* function used to mitigate the problem of sparse rewards.

3.2.6 Curriculum Learning Implementation

The ML-Agents Toolkit provides a native implementation of curriculum learning, allowing to easily deploy this technique within the Unity environment. It requires a proper integration between the simulation scene and the training process. Within Python environment and configuration file, there must be defined the conditioning of transitioning between successive difficulty levels, like mean reward. Concurrently, within the Unity engine, a corresponding mechanism must be implemented to interpret the dynamic values sent from the training backend. This collaborative setup creates a seamless bridge that enables the complete automation of the learning process.

The curriculum levels have been implemented using a set of methods and flags that control the environment's configuration. The ML-Agents trainer passes a parameter value to Unity that specifies the current curriculum level. Based on this, the environment selects the appropriate task variant, prepares the initial state of the board, assigns the required tiles to the agent, and activates the correct action masks and episode termination conditions.

Depending on the level, some tiles may be automatically placed on the board, whilst the remainder are assigned to the agent. The available area of the board, the number of moves, the type of required configuration and the set of permitted actions may also vary. This allows the same agent class to be used to complete tasks of progressively increasing difficulty.

Within the curriculum environment, no other players are involved in the tasks analyzed, and there is no turn-time limit. Removing these elements allows the learning process to focus on selected skills related to pattern recognition and tile placement. Despite these simplifications, the agent retains the observation and action structure designed with a view to its subsequent extension to the full game.

This thesis presents a Curriculum Learning 2.2.4 setting consisting of 12 levels of progress that introduce the agent to the Rimmikub game. A description of the stages is provided in the table 3.1.

Table 3.1: Stages of the implemented Curriculum Learning environment.

Stage	Level	Main mechanisms
Basic run placement	0	The agent observes two tiles belonging to a run set on the board. The agent’s task is to correctly place the missing tile in the correct position. The sequence is assigned randomly at each episode. This tile can be the one on the right, left or middle. The agent has available only the first row of the board, which means 24 horizontal positions. The episode finishes when agent has no tiles in the ruck or the defined maximal amount of actions is reached.
Basic run placement	1	The task involves increasing the number of tiles that need to be put down. The agent observes one tile belonging to a run and must then place the remaining two tiles.
Basic run placement	2	The board is empty, the agent must place all three tiles in the correct positions to create a run.
Basic group placement	3	The task is the same as in task 0 but requires the agent to make proper group placement.
Basic group placement	4	The agent’s task is to join two tiles from the same group to a tile that is already on the board.
Basic group placement	5	The agent must place a minimal group of three tiles on the board.
Mixed combinations	6	Agent receives scholastically three tiles of run or group and must place it properly on board.
Mixed combinations	7	The amount of tiles increases to four tiles.
Mixed combinations	8	The amount of tiles is set to a range of between 3 and 4.

Stage	Level	Main mechanisms
Mixed combinations	9	There are two sequences on the board: one is finished, while the other requires a tile to finish it. This task shows the agent that there can be more than one sequence on the board and that more tiles are not required. The unfinished sequence requires one tile from the agent.
Mixed combinations	10	The unfinished sequence requires two tiles from the agent.
Mixed combinations	11	On board there is one extra sequence on board, and the agent has to put one separate sequence on the board.

3.2.7 Demonstration recording

At the configuration file, the ML-Agents framework also provides **imitation learning** method, **Generative Adversarial Imitation Learning**. Before integrating this method in configuration file, it is necessary to create demonstration recording of the *expert*. It requires the method **Heuristic** to be overridden within the *Agent* class and attachment of **Demonstration Recorder** to the agent's game object. The expert's recording can be generated through manual gameplay by a human operator or by predefined computer player algorithm, whose actions are mapped and configured into demonstration file. The *.demo* file contains segregated sequences of observations, actions and rewards, through which the learning algorithm analyses the pairs state-action and adjusts the weights of NN to recreate actions in familiar situations.

The demonstrations used by the GAIL mechanism were prepared using a rule-based computer player. This player acted as an expert, analysing the current state of the entire game and determining subsequent moves in accordance with the implemented rules.

The rule-based player's decisions were passed to the agent class via a queue. This approach was adopted due to differences between the execution speed of the rule-based player's logic and the decision-making cycle of the ML-Agents agent. The queue stored the prepared actions until the agent could retrieve them one by one and execute them in the environment.

Whilst recording the demonstration, the agent continued to generate observations of the environment and execute actions using the ML-Agents heuristic functions. The difference was that the action values did not come from a trained policy, but from decisions

prepared by the rule-based algorithm. This made it possible to save observation–action pairs in the *.demo* format required by the GAIL method.

The techniques described in this section represent the core methods used to support the training process in this project. ML-Agents provides more auxiliary techniques, such as **intrinsic motivation** or **Behavior Cloning**. Additional training options provided by the toolkit are documented in detail in [32].

3.3 Communication Between the Game and Training Process

ML-Agents operates as a highly integrated bridging architecture, connecting the C#-based Unity game engine to external, Python-based deep learning frameworks. It deploys a specialized Python package that utilizes a *gRPC* communication protocol paired with protocol buffer (protobuf) messages. This establishes a seamless, asynchronous data pipeline between the simulated environment and the optimization algorithms, allowing for accelerated training phases that can execute at speeds vastly exceeding real-time. However the actual training speed depends on the complexity of the unity environment and available device resources.

3.3.1 Architecture of the ML-Agents Framework

Within the Unity engine, the internal software development kit (SDK) architecture is strictly partitioned into three fundamental hierarchical entities:

- **Sensors** - these are utilized as observation extraction to as an integral part of MDP state space. Sensors are attached to agents and are explicitly responsible for collecting observational data. The architecture supports multimodal sensory inputs, meaning a single agent can simultaneously process continuous arbitrary length vectors, ray-cast collision detections, and high-dimensional rendered visual images.
- **Agents** - the **Agent** component is fundamentally attached to a game object within the simulation, granting it the systemic capacity to gather observations from its sensors, execute distinct actions, and process environmental reward signals. A key feature of this architecture is the implementation of a **behavior name**, which identifies the underlying policy. The toolkit enables an unlimited number of agents within a scene to share the same behavior name. When grouped in such a manner, these agents synchronize their learning, executing the same policy and collectively pooling their trajectory experience data during the optimization phase. The architecture also enables policies to derive decisions from various distinct sources, such as hard-

coded heuristic scripts, direct human input, embedded on-device neural networks or external Python API commands.

- **Academy** - It operates as a global singleton entity that manages the overarching logic of the simulation. Its primary functions are to track the discrete temporal steps of the simulation and to manage the lifecycle of all active agents. Additionally, the Academy handles *environment parameters*, which are exposed variables that can dynamically alter the configuration of the simulated world at runtime. Through the Academy, an algorithm can manipulate gravity, re-scale objects, or adjust obstacle density dynamically, which constitutes the architectural backbone required for implementing advanced Curriculum Learning and Domain Randomization protocols.

3.3.2 Training Data Flow

On the external side of the architecture, the provided Python API initializes a **UnityEnvironment** class, which serves as the controller for launching and interfacing with the Unity executable. To maximize interoperability with the broader reinforcement learning research community, the toolkit architecture inherently provides wrapper APIs. These wrappers seamlessly convert the complex Unity communication pipeline into the standardized OpenAI Gym interface, permitting researchers to plug Unity environments directly into pre-existing reinforcement learning codebases without structural modifications.

The continuous execution loop that drives the training process between Unity and Python proceeds through the following computational steps:

1. The **UnityEnvironment** class within the Python API initiates the connection to the Unity executable utilizing the gRPC protocol
2. The internal Academy singleton configures the global environmental parameters (e.g. curriculum difficulty levels) for the upcoming episode.
3. The Sensors attached to the in-game Agents collect the current environmental state data.
4. The Agents package these observations, alongside any received rewards and terminal state flags, and transmit them through the Academy to the Python backend via serialized protobuf messages.
5. The Python deep learning backend processes the incoming batch of experiences. Utilizing algorithms like PPO or SAC, it computes the objective functions and updates the neural network's mathematical weights.

6. The Python API calculates the optimal next actions based on the updated policy and transmits this action vector back across the gRPC connection to the Unity environment.
7. The Unity Agents receive the action vectors and execute the corresponding physical or logical movements.

The simulation progresses through the steps, and steps 2-7 repeat rapidly, until the policy converges to optimal behavior.

3.4 Evaluation Metrics

During training process, the progress of each model was monitored using statistics recorded by TensorBoard, which is integrated with the ML-Agents framework by default. The data are stored in *events.out.tfevents* event files. The analysis focused primarily on cumulative environmental reward, episode length and a custom curriculum-level statistic indicating when the agent was promoted to the next lesson. These metrics were selected because they directly reflected the agent’s learning progress, the time spent on individual curriculum levels and any periods of stagnation or instability.

The analysis of every training run included whether the predefined target curriculum level had been reached within the established training budget, how many training interval steps had been required to reach this level and the corresponding wall-clock training time. The learning process was also examined in terms of stability. This was evaluated based on the variability of the environmental reward, the occurrence of extended stagnation periods and temporary decreases in performance during training. As all models were compared with respect to the same predefined target level, the highest curriculum level reached was not treated as a separate evaluation criterion.

Three principal metrics were selected to analyze the training process:

- **Cumulative Reward** represents the sum of rewards and penalties received by the agent during an episode. Its increase generally indicates an improvement in the agent’s behavior, although temporary decreases may occur after promotion to a more difficult curriculum level. Because the metric includes intermediate rewards and penalties, it should not be interpreted directly as the agent’s win rate.
- **Environment/Lesson Number/level** is a metric recorded by ML-Agents in TensorBoard that indicates the curriculum lesson currently assigned to the agent. It allows training progression to be tracked and the approximate number of environment steps required to advance between successive tasks to be determined. Periods in which the value remains unchanged indicate how long the agent remained at a particular lesson before satisfying its promotion condition.

- **Policy Entropy** describes the randomness of the agent’s policy and therefore the degree of exploration during training. Higher entropy indicates that actions are selected more randomly, whereas lower entropy indicates that the policy has become more deterministic.

Each evaluation episode was classified as either successful or unsuccessful. An episode was considered successful if the agent correctly completed the task defined for the selected curriculum level before the termination condition was met. Otherwise, the episode was classified as unsuccessful. The primary post-training metric was the success rate, calculated as follows:

$$\text{Success Rate} = \frac{N_{\text{successful}}}{N_{\text{successful}} + N_{\text{unsuccessful}}} \quad (3.3)$$

First, the success rate was calculated for each trained model separately. The results obtained by the ten models belonging to the same method were then aggregated to calculate the mean performance and variability between independent training runs. This procedure enabled comparison of the PPO baseline and PPO with GAIL not only in terms of their average effectiveness, but also in terms of consistency of results.

3.4.1 Evaluation process

The evaluation of the trained models is conducted directly within the simulated game environment implemented in the Unity engine. To this end, the exported *ONNX* file is assigned to the component responsible for the agent’s behavior. Next, within the evaluation management object, the index of the agent being analysed, the number of episodes, the name of the resulting *CSV* file and the curriculum level at which the test is to be carried out are specified.

The evaluation module allows the selection of any implemented curriculum level. It is not limited to level 11, which was used in the main experiments. This enables the model’s behavior to be examined both at the target level and on earlier tasks.

During evaluation, the model’s weights are not further updated. The agent uses only the policy stored in the *ONNX* file, and mechanisms used solely during training, such as the additional GAIL signal, no longer influence the decisions made. Once the specified number of episodes has been completed, the module writes to a *CSV* file, including, the agent’s index, the number of episodes played, the level on which it was evaluated on, the number of wins and losses, and the calculated win rate.

3.5 Using the Research Environment

In order to operate the trainer, it is necessary to establish a virtual Python environment with the ML-Agents package and other requisite dependencies installed:

```
venv_3_9\Scripts\activate
```

Then, after launching the environment, use the ML-Agents function:

```
mlagents-learn ConfigurationFile.yaml --run-id=nameOfRun
```

After a message appears indicating that the trainer is waiting for a connection, run the scene in the Unity editor by clicking the **Play** button. This option allows to observe how the environment works and was used primarily during implementation and troubleshooting.

More efficient way of the training is to activate many instances of the game, using the `.exe` file of the game. The environment's executable file is specified using the `-env` argument. Five independent instances of the game were launched simultaneously, and graphics rendering was disabled:

```
mlagents-learn ConfigurationFile.yaml
--run-id=nameOfRun
--env=map\game.exe
--num-envs=5
--no-graphics
```

Each instance generates separate episodes, but all instances are connected to the same policy. The collected observations, actions, and rewards are fed into a single training process, which used them to update a shared neural network.

The training process proceeded as follows:

1. The ML-Agents trainer passes the current value of the *curriculum* parameter to the Unity environment. Based on this value, the environment generated the appropriate board, the agent's hand, and the mission conditions.
2. The agent collects observations describing the current state of the game, including information about the board, the stones in hand, and other game parameters. The observations were passed on to the policy running on the coach's side.
3. Before selecting an action, the game environment generates masks indicating actions that were unavailable in the current state. The policy then selected an action from among the permitted values of the individual branches of the discrete action space.

4. The environment code interpreted the selected values as the type of action, the selected tile, and the source and target positions. Unity then carried out the action, updating the game state and awarding
5. Information about observations, actions, rewards and subsequent states are added to the set of experiences. Once the required number of experiences had been accumulated, the trainer used them to calculate the loss function and update the policy parameters.
6. The episode continued until the task was completed, or a condition for defeat was met. Once the episode had ended, the environment saved its statistics and prepared a new initial state.
7. The process was repeated until a defined target level or the maximum limit on training steps was reached.

For each run, ML-Agents created a directory identified by the value `run-id`. This directory contained *events.out.tf* event files, training checkpoints and the exported model in ONNX format. The event files contained standard ML-Agents metrics as well as custom environment statistics, including the current curriculum level, reward, episode length and promotion information. TensorBoard then enabled the visualisation of metrics such as cumulative environment reward, episode length and policy entropy. The ONNX model, meanwhile, contained the trained policy intended for deployment in Unity without any further parameter tuning.

The evaluation procedure utilized the exported ONNX model, which was assigned to the agent's *Behavior Parameters* component in the evaluation scene. The *EvaluationManager* object specified the index of the agent under test, the number of episodes, the name of the output file, and the level at which the agent was to be evaluated. For each model, 1,000 episodes were run, during which wins and losses were counted. Once the evaluation was complete, the results were saved to a CSV file, including the agent's index, the number of episodes played, the number of wins, the number of losses and the win rate.

Chapter 4

Experiments

This chapter presents the configuration process and the results of experiments conducted in the test environment. The main objective of the research was to empirically verify the effectiveness and assess the stability of RL agents. The following sections discuss in detail used hyperparameters, the experiments conducted, and analyze the results obtained in the context of the research objectives.

4.1 Methodology

This chapter outlines the research framework, training methods employed and training configuration used throughout the study. It also outlines the evaluation procedure and performance metrics employed to ensure fair and consistent comparisons between the approaches under investigation.

4.1.1 Research Framework

The experimental study focuses on comparing two reinforcement learning configurations within a level-limited Curriculum Learning environment: a baseline PPO agent and a PPO agent supported by Generative Adversarial Imitation Learning. The objective is not to train an agent capable of completing the entire Rummikub curriculum, but to evaluate how effectively both approaches acquire the sequence of skills represented by the selected curriculum level, which are described in 3.1. The experiments are intended to determine whether the addition of GAIL improves the learning process compared with the PPO baseline. In particular, the study investigates whether demonstration-based guidance allows the agent to reach curriculum milestones using fewer training steps, reduces variability between independent runs, and produces a more stable learning curve.

The first stage involved preliminary training experiments in the complete game environment. No meaningful learning progress was observed. Due to the highly sparse reward structure of the complete Rummikub environment, the agent was unable to discover ef-

fective strategies through exploration alone. Consequently, repeating the same training procedure using multiple independent runs did not provide statistically meaningful results. Therefore, the complete game environment was excluded from the experiment in this thesis, and the statistical analysis focused exclusively on training an agent that can effectively generate basic sequences on the board based on the presence of another sequence.

Two agent configurations were trained using the Proximal Policy Optimization (PPO) algorithm. Each configuration underwent multiple independent training runs with identical hyperparameter settings and the same ML-Agents seed, ensuring consistent training conditions. While the training configuration remained fixed, as shown in tables 4.1, 4.2, the Unity environment continued to generate random game states, providing different learning experiences across training runs.

Finally, the trained models were evaluated under identical evaluation conditions. The evaluation consisted of two stages. First, the ability of each training run to finish the 11th curriculum level was analyzed. Second, the trained policies were assessed in a separate evaluation procedure by executing multiple independent episodes on the 11th curriculum level and measuring the percentage of successfully completed episodes. The obtained results were subsequently analyzed to compare the effectiveness, robustness, and learning stability of the investigated training configurations. The baseline PPO model was then compared with an enhanced PPO model using Generative Adversarial Imitation Learning (GAIL).

4.1.2 Training methods

This experimental study compares the performance of several training configurations based on the PPO algorithm. In addition to the baseline implementation, selected learning support technique was incorporated to investigate their impact on the agent’s learning performance. The sections 2.2.3, 2.2.4, 3.2 present the theoretical background and implementation details of techniques, The following training configurations were evaluated:

- PPO which is baseline RL algorithm in ML-Agents, used as reference method.
- PPO + GAIL - combined with GAIL using expert demonstrations.

The baseline PPO agent was used as the reference model, and the other configuration examined the contribution of individual learning support technique to overall training performance.

All training configurations used the same neural network architecture and hyperparameter settings. This ensured that any performance differences observed during the experiments were primarily due to the learning support techniques applied, rather than variations in the training configuration.

Table 4.1: PPO hyperparameters and neural network architecture used in all training configurations.

Parameter	Value
Trainer	PPO
Batch size	512
Buffer size	8192
Learning rate	3×10^{-4}
Learning rate schedule	Linear
Beta	0.001
Epsilon	0.15
Lambda	0.95
Number of epochs	3
Discount factor (γ)	0.995
Reward strength	1.0
Time horizon	256
Maximum training steps	2,000,000
seed	42
Observation normalization	Enabled
Hidden units	128
Hidden layers	2
Memory type	LSTM
Memory size	128
Sequence length	32
Number of lessons	12
Performance measure	Reward
Promotion threshold	0.7
Minimum lesson length	500 episodes
Signal smoothing	Enabled
Require environment reset	Disabled

Each agent was trained independently using identical environment settings and the same maximum number of training steps. The baseline PPO configuration served as the reference setup, while the remaining configurations extended the baseline with selected auxiliary learning technique. Hyperparameters specific to Generative Adversarial Imitation Learning were enabled only for the corresponding training configurations, whereas all other PPO parameters remained unchanged.

Table 4.1 summarizes the PPO hyperparameters shared across all training configurations, the neural network architecture used in the experiments and the curriculum learning configuration, including the lesson progression criteria. Additional hyperparameters associated with auxiliary learning technique listed in the Table 4.2.

Table 4.2: Hyperparameters of the auxiliary learning techniques.

Module	Parameter	Value
GAIL	Strength	0.4
	Learning rate	1×10^{-4}
	Discount factor (γ)	0.99
	Use VAIL	False
	Use actions	True

Hardware

This section presents the physical components of device under which the experiment was conducted.

- CPU - AMD Ryzen 9 7940HS with Radeon 780M Graphics, 4.00 GHz
- GPU - NVIDIA GeForce RTX 4050 and AMD Radeon 280M
- 16.0 GB of RAM
- 64-bit Windows 11

4.1.3 Evaluation methods

The evaluation procedure was divided into two complementary stages. The first stage involved analyzing the training process and how individual agents progressed through the Curriculum Learning environment. The second stage involved a post-training evaluation conducted at a fixed curriculum level. This approach enabled both the efficiency of the learning process and the final performance of the trained policies to be assessed.

Following the training stage, each trained neural network was evaluated independently at the 11th curriculum level. During this evaluation, the NN operated in inference mode, with no updates to its parameters. Each network underwent testing for 1,000 episodes in identical curriculum learning level. Since ten models were evaluated for each method, there were 10,000 episodes for PPO and 10,000 for PPO with GAIL, making 20,000 in total.

4.2 Results

This section will present and analyzes the results of experiments, comparing them across different settings.

4.2.1 Baseline

This section presents the results of the baseline model during training, as well as the final evaluation results of the baseline model.

Learning results

The figure 4.1 presents a graph **Cumulative Rewards** 3.4, obtained during the training process of ten basic independent PPO baseline agents. At the beginning of training, nearly every agent achieved scores ranging from -3.9 to -3.6 . During the first 100,000 to 200,000 steps, the environmental reward suddenly increased to 0 and positive values, which may indicate the simplicity of the first levels.

In the subsequent stages, most training runs followed a repeated pattern of gradual reward improvement followed by a sudden decrease. These decreases were most likely associated with transitions to more difficult curriculum levels. After entering a new level, the agents initially obtained lower rewards and subsequently adapted their policies to the new task. Nevertheless, several decreases below -2 indicate that some curriculum stages introduced a considerable increase in difficulty.

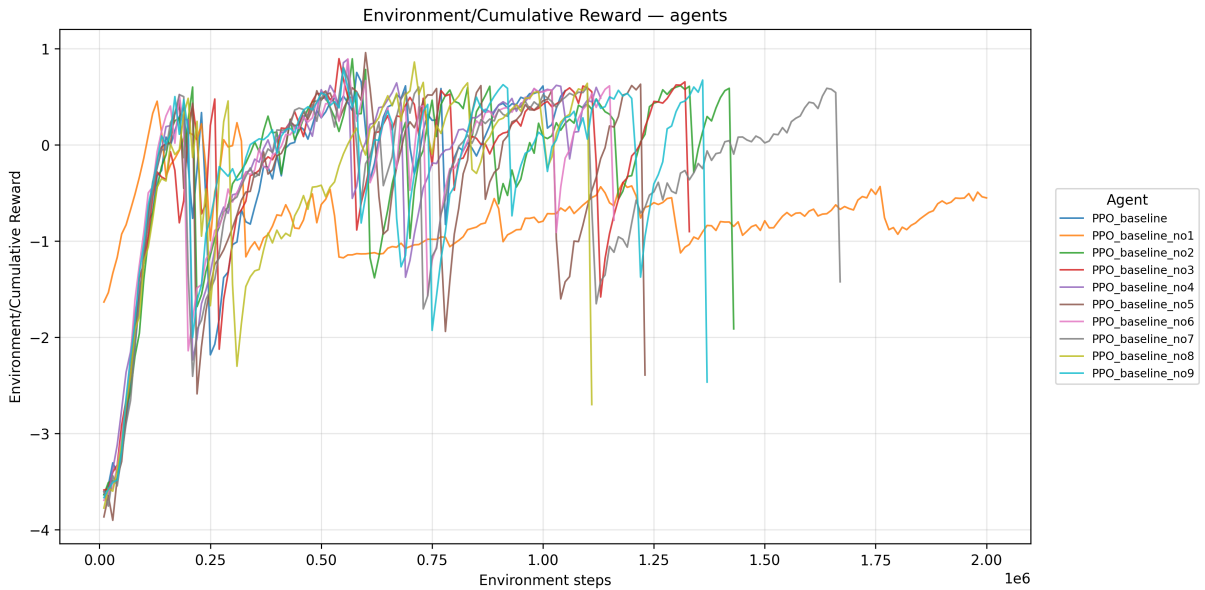


Figure 4.1: Cumulative reward graph of 10 baseline agents

While most agents exhibited similar general learning patterns, differences were observed in the number of steps required to recover after a level transition. The most distinct result was obtained by the *PPO_baseline_no1* agent. Its reward reached positive values during the early stage of training but subsequently remained below zero for most of the recorded process. Although this agent achieved positive rewards earlier than the remaining agents, its rapid initial improvement did not translate into successful progression to the predefined target level. This indicates that the agent encountered difficulties in

adapting to one of the subsequent curriculum tasks within the available training budget. The *PPO_baseline_no7* agent required substantially more training steps than most of the other agents, however, it did reach the target level.

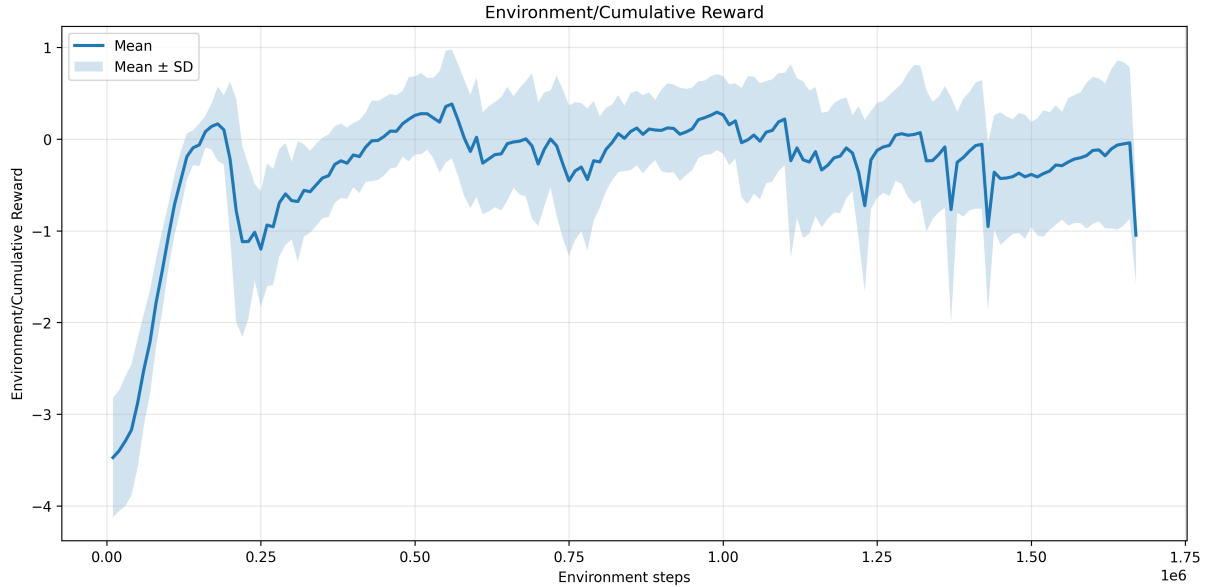


Figure 4.2: Cumulative reward mean and SD graph of 10 baseline agents

Figure 4.2 summarizes the cumulative rewards obtained by the PPO Baseline agents. The mean reward increased rapidly from approximately -3.5 at the beginning of training and reached values close to zero within the first 200,000 environment steps. This confirms that the agents generally acquired the initial curriculum skills relatively quickly. The subsequent fluctuations around zero reflect the repeated process of adapting to increasingly difficult tasks rather than a continuous improvement within a single unchanged environment.

The standard-deviation band becomes wider as training progresses, indicating increasing differences in the learning progress of individual agents. This variation results partly from agents reaching successive curriculum levels at different times and therefore performing tasks of different difficulty at the same training step. Consequently, the averaged curve describes the general learning tendency of the PPO Baseline group, whereas the wide standard deviation shows that the speed of acquiring more complex skills was not uniform across training runs.

The figure 4.3 illustrates the progression of ten independently trained agents through the curriculum using the PPO baseline configuration. The stepwise shape of the curves is a consequence of the structure of curriculum learning, whereby an agent remains at a given level until the necessary promotion criteria have been met.

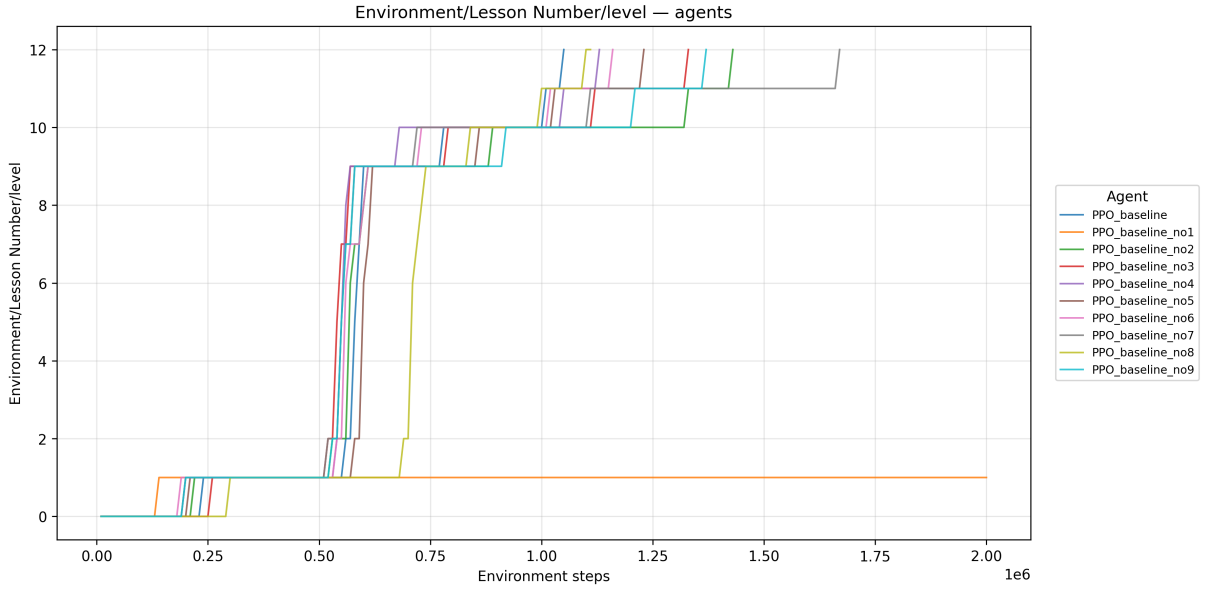


Figure 4.3: Curriculum progression of individual 10 baseline agents

Most agents completed the initial curriculum level after approximately 200,000–300,000 environment steps. The exception was the *PPO_baseline_no1* agent, which advanced to level 1 earlier, after approximately 140,000 steps. However, it remained at this level until the end of its training process at 2 million steps. Consequently, this agent did not reach the predefined target. This observation also explains why its curriculum reward remained predominantly negative in the later part of the training process, the agent was unable to satisfy the promotion criterion for level 1.

The remaining nine agents successfully progressed beyond the initial stages of the curriculum. Most of them remained at Level 1 until they had taken approximately 500,000–600,000 steps, after which they advanced rapidly through Levels 2–9. The short amount of time spent at several of these intermediate levels suggests that they found the tasks associated with these levels easier than those associated with the longer plateaus. However, the exact transition steps should be interpreted as approximate, as the metric was recorded at fixed summary intervals.

Significant differences between the agents emerged at levels 9, 10 and 11. Several agents remained at level 9 for a long time before being promoted to level 10. The time taken to advance beyond level 10 also varied considerably, with some agents achieving this in approximately 1 million steps, while others required more than 1.3 million. These plateaus suggest that the later curriculum tasks were more challenging and required more experience to meet the promotion criterion.

The *PPO_baseline_no7* agent required the largest number of environment steps among the successful training runs. It remained at level 11 for an extended period and reached the final target only after approximately 1.6 million steps. By contrast, the fastest agents reached the same stage after approximately 1.1 million steps. Overall, nine out of

ten agents reached the target, but their progression rates differed considerably despite the use of the same training configuration.

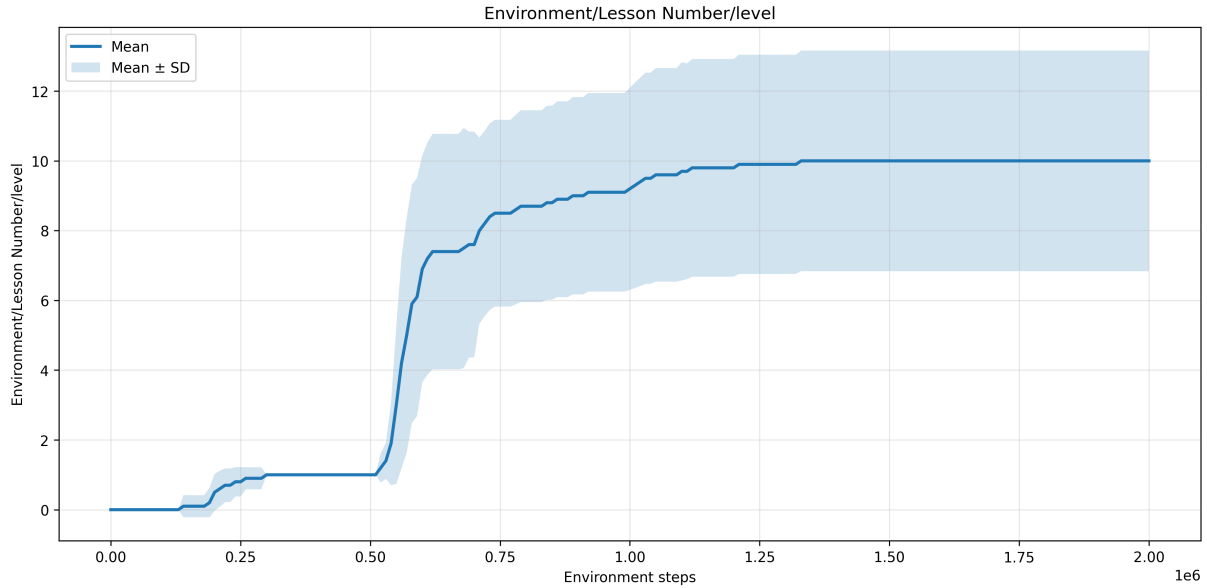


Figure 4.4: Curriculum progression mean and SD of individual 10 baseline agents

Figure 4.4 summarizes the curriculum progress of the PPO Baseline agents. The mean lesson number increased rapidly after approximately 500,000 environment steps, confirming that most agents progressed through several intermediate tasks within a relatively short training interval. The subsequent slower increase indicates that the later curriculum tasks required more experience and produced greater differences in progression speed.

The widening standard-deviation band reflects the increasing separation between agents that reached the target and the single agent that remained at an early curriculum level and failed to complete the training objective. The decrease in the mean observed in the later part of the graph does not represent a regression of the trained policies. Instead, agents that reached the target stopped producing further observations, leaving the unsuccessful and slower runs with a progressively greater influence on the calculated statistics. Therefore, the final section of the curve is based on a changing and decreasing number of active runs and should not be interpreted as the mean progress of all ten agents.

The figure 4.5 demonstrate changes in policy entropy value 3.4 during the training agents using the PPO baseline configuration. At the beginning of training, most agents exhibited an entropy value of approximately 5.9. During the first 500,000 environment steps, entropy gradually decreased to approximately 2.4–2.6. This general downward trend indicates that the agents progressively reduced random exploration as they acquired more consistent action-selection strategies for the initial curriculum tasks.

Most agents experienced a substantial increase in entropy between approximately 550,000 and 700,000 steps. According to the curriculum level progression shown in 4.3, this interval corresponded to rapid advancement through several successive levels of the

curriculum. Therefore, the increase in entropy suggests that the agents began to explore a wider range of actions after encountering new tasks and changing environmental conditions. After this period, entropy generally stabilized between 3.3 and 3.9, although temporary fluctuations remained visible. Many of these fluctuations coincided with progression to subsequent curriculum levels and may reflect the need to adapt the learned policy to the requirements of new tasks.

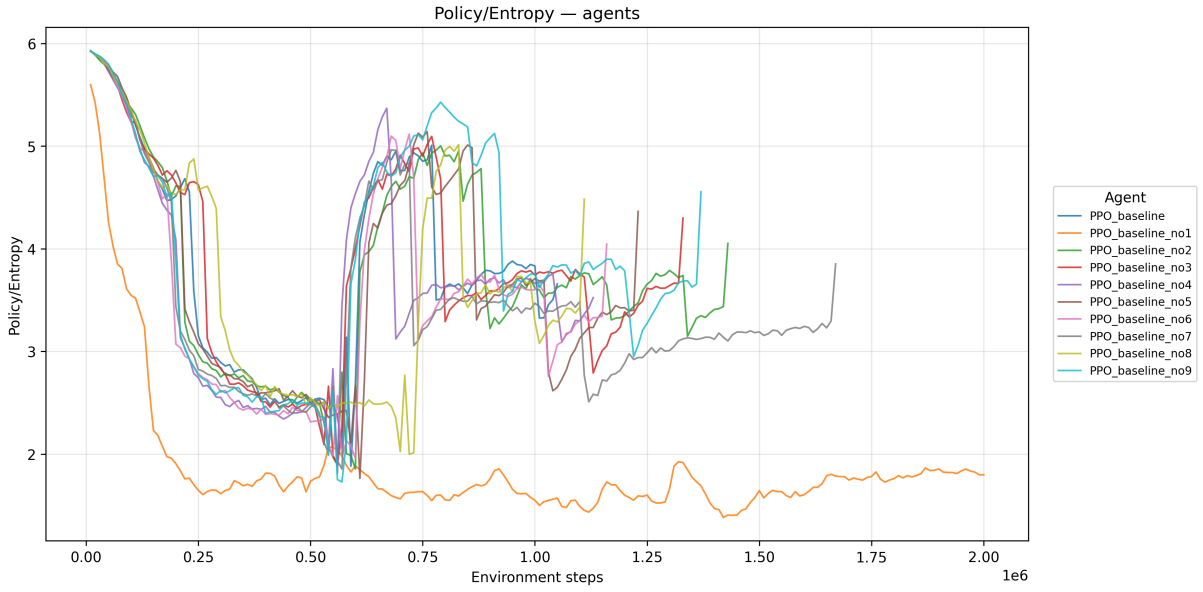


Figure 4.5: Entropy chart of individual 10 baseline agents

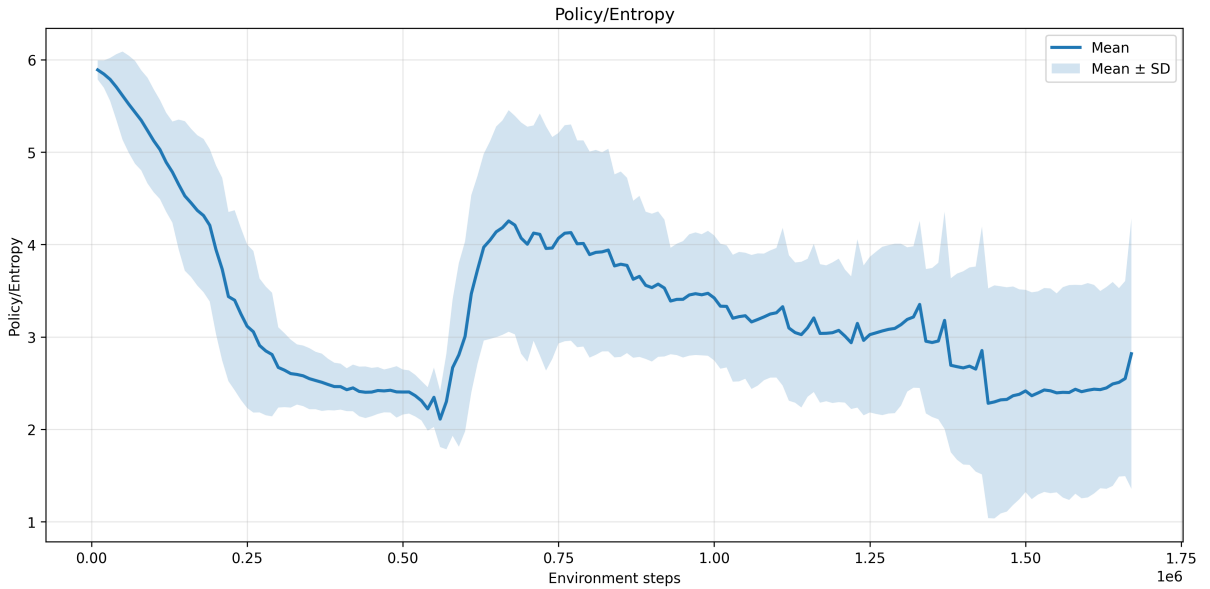


Figure 4.6: Entropy mean and SD chart of individual 10 baseline agents

The figure 4.6 summarizes the changes in policy entropy for the PPO Baseline agents. The initial decrease from around 5.9 to 2.3 shows that the policies changed gradually from

highly exploratory behavior to more consistent action selection as the agents acquired the initial curriculum skills. The subsequent sharp increase at around 600,000 environment steps is suggestive of renewed exploration, prompted by the introduction of substantially more complex tasks. After this point, the mean entropy decreased gradually again, indicating the agents’ policies progressively adapting and stabilizing.

The standard deviation band is relatively narrow during the period of lowest entropy, showing that the initial policies were developed similarly. It widens after the increase in task difficulty, indicating greater differences in exploration and adaptation between agents progressing through the curriculum at different rates. In the final part of the graph, the statistics are additionally influenced by the decreasing number of active training runs, since agents that have reached the target no longer produce further entropy observations.

Table 4.3: First recorded training steps at which individual baseline agents completed the level.

Run	L0	L1	L2	L3	L4	L5	L6	L7	L8	L9	L10	L11
0	240	560	580	580	590	590	590	600	600	780	1 010	1 050
1	140	–	–	–	–	–	–	–	–	–	–	–
2	220	530	570	570	580	580	580	600	610	880	1 320	1 430
3	260	520	540	540	550	550	550	570	580	790	1 120	1 330
4	200	540	550	550	560	560	560	570	570	680	1 050	1 130
5	210	580	600	600	610	610	610	620	620	860	1 030	1 230
6	190	540	560	560	570	570	570	600	610	730	1 020	1 160
7	200	520	550	550	560	560	560	580	580	720	1 110	1 670
8	300	690	710	710	720	720	720	730	740	840	1 000	1 110
9	200	530	550	550	560	560	560	580	590	910	1 210	1 360
Mean	216	556.7	578.9	578.9	588.9	588.9	588.9	605.6	611.1	798.9	1096.7	1274.4
SD	41.03	50.5	49.5	49.5	49.5	49.5	49.5	46.7	48.1	74.4	101.5	183.9

Values are reported in thousands of environment steps. TensorBoard summaries were recorded every 10 000 environment steps. Therefore, each value corresponds to the first recorded training step at which the given curriculum level was observed rather than the exact promotion step. A dash (–) indicates that the corresponding level was not reached during training. Mean and standard deviation were calculated only for agents that reached the corresponding curriculum level.

The *PPO_baseline_no1* agent exhibited a distinctly different entropy pattern. Its entropy decreased rapidly from around 5.6 to below 2.0 within the first 250,000 steps, then remaining between around 1.4 and 1.9 for most of the training process. Taken together with its curriculum-level progression, this shows that the agent maintained a considerably less exploratory policy while remaining at level 1. This may indicate premature convergence to an insufficient action-selection strategy to satisfy the promotion criterion.

The remaining agents maintained higher entropy values and continued to progress through the curriculum. Temporary increases near the end of several training runs ap-

pear to coincide with transitions to more difficult levels. Overall, the results suggest that successful agents repeatedly increased their level of exploration in response to changes in the curriculum, subsequently reducing or stabilizing it as they adapted to new tasks. In contrast, the unsuccessful *PPO_baseline_no1* agent maintained relatively low entropy and did not progress beyond level 1.

The table 4.3 presents the first recorded training steps at which the individual PPO baseline agents reached successive curriculum levels. Nine out of ten agents reached level 12. The only unsuccessful run was agent 1, which reached level 1 after approximately 140,000 environment steps but did not progress to any subsequent level within the training budget.

On average, the agents reached level 1 after taking 216,000 steps in the environment. Reaching level 2 required substantially more training, with an average of around 556,700 steps. The difference of around 340,700 steps between the mean values for levels 1 and 2 suggests that level 1 was the first significant hurdle in the curriculum. However, the relatively low standard deviation of around 50,500 steps for reaching level 2 suggests that the successful agents experienced a similar degree of difficulty at this stage.

After reaching level 2, the successful agents progressed rapidly through the subsequent intermediate levels. The mean number of steps recorded increased from around 556,700 at level 2 to just 611,100 at level 9. In several cases, the same number of steps was recorded for multiple consecutive levels. This does not necessarily mean that the levels were completed at exactly the same training step. As TensorBoard summaries were recorded every 10,000 environment steps, multiple curriculum transitions could occur within a single recording interval.

An increase in training requirements was observed for later stages of the curriculum. On average, level 10 was reached after approximately 798,900 steps, while level 11 required an average of approximately 1,096,700 steps. This difference of around 297,800 steps indicates that Level 10 was one of the most demanding stages in the curriculum. Level 12 was reached after an average of around 1,274,400 steps in the environment.

Variability between successful agents increased at the later stages of the curriculum. The standard deviation rose from approximately 48,100 steps for level 9 to 74,400 for level 10, 101,500 for level 11, and 183,900 for level 12. Agent 0 reached level 12 first, after approximately 1,050,000 steps, whereas agent 7 required approximately 1,670,000 steps. The difference of 620,000 steps between these runs demonstrates considerable variation in learning speed despite the use of the same training configuration.

Overall, the results suggest that levels 1, 9, 10 and 11 required the most training, while levels 2–8 were completed relatively quickly by the successful agents. The increasing standard deviation at later stages further suggests that agents’ abilities to adapt differed more strongly as the curriculum tasks became more complex.

Table 4.4: Training outcome of individual PPO baseline agents.

Agent	Training stopped at step	Target reached	Training time [h]
0	1 050	Yes	3.83
1	2 000	No	9.16
2	1 430	Yes	5.41
3	1 330	Yes	4.96
4	1 130	Yes	4.12
5	1 230	Yes	4.48
6	1 160	Yes	3.98
7	1 670	Yes	5.95
8	1 110	Yes	4.03
9	1 360	Yes	5.27
Mean	1347	90%	5.12
SD	278.96		1.5

Training stopped indicates the first recorded training step at which curriculum level 12 was reached or the predefined training limit of 2 million environment steps if the target was not achieved. Target reached denotes whether the agent successfully reached the predefined target curriculum level. Training time is reported as the total active training time in hours.

The table 4.4 summarizes the training outcomes of the ten agents using the PPO baseline configuration. Nine of the agents reached the predefined target level, giving a success rate of 90%. Agent 1 was the only unsuccessful run. Its training continued until the predefined limit of two million environment steps, lasting 9.16 hours, but did not reach the target.

Of the successful runs, Agent 0 took the shortest time to reach the target, achieving this after approximately 1,050,000 environment steps and 3.83 hours of training. Agent 7 took the most steps of all the successful agents to reach the target, doing so after approximately 1,670,000 steps and 5.95 hours. The difference between the fastest and slowest successful runs was therefore approximately 620,000 environment steps and 2.12 hours. While the recorded training time was generally associated with the number of environment steps required to reach the target, the relationship was not strictly proportional. For example, Agent 4 reached the target after around 1,130,000 steps in 4.12 hours, whereas Agent 8 reached it earlier, after around 1,110,000 steps, but still required 4.03 hours. By contrast, Agent 6 completed around 1,160,000 steps in just 3.98 hours. Thus, although agent 6 required more environment steps than agents 4 and 8, it completed its training in less wall-clock time.

This difference suggests that training duration is also affected by variations in computational throughput. Wall-clock training time, on the other hand, describes the practical computational cost of conducting a training run under the available hardware conditions.

When all ten runs are considered, training stopped after an average of approximately 1,347,000 environment steps, with a standard deviation of approximately 278,960 steps. The average training time was 5.12 hours, with a standard deviation of 1.50 hours. However, these aggregate values include the unsuccessful agent, whose training continued until the maximum step limit. Consequently, they describe the overall computational cost of the experiment rather than the average time required to reach the target.

Evaluation results

The table 4.5 presents the evaluation results of the ten PPO baseline agents. Each agent was evaluated over 1,000 independent episodes on curriculum level 11, resulting in a total of 10,000 evaluation episodes.

Table 4.5: Evaluation results of PPO baseline agents over 1000 episodes on curriculum level 11.

Agent	Episodes	Wins	Losses	Win rate [%]
0	1000	721	279	72.10
1	1000	354	646	35.40
2	1000	703	297	70.30
3	1000	631	369	63.10
4	1000	714	286	71.40
5	1000	670	330	67.00
6	1000	685	315	68.50
7	1000	675	325	67.50
8	1000	651	349	65.10
9	1000	705	295	70.50
Total	10 000	6509	3491	
Mean				65.09
SD				10.26

Collectively, the agents achieved 6,509 wins and 3,491 losses, giving them an average win rate of 65.09%. The standard deviation of the individual agents’ win rates was 10.26 percentage points, indicating variation in evaluation performance between the trained policies.

Agent 0 achieved the highest win rate of 72.10%, followed by agent 4 with 71.40%, agent 9 with 70.50%, and agent 2 with 70.30%. Among the agents that reached the predefined target during training, the lowest evaluation result was obtained by agent 3, with a win rate of 63.10%. Therefore, the win rates of the successful training runs ranged from 63.10% to 72.10%. This relatively limited range indicates that agents that successfully completed the target training stage generally acquired policies of comparable effectiveness, despite the previously observed differences in learning speed.

Agent 1 produced a distinctly lower win rate of 35.40%. This was also the only agent that did not reach the predefined target level during training, as it remained at curriculum level 1 until the training limit was reached. Its considerably weaker evaluation result is therefore consistent with its incomplete curriculum progression. Nevertheless, the agent was still able to win some evaluation episodes, suggesting that the strategies learned during the initial curriculum stages provided limited transferable capabilities or that some level 11 instances could be solved without a fully developed policy.

Overall, the evaluation results indicate that reaching the predefined curriculum target was associated with substantially better performance on level 11. However, reaching the target did not result in perfect task execution, as even the best-performing agent lost approximately 28% of its evaluation episodes. The results therefore demonstrate that the agents acquired useful and relatively consistent strategies, while leaving room for further improvement in policy reliability.

4.2.2 GAIL as auxiliary method

This section presents the results of the model during the training process, as well as the final evaluation results of the GAIL model.

Learning results

Figure 4.7 shows a graph of **Cumulative Rewards** obtained during the training of ten independent agents using the **GAIL** auxiliary method. In the initial training interval, between 0 and 200,000 steps, there is a rapid increase in the environment’s reward, with each agent exhibiting a very similar rate of increase. Once the agents achieved scores close to 0.5, there was a sharp drop in the reward, which may indicate that the agent advanced to the next level, where it encountered difficulties. This pattern repeats throughout the entire training process, the environment reward rises and then suddenly drops. This may indicate that the agent is successfully progressing to subsequent levels, where it must readjust its policy to accommodate the increasing difficulty of the tasks.

After around 200,000 steps, the learning trajectories of the agents begin to diverge. Declines and returns to positive rewards occur at different times, some agents experience steeper declines in environmental rewards and complete training after varying numbers of steps.

Among all runs, agent *PPO_GAIL_no9* draws particular attention, as it experienced a sharp drop in the environmental reward around the 900,000th step. A similar phenomenon was observed in the case of agent *PPO_GAIL* around the 1.1 millionth step. These drops may indicate short-term policy instability or difficulties in adapting the policy to changing environmental conditions. In both cases, however, the reward was gradually restored and returned to positive values. This suggests that the observed deterioration

was temporary and did not lead to a permanent breakdown of the learning process.

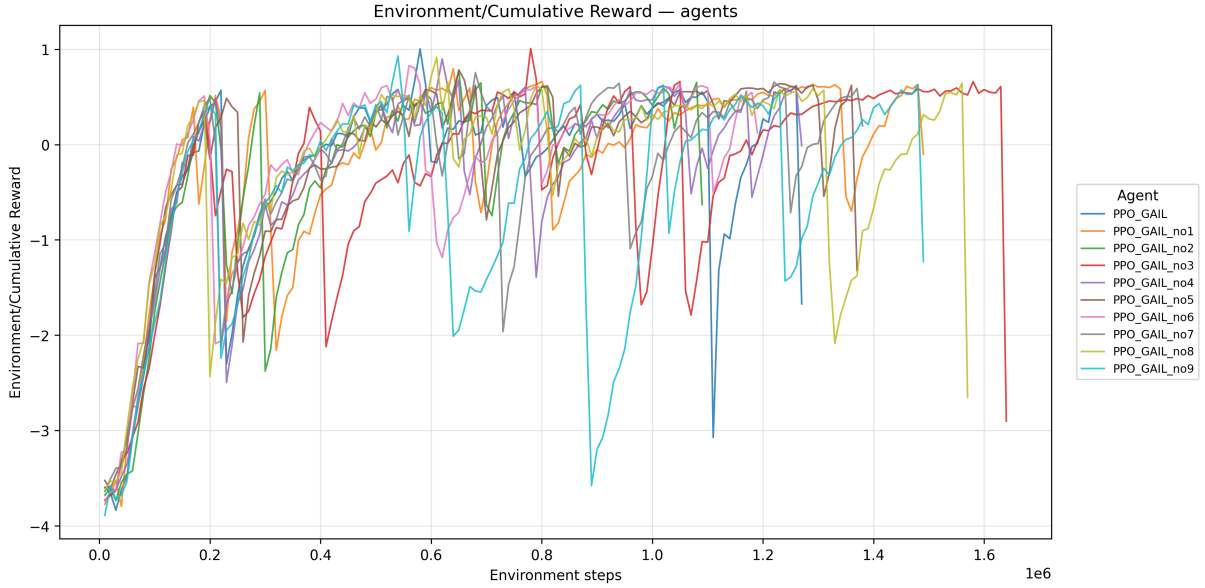


Figure 4.7: Cumulative reward graph of 10 GAIL agents

Nevertheless, given that every agent completed training before reaching 2 million steps, it can be concluded that each agent ultimately reached the expected level.

Figure 4.9 displays the stage and time at which each of the ten GAIL agents was at, as well as when they advanced to the next level. The first promotions occurred after just 200,000 steps, with the agent *PPO_GAIL_no3* being the last to be promoted, advancing to level one after 400,000 steps. Level zero marked the beginning of the learning process, so it is understandable that initially grasping the rules of the game might take more time.

Subsequent level-ups occurred within the range of 500,000 to 750,000. Once the conditions for promotion to a higher level were met, a sudden increase in levels could be observed. This suggests that levels 2 through 8 were similar enough that, after mastering levels 0 and 1, agents did not need to adjust their approach to meet the promotion criteria.

Figure 4.8 summarizes the curriculum rewards obtained by the PPO+GAIL agents. The mean reward increased rapidly from approximately -3.7 to positive values within the first 200,000 environment steps, indicating that the agents generally acquired the initial curriculum skills. The subsequent sharp decrease and gradual recovery reflect adaptation to more difficult tasks. During the remaining training period, the mean reward fluctuated mainly around zero and positive values, showing repeated adaptation as the agents progressed through the curriculum.

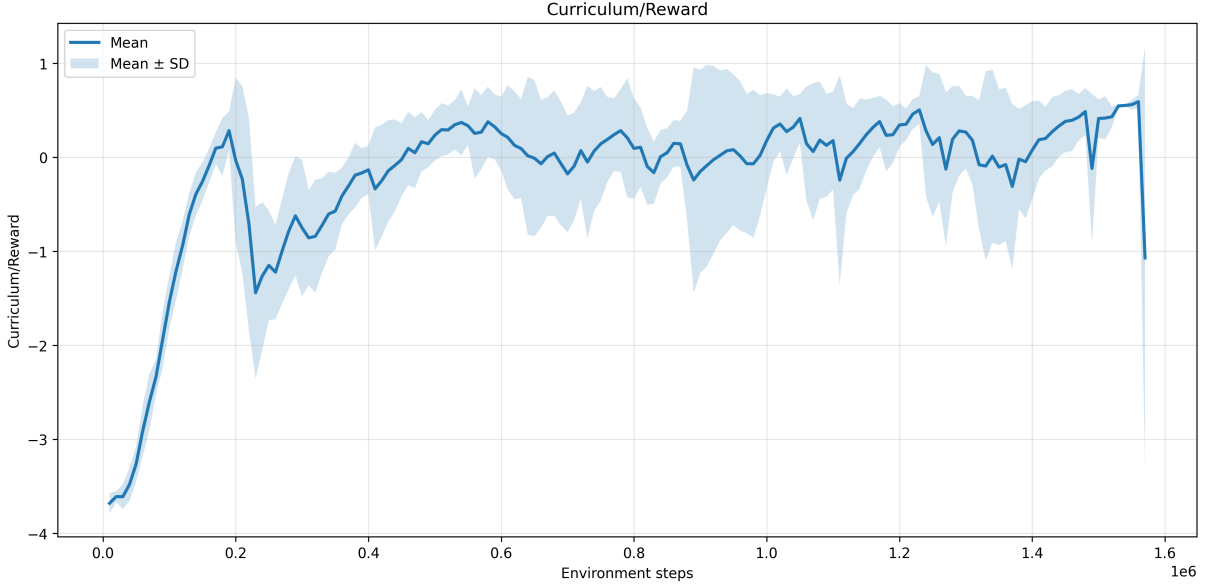


Figure 4.8: Cumulative reward graph of 10 GAIL agents

The standard-deviation band becomes wider after the initial stage, indicating differences in the moments at which agents entered and adapted to more difficult tasks. Nevertheless, all PPO+GAIL agents ultimately reached the training target. The abrupt change at the end of the graph should not be interpreted as a deterioration of the entire group because, by that point, most agents had already completed training, and the calculated statistics were based only on the remaining active runs.

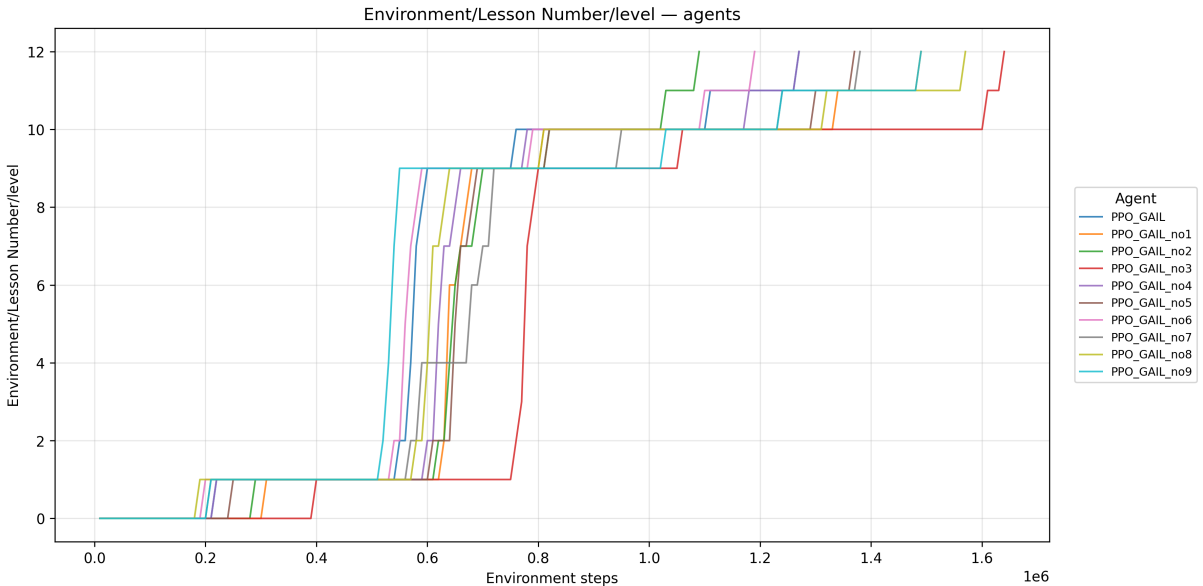


Figure 4.9: lesson progress graph of 10 GAIL agents

Level 9 presented another challenge, requiring the agent to spend more time meeting the promotion requirements. The task begins with the agent placing a sequence on the board, while another completed sequence is already present on the map. This represents

a significant change for the agent, as in previous levels there was only one sequence on the map. This explains why more time was spent on this task. Agents advanced at different times, some advanced between 770,000 and 800,000, while others needed more time and did not advance until they reached between 950,000 and 1,050,000.

Referring to the plot 4.7 of **Cumulative Rewards**, it can be seen that the agent *PPO_GAIL_no9* had already been at level 9 for some time by the 900,000th step and did not advance to level 10 until around the 1.03 millionth step. This may indicate that the significant decrease in the environmental reward during this period may have been caused by short-term policy instability. Ultimately, however, the reward was restored, and the agent eventually met the criteria for promotion to level 10, suggesting that there was no permanent breakdown in the agent’s policy.

However, the situation was different for agent *PPO_GAIL*, which experienced a similar sudden drop around 1.1 million steps 4.7. However, as shown in Figure 4.9, this drop in performance was influenced by an advancement to a higher level, which is to be expected during the learning process.

As agents progress to Level 10, their differences become more apparent. While some agents progress to Level 11 quickly, others remain at Level 10 for a considerable amount of time. Spending a long time at Level 10 is consistent with the nature of the task. The agent must place two items in the presence of a complete sequence, requiring them to perform several correct, interdependent actions. This is more challenging than adding a single missing item at Level 9.

Level 11 also has characterized by varying completion times. While some agents complete it relatively quickly, while others spend more time on it. For example, agents *PPO_GAIL_no2* and *PPO_GAIL_no3* completed Level 11 relatively quickly. Conversely, agents *PPO_GAIL_no9* and *PPO_GAIL_no8* took longer to successfully learn Level 11. Agent *PPO_GAIL_no2* completed the training the fastest, requiring approximately 1.1 million steps to complete Level 11. In contrast, agent *PPO_GAIL_no3* took the longest, requiring 1.65 million steps. The difference between them is as much as 550,000 steps.

Agent *PPO_GAIL_no3* is particularly noteworthy, as he was the last to reach each level compared to the other agents. Another interesting case is agent *PPO_GAIL_no9*, who advanced the fastest from level 1 to level 9. He spent a long time at level 9 despite his rapid advancement during the earlier stages of learning.

Overall, the GAIL auxiliary method allowed all agents to reach the desired outcome. However, it did not eliminate significant variability in the learning rate or short-term declines in performance at an unchanged level of the curriculum.

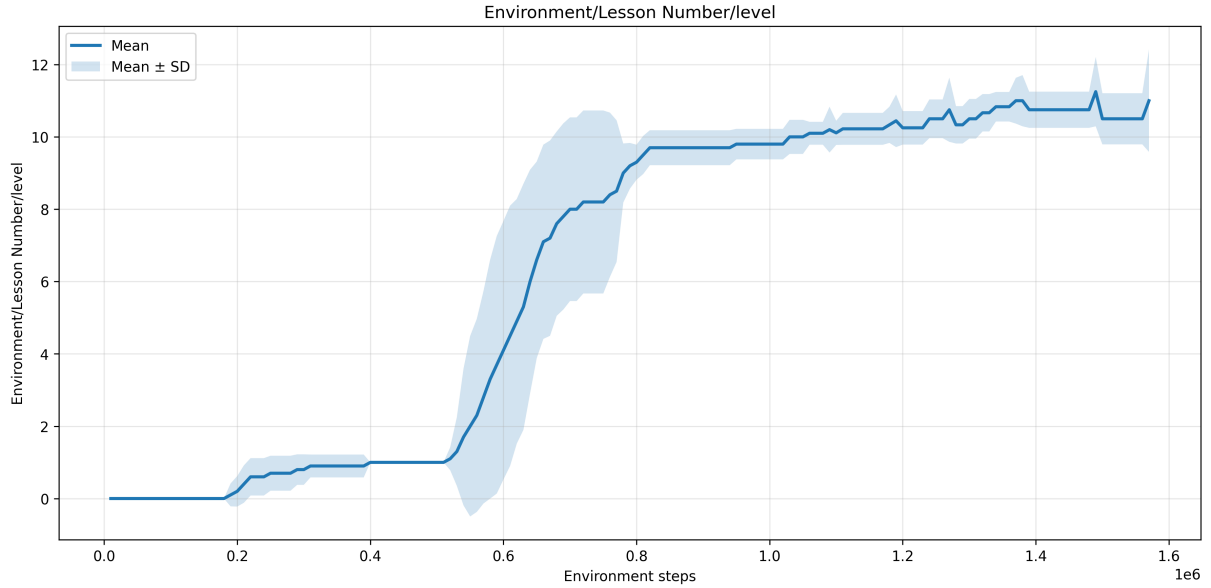


Figure 4.10: lesson progress mean and SD graph of 10 GAIL agents

Figure 4.10 summarizes the curriculum progression of the PPO+GAIL agents. The mean lesson number remained low during the initial training stage and then increased rapidly between approximately 500,000 and 800,000 environment steps. After reaching the later curriculum tasks, the progression became slower, indicating that these skills required substantially more experience than the earlier tasks.

The standard-deviation band is widest during the rapid progression through the intermediate lessons, showing that individual agents completed these tasks at different rates. It narrows in the later stages as the agents converge towards the final lessons. All PPO+GAIL agents ultimately reached the training target, demonstrating consistent completion across independent runs. The small decreases in the final part of the mean curve do not indicate regression, but result from agents completing training at different times and no longer contributing observations to subsequent points.

Figure 4.11 shows the entropy curve for ten agents trained using the GAIL method. Initially, each agent started with a value of approximately 5.9, and the decrease in entropy followed an almost identical pattern across all training scenarios.

This indicates that each agent had an equally random policy at the beginning, which is consistent with the use of the same network configuration and algorithm. The graph becomes variable around the 200,000th step, which is consistent with the graph in 4.9, as this is when the first promotions to subsequent levels occurred, which influenced the agents' policies. Entropy continues to decrease until the range between 580,000 and 800,000 steps. There is a sudden increase in entropy, which is associated with subsequent agent promotions, followed by the appearance of new observations in the environment. In this situation, the agents began to explore the environment more rather than optimize their performance.

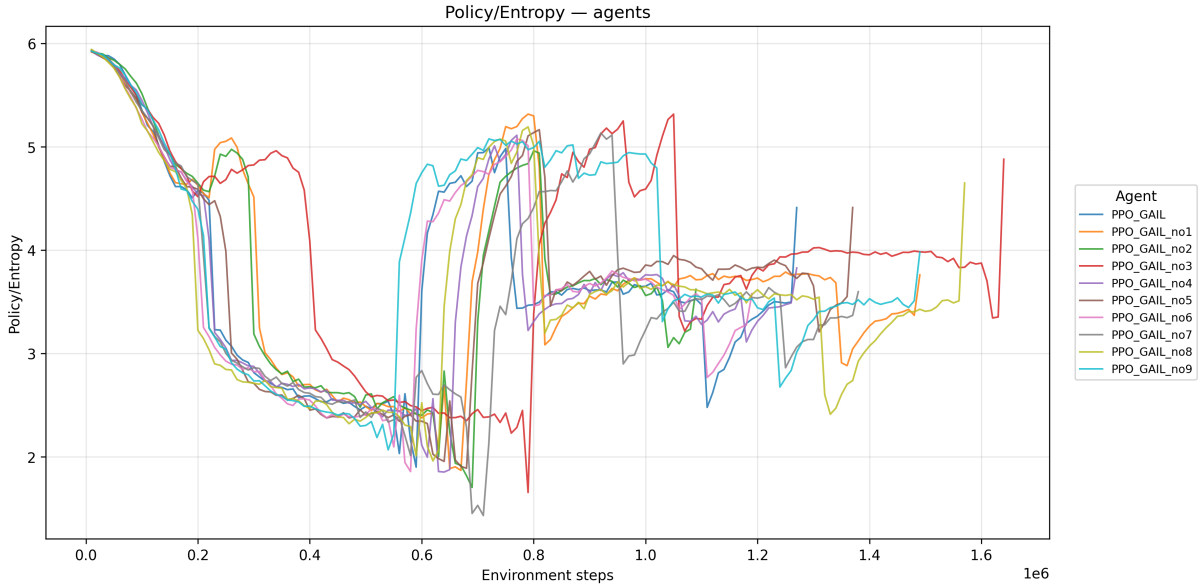


Figure 4.11: Entropy chart of individual 10 gail agents

After a sudden spike, there was a sharp decline, which has now stabilized within the range of 3 to 4. Occasionally, the values have fallen below this range, but they have always returned to it. These drops and spikes are directly linked to promotions, but the fluctuations are less significant than they were initially. This may indicate a gradual stabilization of the agents' behavior. Meanwhile, the agents maintained a moderate level of exploration, enabling them to adapt their strategies to subsequent learning levels.

Agent *PPO_GAIL_no9* experienced a significant decrease in reward around 900,000 steps, despite not receiving a promotion at that time. However, the entropy graph shows no significant change in the agent's curve during this period. This suggests that the temporary drop in reward may have resulted from a series of more difficult or unfavorable episodes rather than a clear change in the level of exploration.

Figure 4.12 summarizes the policy entropy of the PPO+GAIL agents. The mean entropy initially decreased from approximately 5.9 to 2.4, indicating that the agents gradually reduced random exploration and developed more consistent behavior while acquiring the initial curriculum skills. Around 600,000 environment steps, the entropy increased again and reached approximately 4.5, suggesting renewed exploration after the introduction of substantially more complex tasks. It subsequently stabilized mainly between 3.4 and 4.1, indicating that the agents maintained a moderate degree of exploration while adapting to the later curriculum stages.

The standard-deviation band is narrow during the initial decrease, showing similar early policy development across the training runs. It becomes considerably wider during the renewed exploration stage, indicating that individual agents responded to the increase in task difficulty at different rates. The later narrowing of the band suggests more consistent policy behavior after adaptation. The final increase should be interpreted cautiously

because most agents had already completed training, leaving only a small number of active runs in the calculation.

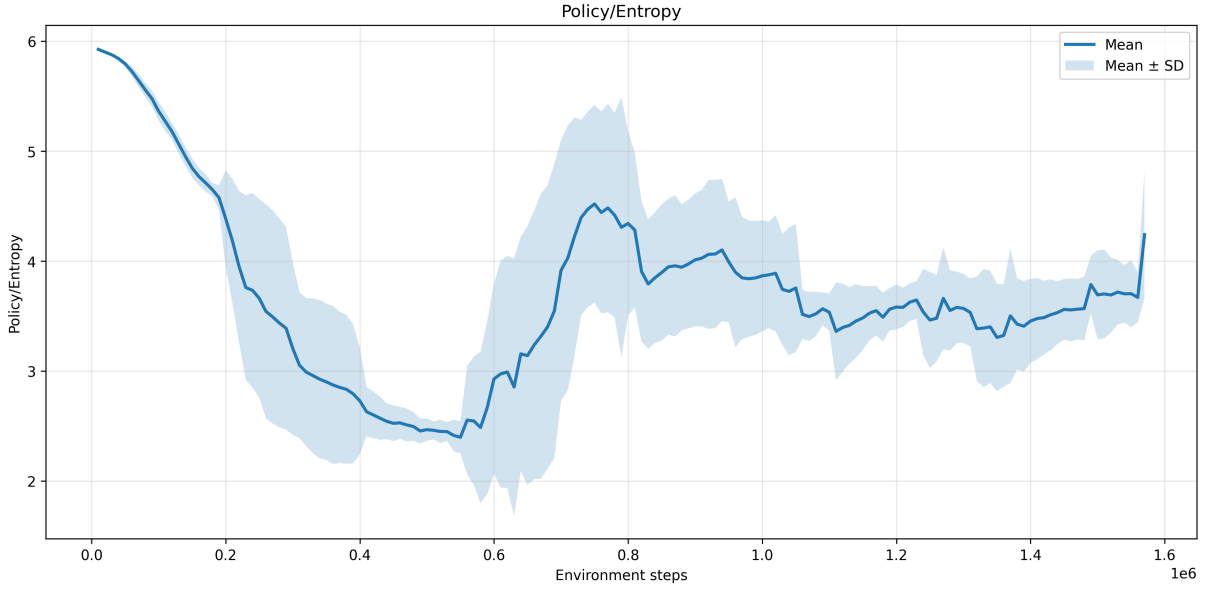


Figure 4.12: Entropy chart of mean and SD of individual 10 gail agents

Table 4.6: First recorded training steps at which individual GAIL agents completed the level.

Run	L0	L1	L2	L3	L4	L5	L6	L7	L8	L9	L10	L11
0	220	550	570	580	580	580	590	590	600	760	1 110	1 270
1	310	630	640	640	650	650	660	670	680	810	1 340	1 490
2	290	620	640	650	650	650	660	690	700	820	1 030	1 090
3	400	760	770	780	780	790	790	790	800	1 060	1 610	1 640
4	220	600	620	620	630	630	630	650	660	780	1 180	1 270
5	250	610	650	650	660	660	660	680	690	820	1 300	1 370
6	200	540	560	560	570	570	570	580	590	780	1 090	1 190
7	210	570	590	590	680	690	700	720	720	950	1 240	1 380
8	190	580	600	610	610	610	620	630	640	810	1 320	1 570
9	210	520	530	540	540	540	550	550	560	1 030	1 240	1 490
Mean	250	598	617	621	635	637	643	655	664	862	1246	1376
SD	62.2	63.8	62.9	63.8	64.1	67.1	65.7	67.9	67.0	103.9	156.1	165.0

Values are reported in thousands of environment steps. TensorBoard summaries were recorded every 10 000 environment steps. Therefore, each value corresponds to the first recorded training step at which the given curriculum level was observed rather than the exact promotion step. A dash (–) indicates that the corresponding level was not reached during training. Mean and standard deviation were calculated only for agents that reached the corresponding curriculum level.

The table 4.6 presents the time intervals within which each agent, using the GAIL auxiliary method, successfully completed a given level and advanced to the next one.

The first challenge was to complete Level Zero, which took an average of 250,000 steps, a task that may involve learning to map observations to one’s actions. Agent *PPO_GAIL_no8* took the least time to complete this level, advancing after 190,000 steps. In contrast *PPO_GAIL_no3* did not advance until after 400,000 steps, which makes it a difference of 210,000 steps.

Level 1 proved to be another challenge. On average, agents reached this level after 250,000 steps and completed it after 598,000 steps, which means it took an average of 248,000 steps to complete Level 1. However, at this level, the agent *PPO_GAIL_no9* performed the best, advancing after 520,000 steps. Between its advancement from level zero, which occurred after 210,000 steps, it took approximately 310 steps to reach level two.

Completing levels 2–8 required fewer steps, as can also be seen in Figure 4.9. The agents completed level 1 after 598,000 steps and level 8 after an average of 664,000 steps, which means that the agents successfully completed levels 2 through 8 in an average of 66,000 steps. This suggests that the given set of tasks was similar enough to the previous levels that, after mastering tasks 0 and 1, the agents could quickly adapt to levels 2–8.

Level 9 proved to be a greater challenge for the agents. On average, advancing from level 9 to 10 required 198,000 steps. The first agent to complete level 9 was *PPO_GAIL*, who advanced to level 10 after 760,000 steps. However, the agent *PPO_GAIL_no3* advanced the latest, reaching level 10 after 1.06 million steps, which means that the difference between the time of advancement for agent *PPO_GAIL* and agent *PPO_GAIL_no3* is 300,000 steps. However, agent *PPO_GAIL_no9* took the longest to complete Level 9. He completed level 8 after 560,000 steps, but finished level 9 after 1.03 million steps, which shows that it needed 470,000 steps to complete the level.

Level 10 turned out to be the most time-consuming for agents on average, as they completed this level after 1.246 million steps on average, meaning that agents needed an average of 384,000 steps after completing Level 9. Agent *PPO_GAIL_no2* performed best at this stage, completing Level 10 after 1.03 million steps. Agent *PPO_GAIL_no9* also completed this level relatively quickly, after 210,000 steps. In contrast, agents *PPO_GAIL_no1*, *PPO_GAIL_no3*, and *PPO_GAIL_no8* required over 500,000 steps to meet the promotion criteria.

Reaching the target level was faster, as it took an average of 130,000 steps between levels. However, the time it took to complete Level 11 varied among the agents. Agent *PPO_GAIL_no3* needed only 30,000 steps to meet the promotion requirements, and agent *PPO_GAIL_no2* required 60,000 steps. In contrast, agents *PPO_GAIL_no8* and *PPO_GAIL_no9* required as many as 250,000 steps to meet the promotion criteria.

Throughout the learning process, the standard deviation increases as the curriculum

progresses. Initially, the standard deviation is very similar, as are the values across levels 2–8, where agents advanced quickly. However, the greatest differences appear at levels 9–11, with the highest value occurring at level 11. This result may not only be due to the length of study at level 11, but also to varying delays in studying the previous levels.

While all agents achieved the set training goal, the pace of their progress varied greatly. Levels 0, 1, 9 and 10 were the most demanding in terms of the number of steps required, with level 10 taking the most time on average. Levels 2–8 were completed quickly, possibly indicating the transfer of previously acquired skills. Values in the second column are

Table 4.7: Training outcome of individual PPO GAIL agents.

Agent	Training stopped at step	Target reached	Training time [h]
0	1270	Yes	12.78
1	1490	Yes	10.97
2	1090	Yes	10.13
3	1640	Yes	13.32
4	1270	Yes	9.31
5	1370	Yes	9.76
6	1190	Yes	8.43
7	1380	Yes	10.88
8	1570	Yes	10.62
9	1490	Yes	10.98
Mean	1376	100%	10.7
SD	165.0		1.40

reported in thousands of environment steps and correspond to the first TensorBoard summary in which level 12 was observed.

The table 4.7 displays the success rate in reaching the goal, the number of steps required, and how long it took each agent to complete the training.

The target effectiveness is 100%, which means that every agent using the GAIL auxiliary method reaches the target level.

In terms of the number of steps taken, the most efficient agent was *PPO_GAIL_no2*, which completed training after 1.09 million steps. By contrast, *PPO_GAIL_no3* took the longest, performing 1.64 million steps before reaching the goal. This difference of 550,000 steps shows that identical hyperparameters do not lead to an identical training process.

The average training time was 10.7 hours, with a standard deviation of 1.4 hours. However, the most time-efficient agent was *PPO_GAIL_no6*, which reached the goal after 8.43 hours, even though it completed 1.19 million steps during training. Agent *PPO_GAIL_no3* took the longest to learn, at 13.32 hours. The difference between the most efficient and least efficient times is as much as 4.89 hours.

The results of agents *PPO_GAIL* and *PPO_GAIL_no4* are striking. On average, they finished training at similar times, but the *PPO_GAIL* agent completed training

after 12.78 hours, while *PPO_GAIL_no4* finished training after just 9.31 hours.

Overall, the duration of the learning process was influenced not only by the number of steps in the environment required to achieve the goal. The instantaneous load on the device’s components, the complexity of the episodes, and data logging to TensorBoard also had a significant impact. In the case of the GAIL auxiliary method, the additional imitation signal and discriminator updates may have also contributed to the computational cost. The table shows that GAIL achieved 100% effectiveness in accomplishing the set goal, but the number of steps and training time varied significantly. The *PPO_GAIL_no2* agent was the most efficient in terms of the number of trials, while the *PPO_GAIL_no6* agent was the most efficient in terms of elapsed time. The lack of a strict proportionality between steps and hours indicates significant differences in the computational throughput between individual runs.

Evaluation results

The table 4.8 presents the test results of ten GAIL agents trained at level eleven.

Table 4.8: Evaluation results of PPO GAIL agents over 1000 episodes on curriculum level 11.

Agent	Episodes	Wins	Losses	Win rate [%]
0	1000	731	269	73.1
1	1000	701	299	70.1
2	1000	724	276	72.4
3	1000	739	261	73.9
4	1000	709	291	70.9
5	1000	709	291	70.9
6	1000	684	316	68.4
7	1000	700	300	70.0
8	1000	624	376	62.4
9	1000	677	323	67.7
Total	10 000	6998	3002	
Mean				69.98
SD				3.1

Each agent was tested over 1,000 episodes, for a total of 10,000 episodes across the entire evaluation. The agents collectively achieved 6,998 wins and 3,002 losses, resulting in an overall success rate of 69.98%.

Agent *PPO_GAIL_no3* achieved the best result, with a score of 73.9%, but the results do not differ significantly from those of the other agents, the second-best agent, *PPO_GAIL_no2*, scored 72.4%. This shows that despite a significantly longer training time, agent *PPO_GAIL_no3* achieved very similar results.

Agent *PPO_GAIL_no8* performed the worst, achieving a score of 62.4%. The difference between the best and worst results is as much as 11.5 percentage points. This is a significant discrepancy, given that the agents’ configurations were identical. Achieving the same training goal did not, therefore, lead to identical final performance, and the number of steps required to advance was not a clear predictor of the subsequent evaluation result.

4.2.3 Cross-model comparison

The table 4.9 presents a summary of the average completion times for each level of the curriculum, the standard deviation, and the difference between them. The average scores were calculated among agents who completed a given level.

Table 4.9: Comparison of curriculum-level completion and transition times for PPO baseline and PPO+GAIL agents.

Level	PPO Baseline			PPO+GAIL			Difference	
	Completion	SD	Transition	Completion	SD	Transition	Completion	Transition
L0	216.0	41.0	216.0	250.0	62.2	250.0	+34.0	+34.0
L1	556.7	50.5	340.7	598.0	63.8	348.0	+41.3	+7.3
L2	578.9	49.5	22.2	617.0	62.9	19.0	+38.1	-3.2
L3	578.9	49.5	<1.0	621.0	63.8	4.0	+42.1	>+3
L4	588.9	49.5	10.0	635.0	64.1	14.0	+46.1	+4.0
L5	588.9	49.5	<1.0	637.0	67.1	2.0	+48.1	>+1.0
L6	588.9	49.5	<1.0	643.0	65.7	6.0	+54.1	>+5
L7	605.6	46.7	16.7	655.0	67.9	12.0	+49.4	-4.7
L8	611.1	48.1	5.5	664.0	67.0	9.0	+52.9	+3.5
L9	798.9	74.4	187.8	862.0	103.9	198.0	+63.1	+10.2
L10	1096.7	101.5	297.8	1246.0	156.1	384.0	+149.3	+86.2
L11	1274.4	183.9	177.7	1376.0	165.0	130	+101.6	-47.7

Values are reported in thousands of environment steps. *Completion* denotes the first recorded training step at which the corresponding level was completed. *Transition* denotes the mean number of steps required to complete the level after entering it. Positive differences indicate larger values for PPO+GAIL. Transition statistics were calculated only from agents that completed both boundaries of the corresponding transition.

It is important to note that the results include averages for agents who reached each level. For levels 1–11, the baseline average is calculated from nine agents, whereas for the GAIL models, it is calculated from ten agents. Among the agents that completed the analysed levels, the PPO Baseline recorded an average of fewer cumulative steps to complete each level. However, the differences in time taken to complete individual levels were inconsistent.

Level zero was the first time-consuming training stage for both models, but it posed greater difficulties for the GAIL model, with a difference of 34,000 steps between the models. The results suggest that incorporating GAIL did not accelerate the initial task in the curriculum. PPO+GAIL agents required more training and had more variable completion times for the level.

Level one already required a relatively similar amount of time, as the average difference between the two models was only 7,300. It can therefore be concluded that this level posed a similarly significant challenge for both methods, although PPO+GAIL required slightly more training time on average. In order to complete Level 2, the baseline required an additional 3,200 steps. However, this is a minor difference compared to the total number of steps required so far.

At levels 3–8, both configurations made rapid progress, and the transitions often occurred within a single TensorBoard recording interval. Although they reached these levels at different times, both models achieved them after a similar number of steps. This may indicate that tasks 3–8 were relatively easy to master once the models had acquired the necessary skills, or that they largely relied on behaviors learned in previous levels.

Level nine presented another bottleneck for both models. For the baseline model, the transition took an average of 187,800 steps, while for the model using the GAIL auxiliary method, it took an average of 198,000 steps. The difference of 10,200 steps was small relative to the total number of steps required at this level.

Level 10 shows the greatest difference between the models. The baseline model completed Level 10 after approximately 297,800 steps, while the GAIL model completed it after approximately 384,000 steps. The average difference between the models is as high as approximately 86,200 steps, which is the largest difference in the average time to complete a single level.

Level 11, indicating the achievement of the learning goal, showed that the GAIL agents performed better at this stage, requiring approximately 47,700 fewer steps than the baseline agents. However, in the final analysis, the baseline agents required fewer steps overall, as they successfully completed training after an average of 1,274,400 steps, while the GAIL model required an average of 1,376,000. However, it should be noted that the final results for the baseline included only agents that had learned Level 11. The difference in the number of steps to complete Level 11 is 101,600, which is not a significant difference over the course of the entire training. Given the use of expert demonstrations, it was assumed that the additional GAIL signal might reduce the number of steps needed to master the tasks. The results did not confirm this assumption with respect to the entire curriculum analyzed.

PPO+GAIL did not accelerate the achievement of the overall goal. Both configurations progressed through L1–L9 at a similar pace. PPO+GAIL’s main loss occurred at L10 and some of this was offset at L11. However, the speed comparison only applies to successful runs. Meanwhile, nine out of ten Baseline agents and all PPO+GAIL agents achieved the goal.

The table 4.10 presents a summary of the results for both RL models, including training reliability, training cost, and final policy quality. In the analyzed sample, all ten PPO+GAIL agents and nine out of ten PPO Baseline agents achieved the objective.

This means that PPO+GAIL demonstrated greater observed reliability across all trained agents.

The average number of steps to termination is 1,347,000 for the PPO baseline, and 1,376,000 steps for the PPO GAIL agent, a difference of only 29,000 steps. However, this result for the baseline is the average of the results of all agents, including the *PPO_baseline_no1* agent, which terminated training after 2 million steps because it failed to reach the final goal within the specified time. Considering only the agents that reached the specified goal, this average is 1,274,400 steps, resulting in a difference of 101,600 steps between the models. For all agents, runs stopped on average at a similar number of steps, however, for the agents that reached the goal, the baseline model required fewer steps on average.

Table 4.10: Comparison of curriculum training outcomes.

Metric	baseline all	baseline successful	PPO + GAIL
Agents reaching the target	9/10	-	10/10
Mean stopping step	1347	1274,4	1376
SD of steps	278.96	183,9	165.0
Mean training time [h]	5.12	4,67	10.7
SD of training time [h]	1.5	0.71	1.4
Evaluation mean [%]	65.09	68,39	69.98
Evaluation SD [%]	10.26	2.83	3.1
Evaluation max result [%]	72.1	72.1	73.1
Evaluation min result [%]	35.3	63.1	62.4

The standard deviation of 278.96 thousand for the baseline also takes into account the agent that failed to achieve the intended goal. However, when considering only the successful agents, the standard deviation of the baseline model is 183.9 thousand steps, which is still greater than that of the GAIL model, whose standard deviation is 165 thousand steps.

The average training time varies significantly across models. The baseline model required an average of 5.12 hours, while the GAIL model required an average of 10.7 hours, representing a computational demand that is twice as high. This result may be directly attributable to the computational cost of updating the discriminator and processing demonstrations, as well as differences in device load.

The final evaluation of both models shows an accuracy of 65.09% and a standard deviation of 10.26 for the baseline model, and an accuracy of 69.98% and a standard deviation of 3.1 for the GAIL model. This shows that, ultimately, agents trained using the GAIL auxiliary method achieved a result that was nearly 5% better in the environment. However, for the baseline model, these results include an ineffective agent, when considering only agents that successfully completed training, the average is 68.39% with a standard

deviation of 2.83. This means that when considering only effective agents, the effectiveness of the models’ policies is very similar.

The minimum and maximum evaluation results further demonstrate the influence of the unsuccessful baseline agent. When all baseline agents are considered, the observed win rates range from 35.3% to 72.1%. After excluding the agent that failed to reach the training target, the minimum increases to 63.1%, while the maximum remains unchanged at 72.1%. The successful baseline agents therefore cover a range of 9.0 percentage points. The PPO+GAIL results range from 62.4% to 73.1%, corresponding to a range of 10.7 percentage points. Thus, after restricting the comparison to successfully trained policies, the extreme evaluation results of both methods are similar, with PPO+GAIL achieving a slightly higher maximum and PPO Baseline a slightly higher minimum.

4.3 Conclusions

Both configurations demonstrated the ability to learn the tasks defined in the curriculum. Most of the baseline agents and all of the GAIL agents reached the target level, which was completing Level 11.

However, this did not mean that the task had been fully mastered. During the evaluation, both methods achieved results in the range of 65%–70%. The criterion for curriculum advancement was therefore not equivalent to 100% reliability of the policy.

The overall progression structure was similar for both models. Both models required more steps at Level 0, which involved mastering the basics of the environment’s mechanics. Level 1 also proved to be the first challenge, followed by rapid progression between levels 2 and 8, which may indicate that the skills acquired in the first two levels were transferable to subsequent tasks. For both models, level 9 marked a renewed increase in difficulty, and level 10 proved to be the most difficult.

In both configurations, the reward initially rose from strongly negative values to values close to zero or positive. Most of the sharp drops could be linked to a change in the curriculum level. Therefore, they did not automatically indicate PPO instability. There were also isolated exceptions. For example, a large drop in agent *PPO_GAIL_no9* occurred without a promotion. This can be described as a short-term deterioration in performance or an isolated fluctuation. Since the agent later recovered its reward and continued its progression, this was not a permanent breakdown in the learning process.

The entropy of both configurations exhibited a cyclical pattern, initially reaching high values, then decreasing as the task was mastered, and subsequently increasing upon reaching a new level. Later in the training, the entropy of most agents remained mainly between 3 and 4, and its fluctuations were smaller. This indicates a relative stabilization of the exploration level, subsequent updates no longer caused such large changes in the diversity of selected actions. The most notable case was the unsuccessful *PPO_baseline_no1*, whose

entropy dropped rapidly and remained significantly lower than that of the other agents for a long time. This may suggest premature convergence to an ineffective strategy.

The scope of the acquired behavior should be distinguished from mastery of the full Rummikub game. The curriculum examined selected tile-placement skills in a controlled, single-agent environment, with no opponents or turn-time limit. These skills included completing and independently creating runs and groups, selecting suitable placements for multiple tiles, identifying different arrangement types and making decisions when faced with additional arrangements on the board. Consequently, the results demonstrate the agents’ ability to acquire these particular rule-based placement skills, rather than a complete multiplayer Rummikub strategy.

The results suggest that acquiring a new skill depends not only on the number of required actions, but also on the type of change introduced to the task. When a level involved extending a previously learned behavior by requiring more tiles to be placed under similar conditions, the agents generally progressed quickly. This indicates that representations and action patterns acquired in earlier tasks can be transferred to more demanding variants. However, introducing qualitatively different decision problems, such as selecting actions in the presence of multiple arrangements, resulted in longer transition periods, even when the number of additional moves remained unchanged.

Therefore, when a curriculum introduces a substantial qualitative increase in difficulty, a temporary decrease in reward can be expected, alongside renewed growth in policy entropy and greater variation between training runs. This behavior does not necessarily indicate algorithmic instability. If it coincides with a curriculum transition and is followed by reward recovery and further progression, it is more accurately understood as a period of renewed exploration and adaptation. However, a sharp deterioration that occurs without a task transition, particularly if the policy does not subsequently recover, provides stronger evidence of unstable learning or convergence to an ineffective strategy.

The results also show that meeting a curriculum promotion threshold should be considered evidence of achieving the required level of average training performance, rather than full mastery of the relevant skill. Evaluation win rates below 100% demonstrate that a policy may satisfy the advancement condition while failing in a considerable proportion of independently generated episodes.

The PPO Baseline configuration appears to be more appropriate when the environment can provide informative intermediate rewards, when action masking can limit invalid decisions, and when the required behaviour can be divided into gradual curriculum stages. Under these conditions, independent exploration was sufficient for most agents to acquire the desired skills while avoiding the additional computational cost of imitation learning. Therefore, this configuration is preferable when training time and computational efficiency are the primary constraints.

PPO+GAIL may be more appropriate in situations where successful exploration is

challenging, representative expert demonstrations are available, or reducing the likelihood of unsuccessful training runs is more important than minimising computational cost. In the analysed sample, GAIL did not accelerate curriculum completion or produce a clear improvement in the evaluation results of successfully trained policies. However, all ten PPO+GAIL agents reached the target. This suggests that the auxiliary imitation signal may improve consistency in reaching the training objective, even when it does not improve sampling efficiency.

The effectiveness of GAIL is likely to depend on how closely the demonstrations correspond to the skills being trained. In this study, the demonstrations were recorded during full-game behaviour, whereas the training used isolated curriculum tasks. Therefore, the demonstrations may contain uncommon or unavailable states and actions in individual curriculum stages. Creating demonstrations specifically for the relevant skill categories could provide a more informative imitation signal. However, this possibility requires separate experimental verification.

Overall, PPO Baseline provided the more computationally efficient solution for learning the selected tile-placement skills, whereas PPO+GAIL provided greater observed reliability of reaching the curriculum target at the cost of substantially longer training. The results do not suggest that either configuration is universally superior. PPO Baseline is preferable when reward shaping, action masking and gradual curriculum progression already make exploration feasible. PPO+GAIL is more justified when appropriate demonstrations are available and the priority is to increase training consistency in a difficult exploration problem, provided that the additional computational cost is acceptable.

Chapter 5

Summary

This chapter summarizes the work performed and the experiments conducted, and outlines the potential future developments of the project.

5.1 Conclusions

The objective of this study was to apply artificial intelligence algorithms to train a computer player in the logical board game, which was Rummikub. The game environment was characterized by a highly complex action space, randomness, and long-range dependencies between decisions. The **Sparse Reward Problem** arose, which was mitigated by applying action masking, reward shaping, curriculum learning, and imitation learning. The goal of the experiment was to compare whether a model incorporating imitation learning would reach a specified curriculum level while requiring fewer environmental steps. However, the resulting policies do not represent a complete player capable of playing a full game of Rummikub, but agents trained to perform selected tile-placement tasks in a simplified single-agent environment without opponents, turn management, or long-term game strategy.

As part of this thesis, a game environment was designed in Unity with an integrated ML-Agents tool, a reward and punishment system, an expert demonstration, and the implementation of masks and a curriculum-based learning environment. The project also involved configuring models, training, and testing 20 independent agents within the environment.

The objective of this study was achieved within the scope of the designed experiment. The experiments conducted confirmed that gradually increasing difficulty and reward shaping enable reinforcement learning agents to master selected mechanisms of a complex logic game. The addition of GAIL did not shorten training or significantly improve the quality of the successfully trained policies, but in the sample studied, it was associated with greater reliability in achieving the objective.

5.2 Future development

The current experiments were concluded after completing Level 11 and reaching Level 12, whereas the full game setup includes a much larger number of stages. This would allow us to verify whether the agents retain their previously acquired skills and whether the learned policy remains stable after the introduction of new game mechanics. It would also allow us to analyze whether the differences between the baseline and GAIL models would become more pronounced as the difficulty level increases.

In the current experiments, GAIL used demonstrations recorded during a full game. The states and actions occurring in these demonstrations did not necessarily correspond to the simplified conditions of individual curriculum levels. Preparing separate demonstrations for levels or groups of similar levels could provide the agent with a more appropriate imitation signal. This would allow us to verify whether the lack of overall acceleration in PPO+GAIL resulted from the limited usefulness of demonstrations derived from a full game.

The results were obtained for a single selected configuration of PPO and GAIL. The lack of a clear advantage for GAIL in terms of the number of steps does not, therefore, mean that the method could not achieve better results with a different set of parameters. In future experiments, it would be worthwhile to test the GAIL signal strength, the CL promotion threshold, the minimum lesson length, and the values of penalties and rewards in the environment.

Another direction is to expand the experiments to include other reinforcement learning algorithms and methods that support exploration. It would be worth testing whether the SAC algorithm, as an off-policy method, performed better in this environment. Integrating auxiliary methods such as **Curiosity**, **RND**, or **HRL** could also reveal whether their inclusion affects the computer player’s performance in the environment.

Bibliography

- [1] Omur Aydogmus and Musa Yilmaz. ‘Comparative Analysis of Reinforcement Learning Algorithms for Bipedal Robot Locomotion’. In: *IEEE Access* 12 (2024), pp. 7490–7499. DOI: 10.1109/ACCESS.2023.3344393.
- [2] Pablo Barros, Anne C. Bloem, Inge M. Hootsmans, Lena M. Opheij, Romain H. A. Toebosch, Emilia Barakova and Alessandra Sciutti. ‘You Were Always on My Mind: Introducing Chef’s Hat and COPPER for Personalized Reinforcement Learning’. eng. In: *Frontiers in robotics and AI* 8 (2021), pp. 669990–. ISSN: 2296-9144.
- [3] ‘Board Games’. eng. In: *The Biblical archaeology review* 49.1 (2023), pp. 26–. ISSN: 0098-9444.
- [4] Andrew Cohen, Ervin Teng, Vincent-Pierre Berges, Ruo-Ping Dong, Hunter Henry, Marwan Mattar, Alexander Zook and Sujoy Ganguly. ‘On the Use and Misuse of Absorbing States in Multi-agent Reinforcement Learning’. In: *RL in Games Workshop AAAI 2022* (2022). URL: <https://arxiv.org/abs/2111.05992>.
- [5] Stockfish Developers. *Stockfish - Open Source Chess Engine*. 2024. URL: <https://github.com/official-stockfish/Stockfish> (visited on 22/12/2024).
- [6] Zihan Ding. ‘Imitation Learning’. In: *Deep Reinforcement Learning*. Ed. by Hao Dong, Zihan Ding and Guangxu Zhao. Singapore: Springer Nature Singapore Pte Ltd., 2020. Chap. 8, pp. 273–306. DOI: 10.1007/978-981-15-4095-0_8. URL: https://doi.org/10.1007/978-981-15-4095-0_8.
- [7] Adrian Goldwaser and Michael Thielscher. ‘Deep Reinforcement Learning for General Game Playing’. eng. In: *Proceedings of the ... AAAI Conference on Artificial Intelligence* 34.2 (2020), pp. 1701–1708. ISSN: 2159-5399.
- [8] Ian Goodfellow, Yoshua Bengio and Aaron Courville. *Deep Learning*. <http://www.deeplearningbook.org>. MIT Press, 2016.
- [9] Nessrine Hammami and Kim Khoa Nguyen. ‘On-Policy vs. Off-Policy Deep Reinforcement Learning for Resource Allocation in Open Radio Access Network’. In: *2022 IEEE Wireless Communications and Networking Conference (WCNC)*. 2022, pp. 1461–1466. DOI: 10.1109/WCNC51071.2022.9771605.

- [10] Shuai Han, Mehdi Dastani and Shihan Wang. ‘Neuro-symbolic Action Masking for Deep Reinforcement Learning’. In: (Feb. 2026). DOI: 10.48550/arXiv.2602.10598.
- [11] Hyo-Seok Hwang, Yoojoong Kim and Junhee Seok. ‘Generative Adversarial Soft Actor-Critic’. eng. In: *IEEE transaction on neural networks and learning systems* 36.7 (2025), pp. 11917–11927. ISSN: 2162-237X. DOI: 10.1109/TNNLS.2024.3493113.
- [12] Claudia Jipa, Carmen Berce and Mirela Lăcrimioara Cosma. ‘The impact of using board games on the development of situational motivation in primary school children’. In: *Journal Plus Education* 33 (Aug. 2023), pp. 10–27. DOI: 10.24250/jpe/SI/2023/CJ/CAB/MC/.
- [13] Pascal Klink, Carlo D’Eramo, Jan Peters and Joni Pajarinen. ‘On the Benefit of Optimal Transport for Curriculum Reinforcement Learning’. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 46.11 (Nov. 2024), 7191–7204. ISSN: 1939-3539. DOI: 10.1109/tpami.2024.3390051. URL: <http://dx.doi.org/10.1109/TPAMI.2024.3390051>.
- [14] Marek Kopel and Adam Pocijowski. ‘Applying ML Algorithms to Video Game AI’. In: *Intelligent Information and Database Systems*. Vol. 11683. Lecture Notes in Artificial Intelligence. Springer Nature Switzerland AG, 2019, pp. 353–362. DOI: 10.1007/978-3-030-28377-3_29. URL: https://doi.org/10.1007/978-3-030-28377-3_29.
- [15] Mihael Lukas, Igor Tomicic and Andrija Bernik. ‘Anticheat System Based on Reinforcement Learning Agents in Unity’. In: *Information* 13.4 (2022), p. 173. DOI: 10.3390/info13040173. URL: <https://doi.org/10.3390/info13040173>.
- [16] Runzhou Luo, Boning Yao and Qingwen Zhang. ‘Research on Board Game Strategy Methods Based on Reinforcement Learning’. eng. In: *Applied and Computational Engineering* 174.1 (2025), pp. 32–38. ISSN: 2755-2721.
- [17] Fanxiao Meng. ‘Research on Multi-agent Sparse Reward Problem’. In: *Highlights in Science, Engineering and Technology* 85 (Mar. 2024), pp. 96–103. DOI: 10.54097/er0mx710.
- [18] M&M Ventures (2014) Limited. *Rummikub Rules (English)*. 2019. URL: <https://rummikub.com/wp-content/uploads/2019/12/2600-English-1.pdf> (visited on 24/03/2026).
- [19] Thomas M. Moerland, Joost Broekens, Aske Plaat and Catholijn M. Jonker. *Model-based Reinforcement Learning: A Survey*. 2022. arXiv: 2006.16712 [cs.LG]. URL: <https://arxiv.org/abs/2006.16712>.
- [20] Muddasar Naeem, Syed Rizvi and Antonio Coronato. ‘A Gentle Introduction to Reinforcement Learning and its Application in Different Fields’. In: *IEEE Access* 8 (Jan. 2020), pp. 209320–209344. DOI: 10.1109/ACCESS.2020.3038605.

-
- [21] Sao Mai Nguyen. *The intrinsic motivation of reinforcement and imitation learning for sequential tasks*. Dec. 2024. DOI: 10.48550/arXiv.2412.20573.
 - [22] Diego Perez-Liebana, Jialin Liu, Ahmed Khalifa, Raluca D. Gaina, Julian Togelius and Simon M. Lucas. ‘General Video Game AI: A Multitrack Framework for Evaluating Agents, Games, and Content Generation Algorithms’. In: *IEEE Transactions on Games* 11.3 (2019), pp. 195–214. DOI: 10.1109/TG.2019.2901021. URL: <https://doi.org/10.1109/TG.2019.2901021>.
 - [23] Evan Prianto, MyeongSeop Kim, Jae-Han Park, Ji-Hun Bae and Jung-Su Kim. ‘Path Planning for Multi-Arm Manipulators Using Deep Reinforcement Learning: Soft Actor–Critic with Hindsight Experience Replay’. eng. In: *Sensors (Basel, Switzerland)* 20.20 (2020), pp. 5911–23. ISSN: 1424-8220.
 - [24] Radisa H. Rachmadi, Rainamira Azzahra, Rayhan A. Darmawan, Patrick A. Nigo and Nunung N. Qomariyah. ‘Developing AI Bots with Minimax Algorithm for Surakarta Board Game’. In: *2021 International Conference on ICT for Smart Society (ICISS)*. 2021, pp. 1–6. DOI: 10.1109/ICISS53185.2021.9533206.
 - [25] David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dhharshan Kumaran, Thore Graepel, Timothy Lillicrap, Karen Simonyan and Demis Hassabis. ‘A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play’. eng. In: *Science (American Association for the Advancement of Science)* 362.6419 (2018), pp. 1140–1144. ISSN: 0036-8075.
 - [26] Yuhan Song. ‘A Survey on Hindsight Experience Replay for Sparse Reward Problems’. In: *Applied and Computational Engineering* 240 (May 2026), pp. 72–77. DOI: 10.54254/2755-2721/2026.GU33500.
 - [27] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. 2nd. Adaptive Computation and Machine Learning series. <http://www.incompleteideas.net/book/the-book-2nd.html>. Cambridge, Massachusetts: The MIT Press, 2018. ISBN: 9780262039246.
 - [28] Richard S Sutton, David McAllester, Satinder Singh and Yishay Mansour. ‘Policy Gradient Methods for Reinforcement Learning with Function Approximation’. In: *Advances in Neural Information Processing Systems*. Ed. by S. Solla, T. Leen and K. Müller. Vol. 12. MIT Press, 1999. URL: https://proceedings.neurips.cc/paper_files/paper/1999/file/464d828b85b0bed98e80ade0a5c43b0f-Paper.pdf.
 - [29] Alan M. Turing. ‘Computing Machinery and Intelligence’. In: *Mind* 59.236 (1950), pp. 433–460.

- [30] Muhtar Çağkan Uludağlı and Kaya Oğuz. ‘Non-player character decision-making in computer games’. In: *Artificial Intelligence Review* 56 (2023), pp. 14159–14191. DOI: 10.1007/s10462-023-10491-7. URL: <https://doi.org/10.1007/s10462-023-10491-7>.
- [31] Unity Technologies. *Unity ML-Agents Toolkit: Learning Environment Design - Agents*. Online access for com.unity.ml-agents@4.0. 2024. URL: <https://docs.unity3d.com/Packages/com.unity.ml-agents@4.0/manual/Learning-Environment-Design-Agents.html>.
- [32] Unity Technologies. *Unity ML-Agents Toolkit: Overview*. Online access for com.unity.ml-agents@4.0. 2024. URL: https://github.com/Unity-Technologies/ml-agents/blob/release_21/docs/ML-Agents-Overview.md.
- [33] Unity Technologies. *Unity ML-Agents Toolkit: Training Configuration*. Online access for com.unity.ml-agents@4.0. 2024. URL: https://github.com/Unity-Technologies/ml-agents/blob/release_21_docs/docs/Training-ML-Agents.md#training-configurations.
- [34] Unity Technologies. *Unity ML-Agents Toolkit: Training Configuration File Description*. Online access for com.unity.ml-agents@4.0. 2024. URL: https://github.com/Unity-Technologies/ml-agents/blob/release_21_docs/docs/Training-Configuration-File.md.
- [35] Ruozhang Xi, Yao Ni and Wangyu Wu. ‘Information-Theoretic Intrinsic Motivation for Reinforcement Learning in Combinatorial Routing’. In: *Entropy* 28 (Jan. 2026), p. 140. DOI: 10.3390/e28020140.
- [36] Her-Terng Yau, Ping-Huan Kuo, Po-Chien Luan and Yung-Ruen Tseng. ‘Proximal policy optimization-based controller for chaotic systems’. eng. In: *International journal of robust and nonlinear control* 34.1 (2024), pp. 586–601. ISSN: 1049-8923.

Appendices

List of abbreviations and symbols

AI Artificial Intelligence

NPC Non-Player Characters

RL Reinforcement Learning

NN Neural Network

ANN Artificial Neural Network

DNN Deep Neural Network

FSM Finite State Machine

BT Behavior Tree

RBS Rule-Base System

MCTS Monte Carlo Tree Search

MLP Multilayer Perceptron

ReLu Rectified Linear Units

$\tanh$ Hyperbolic Tangent

MSE Mean Squared Error

SGD Stochastic Gradient Descent

MDP Markov Decision Process

MC Monte Carlo

TP Temporal Difference

MFRL Model-Free Reinforcement Learning

MBRL Model-Based Reinforcement Learning

DRL Deep Reinforcement Learning

A3C Asynchronous Advantage Actor-Critic

A2C Advantage Actor-Critic

COPPER COnPetitive Prioritized Experience Replay

ACER Actor-Critic with Experience Replay

PER Prioritized Experience Replay

IM Intrinsic Motivation

ICM Intrinsic Curiosity Modules

RND Random Network Distillation

IL Imitation Learning

IfO Imitation from Observation

IRL Inverse Reinforcement Learning

BC Behavioral Cloning

GAIL Generative Adversarial Imitation Learning

HRL Hierarchical Reinforcement Learning

CL Curriculum Learning

HER Hindsight Experience Replay

ML-Agents The Machine Learning Agents Toolkit

SDK Software Development Kit

protobuf protocol buffer

YAML YAML Ain't Markup Language

GAE Generalized Advantage Estimate

List of additional files in electronic submission

Additional files uploaded to the system include:

- source code of the application
- yaml file configuration
- final models
- demonstration file
- TensorBoard event files

List of Figures

2.1	The graph of feedforward neural network	14
2.2	The graph of single perceptron	15
2.3	ReLU plot	16
2.4	logistic sigmoid plot	17
2.5	Tanh plot	17
2.6	Reinforcement learning diagram	23
3.1	The image of behavior parameters of the agent in the Unity	51
4.1	Cumulative reward graph of 10 baseline agents	73
4.2	Cumulative reward mean and SD graph of 10 baseline agents	74
4.3	Curriculum progression of individual 10 baseline agents	75
4.4	Curriculum progression mean and SD of individual 10 baseline agents . . .	76
4.5	Entropy chart of individual 10 baseline agents	77
4.6	Entropy mean and SD chart of individual 10 baseline agents	77
4.7	Cumulative reward graph of 10 GAIL agents	83
4.8	Cumulative reward graph of 10 GAIL agents	84
4.9	lesson progress graph of 10 GAIL agents	84
4.10	lesson progress mean and SD graph of 10 GAIL agents	86
4.11	Entropy chart of individual 10 gail agents	87
4.12	Entropy chart of mean and SD of individual 10 gail agents	88

List of Tables

3.1	Stages of the implemented Curriculum Learning environment.	61
4.1	PPO hyperparameters and neural network architecture used in all training configurations.	71
4.2	Hyperparameters of the auxiliary learning techniques.	72
4.3	First recorded training steps at which individual baseline agents completed the level.	78
4.4	Training outcome of individual PPO baseline agents.	80
4.5	Evaluation results of PPO baseline agents over 1000 episodes on curriculum level 11.	81
4.6	First recorded training steps at which individual GAIL agents completed the level.	88
4.7	Training outcome of individual PPO GAIL agents.	90
4.8	Evaluation results of PPO GAIL agents over 1000 episodes on curriculum level 11.	91
4.9	Comparison of curriculum-level completion and transition times for PPO baseline and PPO+GAIL agents.	92
4.10	Comparison of curriculum training outcomes.	94